






中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

学　　校	长春理工大学
参赛队号	No. 25101860078
队员姓名	1.陈星桦
	2.邵麟涵
	3.涂汇融

中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

题　　目　高速列车轴承智能故障诊断

摘　　要

近年来，高速列车因其快速便捷性，常常被作为人们出行的首要选择。轴承作为高速列车运行系统的关键部件，其故障发生频率关系着整个高速列车系统的运转效率。然而，由于轴承长期处于高转速等复杂工况环境中，其故障率往往很高，因此，对轴承故障率的实时监测与诊断就变得尤为重要。本文以高速列车轴承智能故障诊断为核心，构建了 SVM、XGBoost 以及随机森林模型进行源域故障诊断，构建了 TCA-XGBoost、DANN 模型进行迁移诊断，最终得出了具有高落地性的预测结果并给出了迁移过程的可解释性分析。

针对问题一，将 161 个台架实验数据的采样率与目标域实际列车数据采样率一样都**统一到 32kHz**，对 48kHz 做抗混叠低通和降采样，对 12kHz 做重采样。本文优先选择最接近轴承，故障特征最明显的的驱动端（DE）信号，风扇端（FE）、基座（BA）作为辅助验证。特征从时域（**均值、RMS、峭度、峰值因子、脉冲因子**）、频域（**频谱能量分布、频谱质心、均方频率**）、时频域（**小波包能量熵、希尔伯特包络谱特征、STFT 时频图、二维图像特征**）三个角度提取共 **12 个特征**。提取上述特征得到特征向量并形成数据集（**特征向量矩阵拼接故障标签向量**）。最后可视化处理，做出时域波形、频谱的对比，以及 t-SNE 验证提取的特征能够区分不同类别。

针对问题二，本文首先按文件对源域数据集进行**分层抽样**，以 **7:3** 的比例划分为训练集和测试集，保证两个数据中包括全部种类的故障类型。其中正常工作状态（N）的训练集样本过少，采用**切片重叠采样**，将其样本量扩充至 28 个（与内圈故障与滚动体故障样本数量相同）。然后在训练集和测试集上分别提取第一问中的特征形成特征矩阵，并将训练集的特征矩阵进行标准化，并将其标准化所用参数运用在测试集的标准化中，得到标准化的训练集与测试集特征矩阵。接下来构建 **XGBoost、SVM 以及随机森林模型**，并将特征矩阵分别输入其中对模型进行训练以及优化，最后构建评价指标发现，SVM 与随机森林的预测效果最好，其中 SVM 的 **F1-score 为 0.8609205**，随机森林的 **F1-score 为 0.8590909**，体现了较好的预测性能。

针对问题三，首先在目标域文件数据中提取出与第二问相同的特征矩阵，并用第二问中源域训练集标准化的参数将目标域数据集提取的特征矩阵进行标准化。本题先将第二问中源域训练集和测试集的特征矩阵与故障类型拼接，将

整个源域数据看作接下来输入模型的训练集，将目标域特征矩阵看成测试集，对其故障类型进行分类。接下来构建无监督的 **DANN 域对抗神经网络**，输入训练集进行训练与优化，后给出 16 个文件的故障类型标签，发现 MMD 值随着训练进行，快速且平稳地下降最终保持稳定在 **0.112** 附近，但目标域故障类型预测的最大置信度为 **0.848**。除此之外，本文也构建了 **TCA-XGBoost 模型**，并进行训练和优化，发现其预测故障类型的最大置信度为 **0.800**。但整体来看，TCA-XGBoost 模型的预测效果更好一些。最后进行可视化分析，对比迁移前后的 t-SNE，发现源域与目标域聚类更加重叠。

针对问题四，以问题三为导向，围绕缓解迁移学习中的“黑箱缺陷”问题，以两个模型为基点，实现事前、迁移过程、事后的可解释性分析。首先，明确可解释性的目标与受众，然后以 TCA-XGBoost 为主要模型训练出的数据作为主要案例，从特征分布轨迹、域差异量化、代表样本跟踪、判别器日志和影响力五大关键实时监控数据出发，着重分析了迁移过程的可解释性，最终从“不可见”的“黑箱缺陷”实现迁移路径的可视化。

关键字：故障诊断、轴承故障、迁移学习、SVM、TCA-XGBoost

目录

目录 3
1 问题重述 5
1.1 研究背景 5
1.2 问题重述 6
1.2.1 数据分析与故障特征提取 6
1.2.2 源域故障诊断 6
1.2.3 迁移诊断 6
1.2.4 迁移诊断的可解释性 6
2 模型假设与符号说明 7
2.1 符号说明 7
3 问题一的分析与求解 8
3.1 问题一的分析 8
3.2 数据的预处理工作 8
3.3 特征分析与特征提取 9
3.3.1 故障机理分析 9
3.3.2 特征分析 13
3.3.3 特征提取 18
3.4 本章小结 20
4 问题二的分析与求解 20
4.1 问题二的分析 20
4.1.1 数据准备与整合 21
4.1.2 变量准备 22
4.1.3 数据描述与背景分析 24
4.1.4 建立 SVM、XGBoost、随机森林并优化 25
4.1.5 建模结果 28
4.2 本章小结 34
5 问题三的分析与求解 34
5.1 问题三的分析 34
5.2 问题求解 35
5.2.1 数据准备与特征工程 35
5.2.2 建立 DANN、TCA+XGBoost 模型 36
5.2.3 输出每个样本的类别预测结果及置信度； 40

5.2.4 建模结果.....40
5.3 本章小结.....50
6 问题四的分析与求解.....51
6.1 问题描述.....51
6.2 问题四的求解.....51
6.2.1 迁移过程的可解释性.....52
7 未来展望.....55
参考文献.....56
附录.....57

1 问题重述

1.1 研究背景

轴承是一个工业的基石，其技术含量和寿命会影响整个机械系统的性能、运转效率和可靠性。对于机械系统来说，轴承最重要的功能就是减少摩擦，通过以滚代滑的核心思想，配合其完美的几何结构设计和表面光洁度，以及高效的润滑水平，将有害摩擦转化为可控高效的运动，大大延长了整个机械系统的寿命。轴承在保证旋转精度的同时还需要承受来自零部件的径向力、轴向力，用以支撑旋转部件。轴承最常见的主要分为两大类：滚动轴承和滑动轴承。其中，滚动轴承包括深沟球轴承、角接触轴承、圆柱滚子轴承、圆锥滚子轴承、推力球轴承、滚针轴承；滑动轴承包括流体动压轴承、流体静压轴承。通常情况下，会根据载荷类型和大小、转速、精度、工况环境、安装空间、成本与维护等条件来选择合适的轴承。例如中国现代高铁列车，基于对轴承高刚度与稳定性、便于预紧和调整、维护方便、能承受复合载荷等的性能需求，普遍采用圆锥滚子轴承。

近年来，中国高铁发展势头迅猛，逐渐成为出行的首要选择。高铁作为大型的轨道运输工具，组件数量庞大，各种零部件间紧密相联，而轴承被誉为“高铁的关节”，其性能以及寿命对高铁起着举足轻重的作用。然而，由于高铁工作条件往往非常恶劣，轴承磨损与消耗巨大，如果未能及时发现轴承故障，将会加速轴承性能的退化，最终导致元件失效，造成不可逆的经济损失，甚至威胁到人民的人身安全。因此对轴承健康状态的在线监测以及故障状态的诊断就变得尤为重要。传统的故障诊断方法需要机器停机时进行故障检验，耗时耗力。随着大数据、人工智能的快速发展，数据驱动的智能故障诊断方法在列车装备运维领域受到了广泛关注。

然而，在实际应用场景中，依赖于大量运营数据所构建的深度学习模型虽然能够拥有更高精度的故障识别能力、更强的工况适应能力、更高效的故障诊断能力，受现场的运行环境和多变工况的影响，以及在途列车的真实故障数据的稀缺问题，进而造成训练数据分布严重失衡，极大地限制了深度学习模型向工程实践的转化，制约了智能化故障诊断的发展。与实际运行数据相对的是，在台架实验环境下采集的轴承数据不仅数量丰富、标签完备，而且其故障演化机理也与在途列车轴承相似。

在这样的背景下，本文研究的核心目标是通过迁移学习技术，结合数学建模的方式，从样本数据与标签相对丰富的领域内学习域不变特征知识，增强网络模型的泛化能力，以提升目标任务的模型性能。通过建立数学模型，解析台架实验

环境下采集的轴承数据，提取代表性的故障特征，构建源域模型，再结合迁移学习技术，将诊断知识迁移至实际运营列车数据，有效缓解样本不平衡问题，解决高速列车轴承智能故障诊断问题。最终，这些研究成果将为高速列车轴承诊断智能化发展提供科学依据和决策支持。

1.2 问题重述

本研究赛题一共涉及四个子问题，均依赖于源域数据和数学建模技术：

1.2.1 数据分析与故障特征提取

问题一要求基于高速列车轴承故障机理，在保留机理特征的前提下，将所给的源域数据集按照目标域数据集的标准，进行数据筛选、特征分析、特征提取。本问题的目标是将源域与目标域最大相似化，为后续的迁移学习做准备

1.2.2 源域故障诊断

问题二要求在任务一提取的时域、频域及包络谱等多维故障特征基础上，合理划分源域数据集为训练集与测试集，设计并实现高效的智能诊断模型，完成对设备健康状态的精准分类。本问题的核心目标是：构建能够准确捕捉故障特征与设备状态之间映射关系的分类器，通过支持向量机（SVM）、XGBoost、随机森林等机器学习算法对比分析，筛选最优模型，并利用准确率、精确率、召回率、F1 分数及混淆矩阵等指标全面评估其诊断性能，最终建立一个高精度、强鲁棒性的源域故障诊断基准模型

1.2.3 迁移诊断

问题三在问题二所构建的源域诊断模型基础上，要求充分考虑源域与目标域在数据分布上的共性与差异，设计合理的迁移学习策略，实现知识从有标签源域向无标签目标域的有效迁移。核心目标包括：构建面向目标域的诊断模型，对未知标签的测试样本进行准确分类与标定；采用 DANN（域对抗神经网络）或 TCA+XGBoost 等迁移方法，缓解因传感器差异、运行环境变化等因素引起的域间分布偏移；通过 t-SNE 可视化、域距离度量等方式展示迁移前后特征空间的演化过程，并对迁移效果进行定量评估与定性分析，最终输出各目标域样本的预测类别及其置信度，为工业设备的跨工况智能运维提供可解释、可推广的技术支持

1.2.4 迁移诊断的可解释性

问题四要求基于问题三中模型的设计、迁移、决策，再结合列车轴承故障机理，对迁移学习的事前/过程/事后进行可解释性分析，可视化和量化迁移诊断过程，以便提高模型的可信度以及可理解性，进而提高实际操作的落地性。

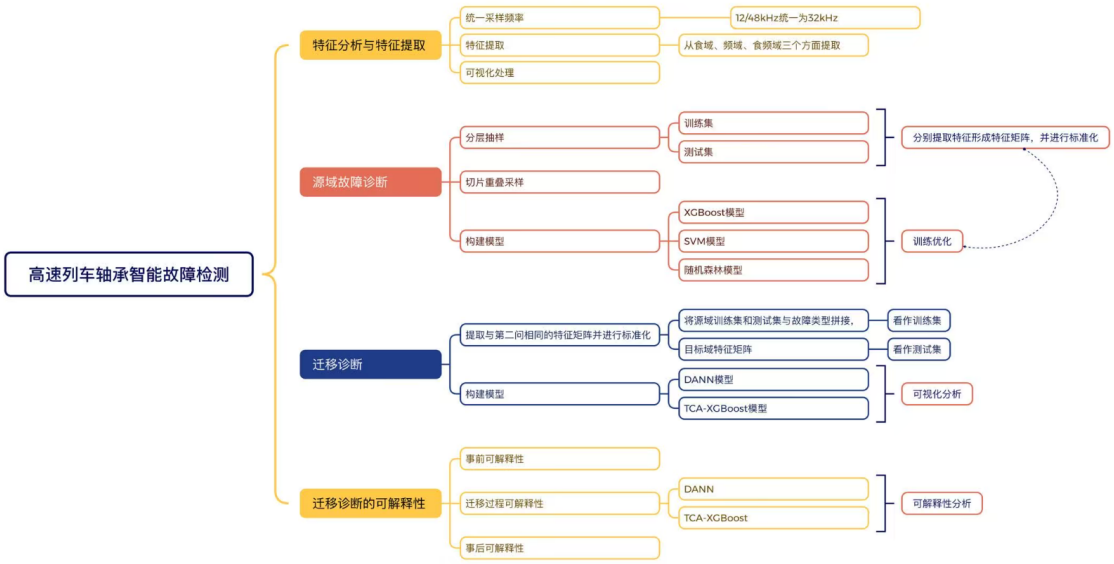


图 1- 1 全文流程图

2 模型假设与符号说明

2.1 符号说明

表2-1 符号说明

符号	意义
w	超平面的法向量
b	偏置项
$\phi(x_i)$	将样本 x_i 映射到高维特征空间的函数
C	惩罚参数
K	核矩阵
L	域差异矩阵
ξ_i	松弛变量
l	损失函数
Ω	正则化项
$f_\theta(x)$	由参数 θ 控制的特征提取器
$g_\phi(f_\theta(x))$	标签分类器
$d_\psi(f_\theta(x))$	域判别器

3 问题一的分析与求解

3.1 问题一的分析

对于问题一，核心任务是从源域数据中筛选有代表性的数据进行特征分析和特征提取。给定的数据集包括目标域数据集和源域数据集，其中，目标域数据集包含了 16 个振动频率为 32kHz 的实际列车数据；源域数据集包含了 161 个台架实验数据，均为.mat 文件。其中，采样频率分别为 12kHz 和 48kHz、转速约为 1750rpm。任务一的主要任务包括以下三个方面：

（1）对源域数据进行有效的数据预处理工作，以便最大化源域与目标域的“相关性”和“可迁移性”。任务一所用的数据集包括 12/48kHZ 的 DE、12KHz 的 FE，48kHz 的 Normal。在本部分中分别使用 matlab、python 等软件对数据进行筛选、提取、处理等工作，通过优先级筛选、下采样频率等方式，选择与目标域故障类型相匹配的数据，以助力后续任务的开展。

（2）在源域数据集中包含了外圈故障（OR）、内圈故障（IR）、滚动体故障（B）、正常状态（N），所以我们围绕这四个故障类型在信号中的故障表现——特定频率成分和时域冲击特征的结合，分析其故障机理，为之后的特征提取提供指引。

（3）以高速列车轴承的故障机理特征为导向，对筛选后的源域数据进行特征提取，在保留相关特征的前提下，把信号按短窗分段提取，分别从时域、频域、时频域提取具有代表性的特征，为后续源域模型训练和迁移学习提供稳定、可解释且便于跨域对齐的输入。

3.2 数据的预处理工作

迁移学习的前提是源域与目标域最大化相似。而源域数据集中所包括的数据无论是采样频率还是转速都与目标域大相径庭，因此，在执行任务前，需要对源域数据进行筛选等数据预处理工作，构建一个具有代表性，能迁移到目标域的训练集。

（1）首先，根据迁移目标特征，选择和目标相似的源样本子集。理论上优先选取转速靠近 600rpm 的台架样本，保证四种列车轴承故障类型都能覆盖。虽然我们不知道列车轴承在行驶过程中所承受的负荷的具体数值，但这个载荷在运行过程中基本是固定的。列车在高载荷下运行时，轴承和相关部件的响应会变得复杂，而非简单线性。同时，由于高载荷会引起更大的振动响应，导致载荷与振动差异增大，进而影响整个系统的稳定性。所以为了避免因为高载荷引入的较强的非线性响应以及过大的载荷差异，筛选数据时优先考虑低载荷下的源子集。在源域数

据中根据变量名或文件名进行筛选，选出例如 0 马力或者 1 马力这样低载荷的数据，用于学习工况不变与工况依赖的特征。

（2）其次，在传感器的位置选择上，由于驱动端（DE）通常靠近轴承，传感器到故障源的机械传递路径最短，机械冲击在到达传感器前衰减最少，所造成的传递损失最小，故障产生的高频瞬态在驱动端（DE）上更明显，背景噪声相对较小，所以特征最显著。尤其是轴承初期缺陷、高频瞬态特征，驱动端（DE）能更早、更可靠地捕捉到。而风扇端（FE）一般不是直接靠近驱动轴承，信号要通过轴、壳体、联轴器等结构传播，在这过程中会衰减高频分量并引入频率选择性，同时可能会出现调制。对于高铁轴承智能化监测问题，目标是轴承状态监测以及监测早期微小缺陷，相较之下，优先使用驱动端（DE）轴向或径向加速度传感器，把风扇端（FE）作为第二通道，用于验证模型的鲁棒性。

（3）再者，为了最终迁移学习的顺利进行，由于目标域所有样本的采样频率为 32kHz，源域样本的采样频率有 12kHz 和 48kHz 两种，故本文选择直接将源域所有样本的采样频率统一到 32kHz，对 48kHz 的样本做抗混叠低通加上降采样，对 12kHz 的样本做重采样，这样不会产生新高频信息。利用 python 对整个源域数据集进行初步的处理和预览，遍历整个文件读取所有的 DE 和 FE 数据。因为目标域中包含外圈(OR)、内圈(IR)、滚动体(B)故障和正常(N)状态这四种标签数据，所以必须确保所筛选数据中也包含上述四种标签数据，才能保证源域数据与目标域数据故障类型相匹配，使迁移时源域与目标域数据共性更多，以助力之后的迁移学习。

3.3 特征分析与特征提取

3.3.1 故障机理分析

在进行特征提取之前，我们需要先分析列车轴承故障机理，理解故障机理，明确特征与机理的对应关系，才能有针对性地选取特征、解释结果、保证模型稳健性，提高模型的可信度与可解释性。

高速列车轴承常因为转速高，滚动体与滚道接触频率大产生周期性冲击，承受列车重量与动态冲击载荷过大以及信号的非平稳性等，产生各种故障。常见的故障主要有：外圈故障（OR）、内圈故障（IR）、滚动体故障（BF）、保持架故障（TF）。如表，外圈故障，缺陷位置位于外圈滚道且始终在空间固定，每次滚动体经过缺陷时产生冲击，在时域表现为规律性脉冲，在频域表现为包络谱在 BPFO 处出现明显峰值及倍频；内圈故障，缺陷位于内圈滚道且在空间中不断移动，随内圈旋转，冲击周期随转速变化，并有调制效应；滚动体故障，缺陷在滚动体表面，接触点随时间在不同位置出现，脉冲信号不如外圈故障和内圈故障规则，呈

现不均匀冲击，在不同时间尺度上能量分布不均；保持架故障，常常会受到一定的磨损，但转速远低于内/外圈故障，由于是低频冲击，往往难以直接检测。

表 1- 1 故障类型

外圈 OR	外圈固定	周期固定 稳定	BPFO	谱峰清晰，倍 频明显	容易检测
内圈 IR	内圈旋转	周期随转速变 化 受调制	BPFI	有转频边带	中等
滚动体 BF	滚动体表面	冲击不规则 非平稳	BSF	调制强，倍频 +边带显著	较难
保持架 TF	保持架	低频冲击	FTF	低频谱峰，常 被噪声淹没	最难

在源域数据集中包含内圈故障数据、外圈故障数据、滚动体故障数据、正常数据。在时域波形图表中，内圈故障数据冲击调制明显，幅值不均匀；外圈故障数据周期性冲击，幅值较大；滚动体故障数据随机冲击，伴随调制；正常数据则是波动平稳，无明显冲击。这些故障在时域的信号表现为峭度升高、脉冲因子增加、峰值因子升高，RMS 随故障严重程度升高。在频域频谱图中，内圈故障数据在 BPFI 附近出现谱峰；外圈故障数据在 BPFO 附近出现谱峰；滚动体故障数据在 BSF 及其倍频附近出现峰值；正常数据则是主频集中在基频与低频谐波。而在小波能量分布图中，故障数据通常表现为能量在中高频分解层增加，越严重能量分布越分散；正常数据能量则是集中在低频层。一般情况下，这些数据在时频域的信号表现为瞬态高频分量，会在小波系数能量中被放大。

以如下内圈故障（IR）样本的多视角诊断图为例。

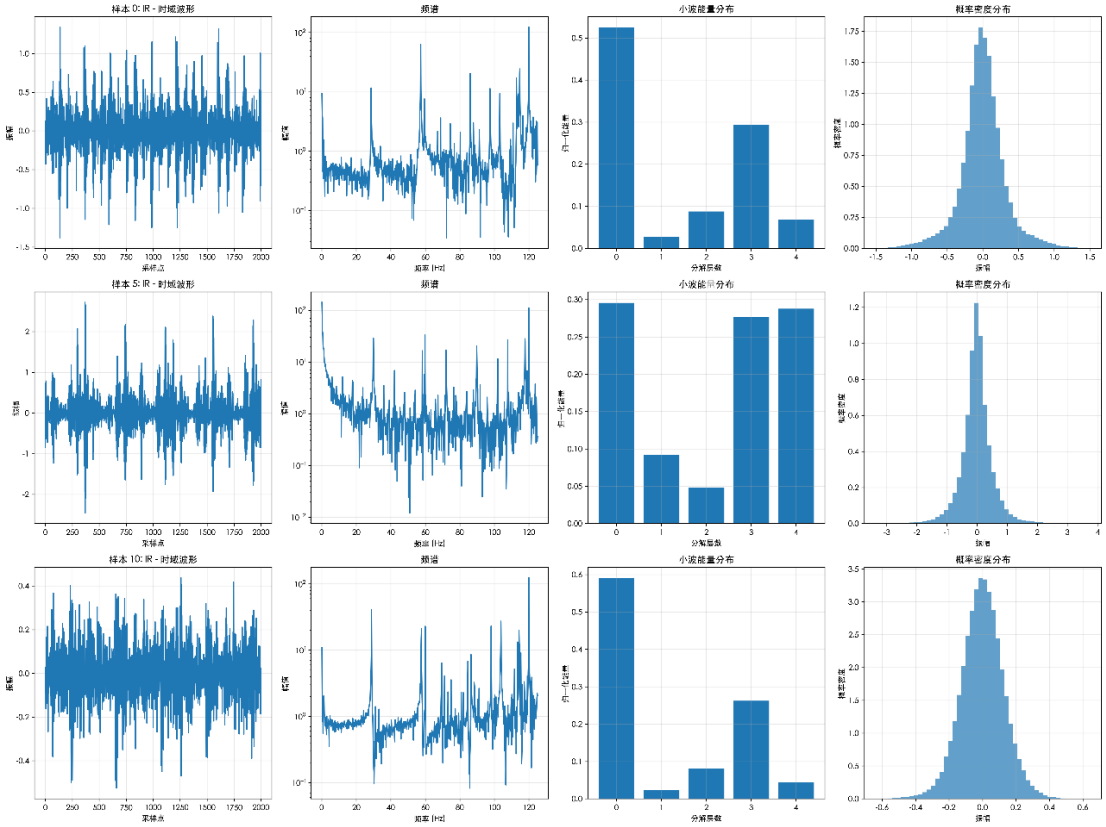


图 1- 2 样本多视角诊断图

上图是未经过筛选处理数据所画出的内圈故障诊断图，在图中，每一行是一段原始信号，从上到下依次是样本 0、样本 5、样本 10，每行从左到右依次显示时域波形、频谱、小波能量分布、振幅的概率密度。

据上图，可以看出，样本 0 峭度大于 3，说明有脉冲，但冲击能量不稳定，包络谱峰值比约等于 1.8，表明故障信号存在但不突出；样本 5 峭度接近 10，远高于正常信号，RMS 和峰值因子大幅上升，说明冲击很强，包络谱在 BPFO 频率处有显著峰，大于 4.5 倍噪声底；样本 10 RMS 和峰值因子很低，幅值分布接近高斯，小波能量大部分集中在低频，冲击高频成分被弱化，包络谱接近噪声水平。样本 5 在时域波形图中有明显的周期性冲击群，与样本 0 相比，脉冲更尖、更强，出现更为规律，而样本 10 则是整体振幅较小，脉冲不明显，波形看起来更接近平稳；在频谱图中，样本 5 有多个谐波，幅值显著高于周围谱基线，且跨较宽频带都有响应，而在样本 10 中虽然仍可见一些峰，但幅值不高，谱线不如样本 5 突出，在中低频处有轻微能量隆起；在小波能量分布图中，样本 5 能量在若干尺度上分布较均匀，但低层与高层都比较突出，说明冲击同时包含高频和中低频成分，而在样本 10 中，能量更集中在最低一层，高层能量很少，说明冲击能量被弱化或被噪声掩盖；在幅值分布图中，样本 5 大幅值频次显著，直方图呈明显的宽分布与厚尾，对应高峭度、大峰值因子的可能性高，样本 10 直方图较窄、近似高斯，厚尾不显著，说明脉冲事件少或幅度较小。由此可得，部分内圈

故障样本，如样本 5，呈现典型的周期性冲击与清晰的包络谱峰，易于检测；部分样本，如样本 10 信号弱，或被噪声掩盖，需要用包络、小波等时频增强手段与段级聚合来提升检测鲁棒性。根据这个机理特征，我们优先选择包络谱峰值、小波能量、峭度、频率方差作为主要判别特征。

再以不同故障类型的小波能量分布热力图为例。

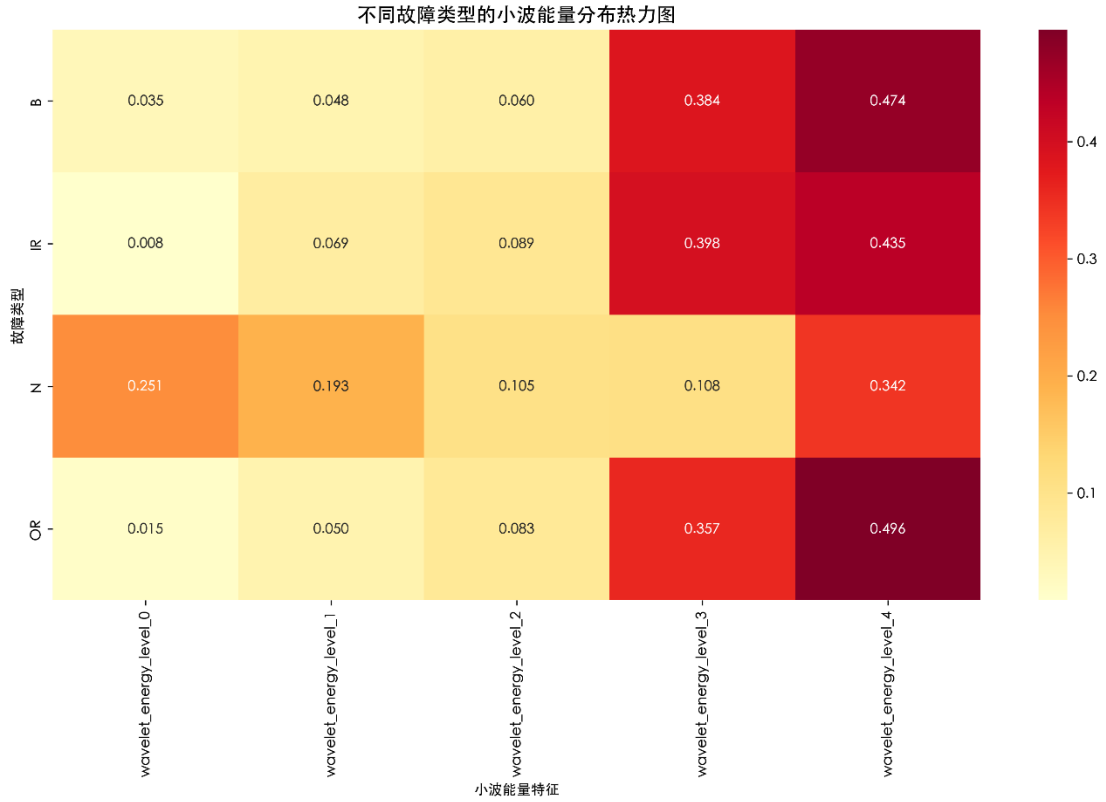


图 1- 3 不同故障类型的小波能量分布热力图

这张热力图显示的是各故障类型在 5 个小波分解层（level_0 到 level_4）上的归一化能量比例。据图可以看出，外圈故障（OR）在最高频层的占比最大，约为 0.496，滚动体故障（B）与内圈故障虽然都在中高频层占比较多，但是二者细节不同，而正常数据样本在最低频层和中低频层占比更多。因此，小波能量向高频聚集是区分故障数据和正常数据的强信号；并且 level_3 和 level_4 的相对比例可用于进一步区分三种故障。

表 1- 2 四种故障类型能量比值表

故障类型	Level_0	Level_1	Level_2	Level_3	Level_4
滚动体故障	0.035	0.048	0.06	0.384	0.474
内圈故障	0.008	0.069	0.089	0.398	0.435
外圈故障	0.015	0.05	0.083	0.357	0.496
正常	0.251	0.193	0.105	0.108	0.342

上表是更详细的数据，可以看出在最高频层（level_4），能量比值排序依次为

外圈故障、滚动体故障、内圈故障、正常；在次高频层（level_3），能量比值排序依次为内圈故障、滚动体故障、外圈故障、正常，三种故障在这两个频层中有细微差别，可做区分。由此可见，小波能量可作为代表性特征，用于区分四种故障类型，便于统计检验与输入。

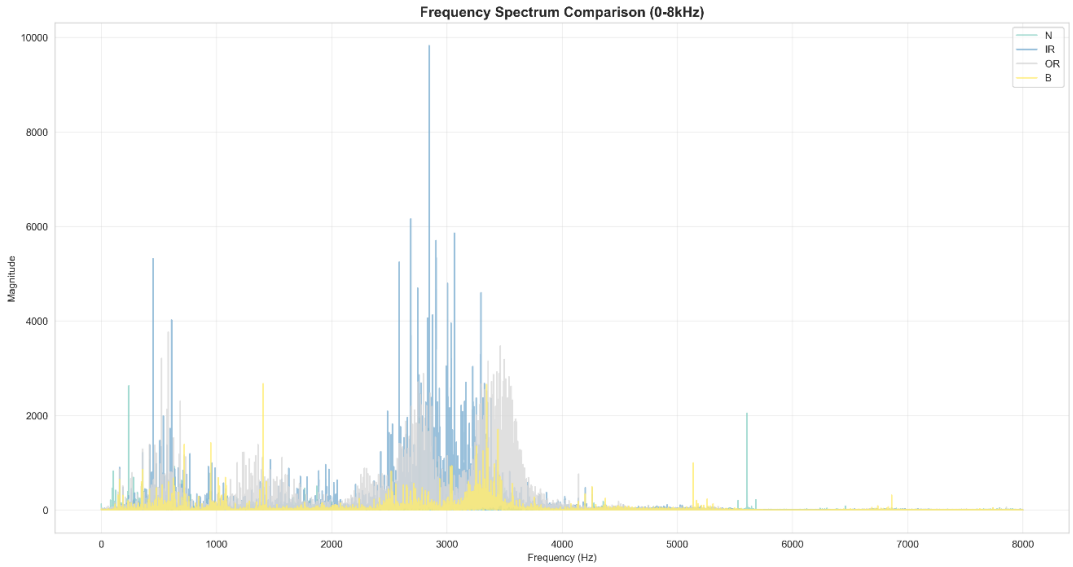


图 1- 4 频谱图

上图是 0—8kHz 频谱图，横坐标是频率，纵坐标是幅值。可以看出整体频带分布大部分集中在 0-4000Hz,在正常数据在低频有少量峰值，在 2500-3200Hz 出现一些中等幅度的谱峰，但总体幅值低于故障类，特征表现为能量分布比较平稳；内圈故障数据在 2500-3500Hz 区间存在大量强峰，峰值幅度可达近 10000，明显高于其他工况，整体表现为高频能量丰富、尖锐峰值密集。外圈故障数据在 2500-3300Hz 区间有明显的集中能量峰，但幅度普遍低于内圈故障，整体表现为频谱能量集中，不如内圈故障尖锐和高辐；滚动体故障在低频段出现一些孤立峰值，在 2500-3300Hz 仍然有峰，但幅度明显小于内圈和外圈故障。据图可以区分正常数据与故障数据，以便后续的特征选择和建模训练。

上述例子已经清晰地展示了对列车轴承故障机理特征研究的重要性，基于这四个高速列车轴承故障机理特征，我们优先保留能表达冲击周期性、包络调制、频谱峰值与瞬态脉冲的特征。还要考虑载荷、转速、传感器位置差异，确保能体现共性与差异性。

3.3.2 特征分析

在所筛选的源域数据集中，包含了几万个采样点，如果直接输入模型，对于计算能力与存储能力要求高，而且大量采样点只反应背景噪声或居于稳定区间，轴承故障特征只在某些瞬态出现，原始信号信息过于冗余，原始波形也很难直观解释对应的轴承故障机理，由下图可以看出。

在我们用随意选取的特征画出特征相关性热力图中，存在明显的冗余性。此图横纵坐标表示不同的振动特征，包括均值、均方频率、谱质心、峭度、脉冲因子、峰值因子、包络均方根、均方根、短时傅里叶变换提取的统计量、小波能量熵、分频带能量等特征；颜色表示相关系数，红色为正相关，蓝色为负相关，白色表示无明显相关性；数值表示特征间的皮尔逊相关系数，范围区间为[-1,1]。据图可以看出，均方根值、包络均方根值、短时傅里叶变换最大值、短时傅里叶变换均值、短时傅里叶变换标准差这些特征冗余性较强。

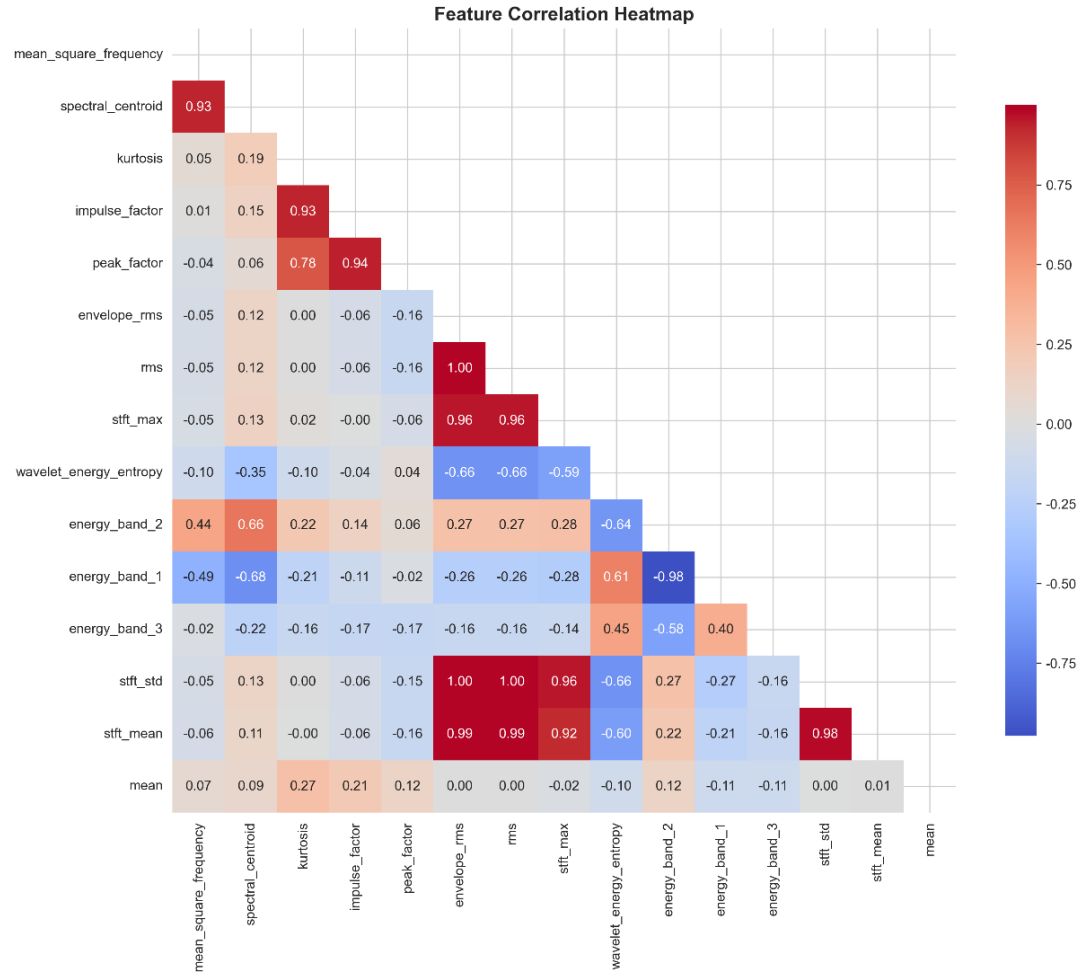


图 1- 5 特征相关性热力图

因此，我们需要进行特征提取，压缩数据维度，提高信噪比，增强分类能力和可解释性，避免数据过度拟合，使跨域迁移更稳健并保证工程实用性。下面将以轴承故障机理为依据，分析所选特征的合理性。

下图展示了不同故障类别的典型特征，行坐标分别对应四种故障类型，列坐标分别是时域波形、频谱、小波能量分布。在时域特征对比方面，内圈故障在时域波形中存在周期性冲击脉冲，幅度波动较大；外圈故障冲击更加稠密，部分样本脉冲能量更高，表现出高频振动成分；滚动体故障存在冲击出现但不规律，呈现随机突发特征；正常数据波形平稳，仅为背景噪声与低频随机波动，无明显冲

击。在频谱特征对比方面，内圈故障通过频率 BPFI 及其谐波处能量增强；外圈故障通过频率 BPFO 位置处峰值突出；滚动体故障频率 BSF 相关谱峰较为复杂；正常类能量主要集中在低频范围，频谱整体平滑，无明显故障特征频率峰。在小波能量分布对比方面，故障类数据能量更多集中在小波分解的高层，反映出高频冲击成分；正常类能量主要集中在低层，表明正常状态下信号以低频平稳成分为主。综上，故障样本相比正常样本在小波的高层上有明显能量集中；时域上故障样本多出现短时冲击，频谱上有窄带峰或高频能量抬升。据此把小波高频能量、谱质心、包络谱在轴承特征频处的峰值、峭度/RMS 作为核心判别特征。

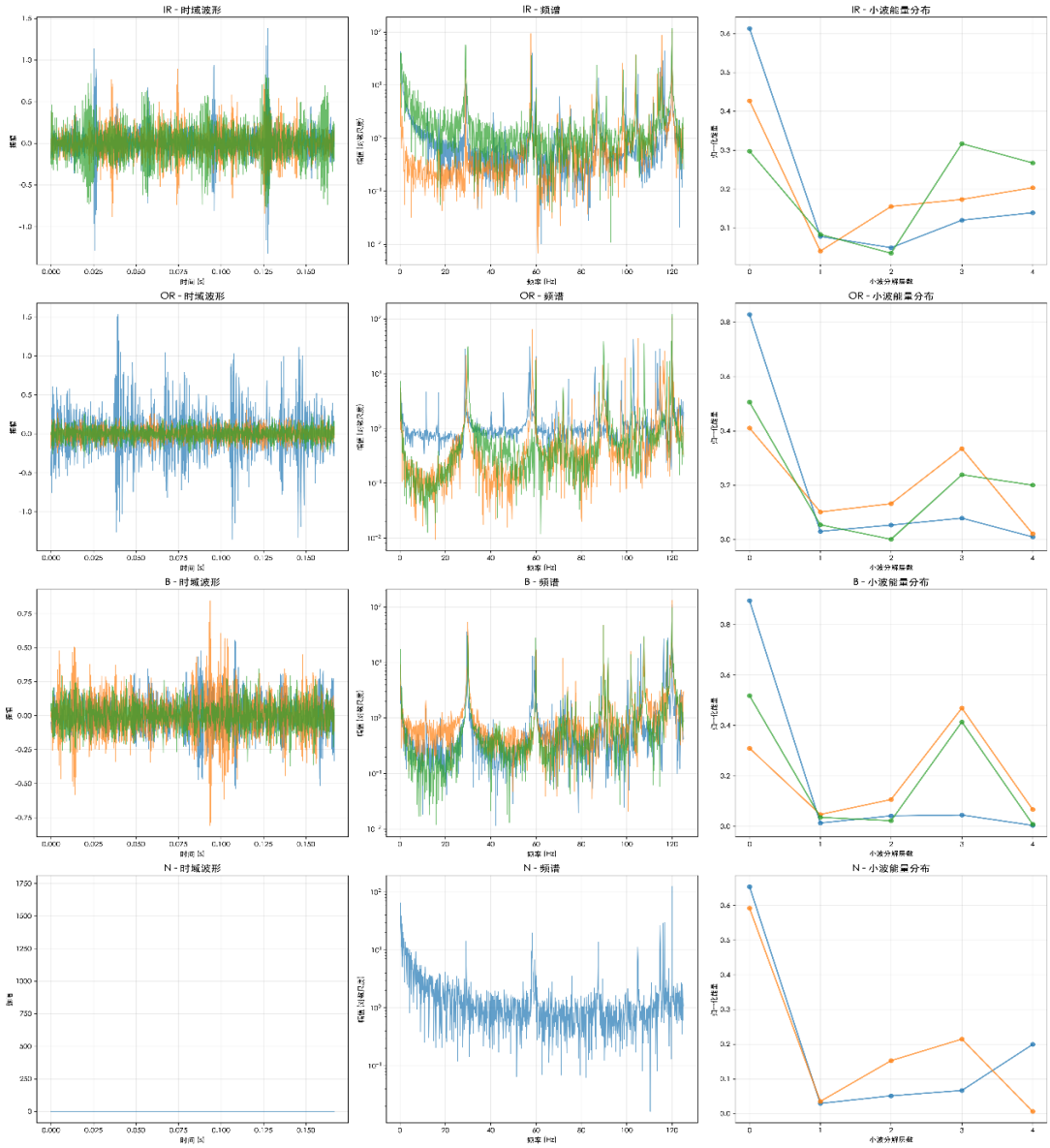


图 1- 6 四种故障类别典型特征图

综上，故障样本相比正常样本在小波的高层上有明显能量集中；时域上故障样本多出现短时冲击，频谱上有窄带峰或高频能量抬升。据此把小波高频能量、谱质心、包络谱在轴承特征频处的峰值、峭度/RMS 作为核心判别特征。

下图为特征重要性图，横坐标为四类故障，纵坐标按重要性从上到下列出特征名称。据图可以看出峭度得分最高，说明故障样本的峭度与正常样本差异极大；均方根与均方根包络紧随其后，说明 总体能量与包络能量在故障类型的区分上也非常有用；多数频带能量得分接近 0，说明这些特征对区分故障类型贡献小。

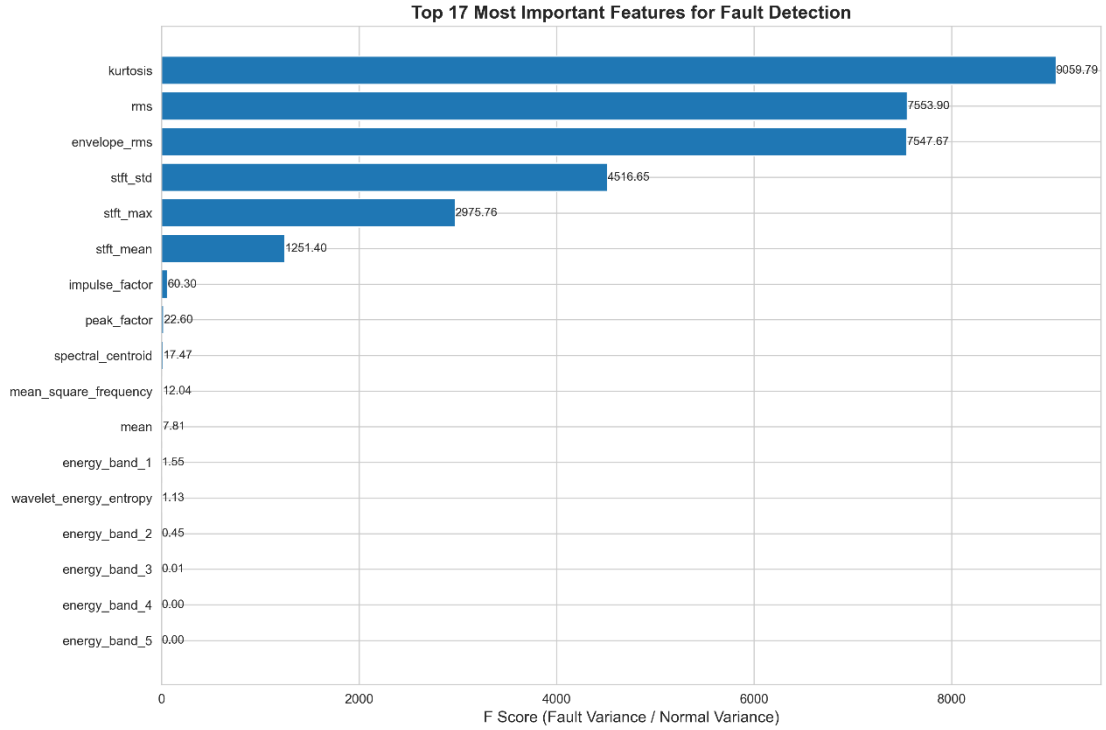


图 1- 7 特征重要性图

基于上述分析，我们分别从时域、频域、时频域提取了具有代表性的特征，包括特征从**时域（均值、RMS、峭度、峰值因子、脉冲因子）、频域（频谱能量分布、频谱质心、均方频率）、时频域（小波包能量熵、希尔伯特包络谱特征、STFT 时频图、二维图像特征）三个角度提取共 12 个特征。**其中，均值、均方根、峭度、峰值因子、波形因子、脉冲因子属于时域特征。均值主要反映直流分量或传感器基线漂移，有时能指示传感器安装偏置或低频的结构振动变化，通常先去均值能得到更稳定的结果；均方根是信号能量的代表，用于表征故障幅值变化，通常对能量增加敏感，对小幅随机噪声有平均化效果；峭度是时域特征中最重要的一个特征，它对冲击性敏感，常在轴承故障早期就能表现出显著偏离（高峭度），有利于早期诊断，适合在线快速监督预警；峰值因子反映了尖脉冲程度，对突发冲击敏感；波形因子描述了波形形状，能帮助区分噪声型与冲击型信号；脉冲因子对孤立冲击敏感，便于把我突发脉冲特性。重心频率、均方频率、频率方差属于频域特征。重心频率反映能量偏向高频还是低频，对不同故障严重度与能量分布变化敏感。轴承局部缺陷会引入高频瞬态分量，导致重心频率上移。把频谱整体变化数量化，以便快速监测频谱偏移；均方频率，对高频更为敏感，局部冲击与裂纹会把能量推向更高频，导致均方频率上升，一般作为重心频率的补

充；频率方差，冲击型故障会使频谱展宽，而平稳机械噪声频谱较窄，结合重心频率可判别“频谱位移”与“频谱扩散”两类信号变化。小波分解能量属于时频域特征，小波是时频局部化工具，能把短时、瞬态高频冲击在合适尺度上放大，适合非平稳的轴承冲击检测，抗非平稳性强。 所选特征囊括了时域、频域、时频域，能较全面的把原始振动信号中的轴承故障机理信息提炼出来，让迁移学习模型更容易区分内圈、外圈、滚动体、正常等类型。

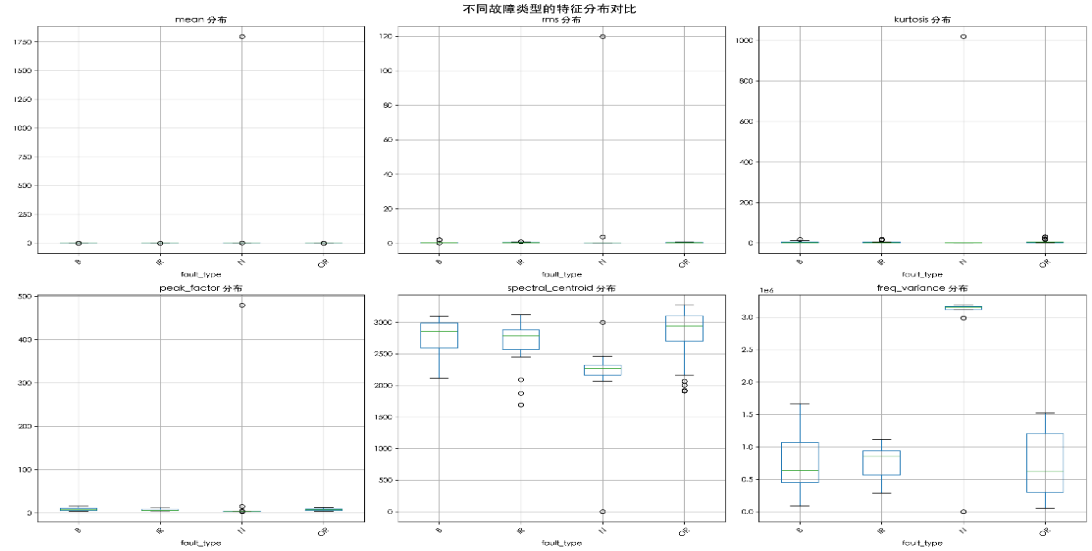
表 1- 3 特征分析表

特征名	定义	物理含义	故障响应
均值（Mean）	$\frac{1}{N}\sum_{n=1}^{N} x[n]$	信号的直流分量或偏移	通常系统会先做去直流/高通，均值接近 0
均方根 RMS（Root Mean Square）	$\sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2}$	信号能量或有效值，常用于表征总体振动能量的大小	故障（尤其严重/发展阶段）通常会提升 RMS（总体能量上升）
峭度（Kurtosis）	$\frac{\frac{1}{N}\sum(x[n]-\mu)^4}{\left(\frac{1}{N}\sum(x[n]-\mu)^2\right)^2}$	衡量“脉冲性”或“尖锐峰值”的程度——脉冲越明显，峭度越高	局部缺陷（轴承早期缺陷）会产生短时高幅冲击，使峭度显著上升，是非常灵敏的指标
峰值因子（Crest Factor）	$\frac{max\|x[n]\|}{\text{RMS}}$	峰值与有效值之比，反映瞬时尖峰在总能量中的占比	当出现单点冲击（如滚动体撞击）时 CF 增大
频谱能量分布（Frequency Spectrum Energy Distribution）	$\sum_{f\in \text{band}b} \|X(f)\|^2, \text{ratio}_b = \frac{E_b}{\sum_b E_b}$	显示能量如何随频率分布，不同故障在特定频段会有增强能量	OR/IR 会在与轴承几何和旋转速率对应的特征频率附近（可能在高频带）出现能量集中
脉冲因子（Impulse factor）	$\frac{max\|x[n]\|}{\frac{1}{N}\sum\|x[n]\|}$	峰值与平均绝对值之比，显式衡量独立峰值相对于平均水平的强度	局部冲击会显著提高 IF；比 CF 对短脉冲也很灵敏
谱质心（Spectral Centroid）	$\frac{\sum_f f\cdot\|X(f)\|}{\sum_f \|X(f)\|}$	频谱“质心”，衡量频谱向高频或低频偏移	如果故障产生高频冲击并激发结构共振，重心频率会向高频移动
均方频率（ mean-square frequency）	$\frac{\sum_f f^2\cdot\|X(f)\|}{\sum_f \|X(f)\|}$	频率的二阶矩开方，强调高频成分（比重心频率对高频更敏感）	高频谐振增强时该值上升
小波包能量熵（Wavelet Packet Energy Entropy）	$-\sum_i p_i \log(p_i)$	衡量能量在不同时频尺度的分布均匀度	冲击使能量在某些小波尺度集中，熵下降；正常或噪声更多是均匀分布，熵

			偏高
希尔伯特包络谱（Hilbert Envelope Spectrum Features）	$\lvert\mathcal{H}\{x(t)\}\rvert$	包络谱把高频冲击的调制信息调到低频，能更清晰地看到与 B/IR/OR 相关的基频	OR/IR 在包络谱上会出现对应的特征频率峰
短时傅里叶变换相关特征(STFT Time-Frequency Features)	$S(t,f)$	反映频谱随时间的变化性	在 S(t,f) 上会显示短时高能量
时频二维图像特征(2DImage Features)		把复杂的时频结构转成能被 CNN;图像特征识别的纹理;形状信息	IR/OR/B 会在时频图上形成不同的纹理

3.3.3 特征提取

源域数据经过筛选后，样本数减少，可以分段扩充样本数，以便提升训练的稳定性，也便于在段级别计算特征与置信度聚合到文件级。所以在进行特征提取前，需要将每个文件切成长度为 L 秒的段，重叠 0%–50%。例如对 8s 文件，L=1s、50% 重叠得 15 段。保存段的元信息包括文件名、起始样本、通道、RPM、载荷、标签等。基于上述的列车轴承故障机理分析与特征分析，对每个源域样本读入信号，得到特征向量，形成数据集，构建高价值特征集合。



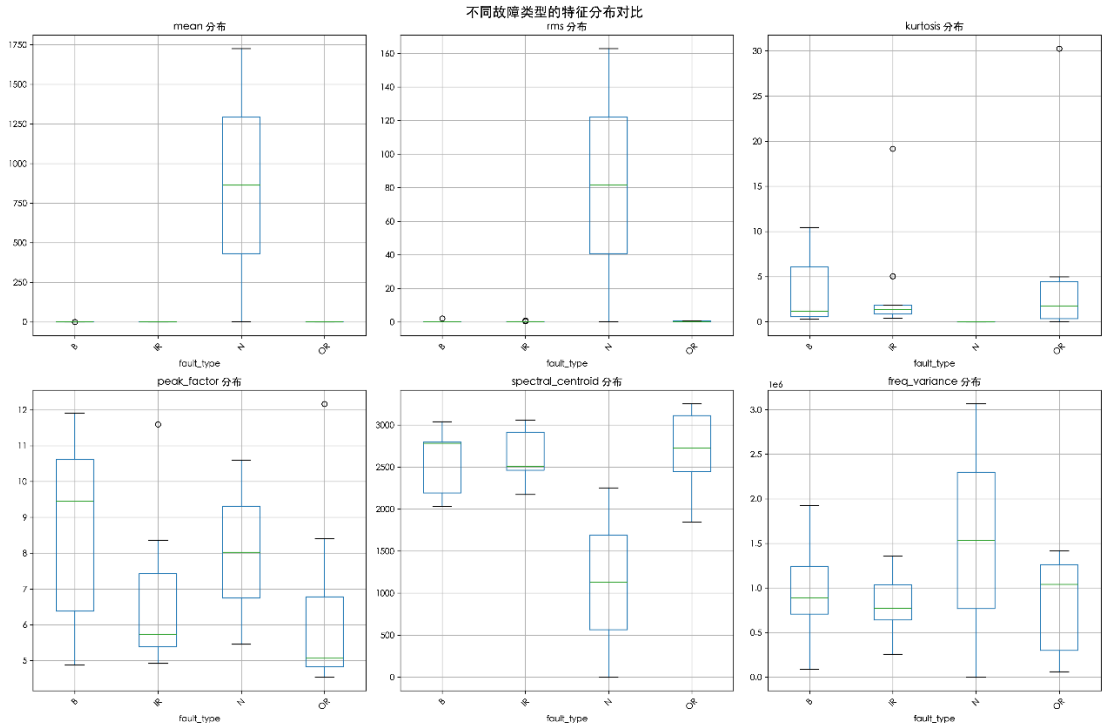


图 1- 8 不同故障类型特征分布对比图

以上两图分别是训练集与测试集的特征分布箱线图。在训练集中，均值大部分样本接近 0，但有极端高值（>1700），离群严重；均方根绝大部分样本 RMS 很小，但有异常值；峭度多数样本低于 10，也存在极端异常值（1000）；峰值因子集中在小于 10 的区间，但个别值大于 400，离群点严重；谱质心较分散，差异不明显；频率方差外圈故障数据显著高于其他类别数据。在测试集中，均值整体偏高，分布范围大，没有极端异常；均方根正常数据 RMS 整体显著大于其他故障数据；峭度分布更合理，大部分集中在 0~20 的区间，极端值不多；峰值因子分布集中在 5~12 的区间，四类故障差异更均衡，但外圈故障最低；谱质心分布更紧凑、差异更大；频率方差正常数据显著高于其他类故障数据。综合来看训练集部分特征存在极端异常值，大多数样本分布集中在较小区间；而测试集分布更均衡，极端离群点明显减少，特征值整体更稳定。由此可得，基于故障机理分析和特征分析所提取的特征集合非常可靠，表现出了良好的区分能力。

完成特征提取之后，对提取的特征进行探索性分析，用 t—SNE 验证提取的特征是否能区分不同类别。

下图中，左下角有一小簇绿色点，代表正常样本，看起来相对分离，说明某些“正常”样本在当前特征空间里特征一致性高；右上以及右中有一簇灰点（代表外圈故障样本）与黄点（代表滚动体故障样本）靠近，各自也有群落特征；中央区域有蓝（IR）、灰（OR）、黄（B）混杂，说明这些类别在当前特征表示下不可线性分割，或这些样本信号本身弱，被噪声掩盖；每个类别并非单一簇，而是可能分成几个子簇，表示类内存在工况、载荷、传感位置等多样性。

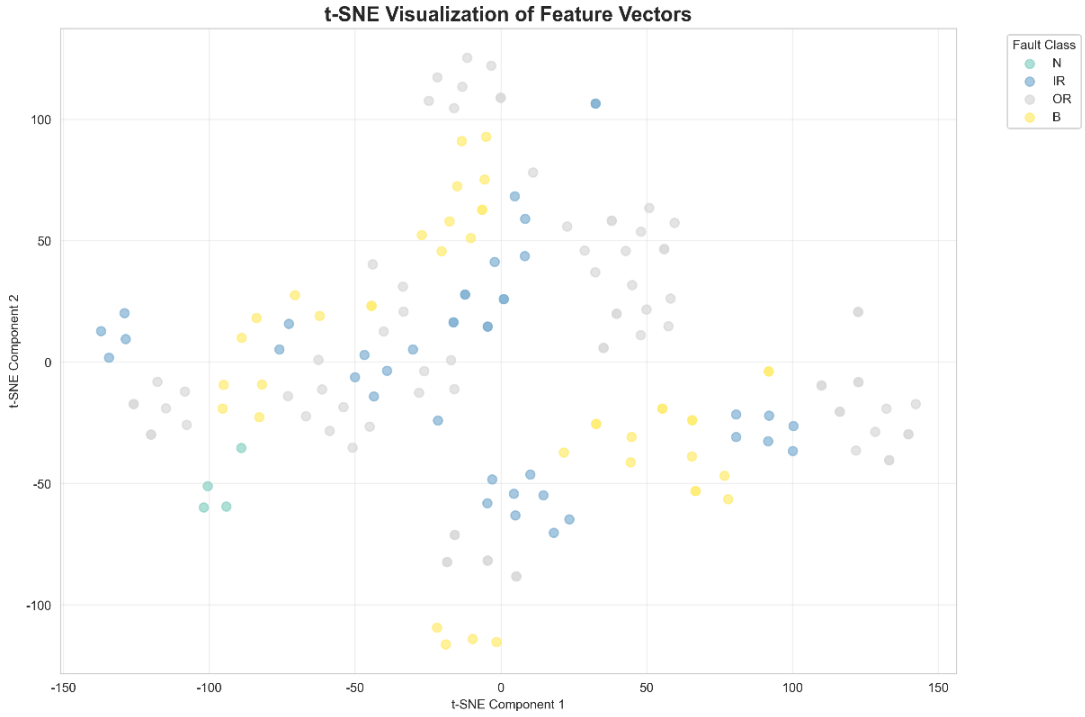


图 1- 9T-SNE 降维可视化图

T-SNE 降维可视化结果表明，正常样本与故障样本有一定分离趋势，因此现有特征在检测“是否存在故障”方面有效，为后续模型训练和迁移诊断提供了基础。

3.4 本章小结

在本章中，利用了 python 与 matlab 等工具对源域数据集进行处理分析，将源域数据集中采样频率不同的四种故障类型数据按照筛选原则，统一采样频率，进行数据去噪，构建了一个与目标域相似的数据集。基于列车轴承故障机理分析，选取有代表性的特征，组成价值最大化特征集，以助力后续模型训练、迁移学习、迁移诊断的开展。

本章首先对所给源域数据集进行筛选，优先使用 DE 传感器，把 FE 传感器作为第二通道，保留不同缺陷尺寸的样本，由于缺失值不多，对缺失值进行简单平均处理，在确保四种故障类型均能覆盖的基础上，对源域数据集统一采样频率，实现基础预处理。其次，深入分析列车轴承故障机理特征，以其为导向，提取具有代表性的特征，并进行探索性分析，得出所选特征具有故障类型可区分性的结论。

4 问题二的分析与求解

4.1 问题二的分析

问题二要求建立智能诊断模型，定量分析故障特征与设备健康状态之间的映

射关系，解决这一问题需要深入理解不同特征如何共同决定设备的故障类型与严重程度。本文采用机器学习模型从已获取的数据中学习故障特征与设备状态之间的关联规律，并寻求最优的故障诊断模型。

首先，变量选择与数据整合是模型构建的重要环节。本问题涉及的自变量是任务一提取的各类故障特征，因变量则是设备状态的分类标签，即故障类型。通过数据标准化技术，将不同量纲的特征数据进行统一缩放，确保数据的一致性和模型训练的稳定性。自变量为经过标准化处理后的特征矩阵，其特征维度包括时域统计特征、频域特征等。因变量为离散的故障类别标签。

其次，对模型进行构建与训练。分为训练集和测试集，其中 70%的数据用于拟合支持向量机、XGBoost 及随机森林模型，剩下 30%数据用于测试模型构建的有效性与准确性。在训练过程中，本文通过统计准确率、精确率、召回率、F1-Score 等指标，并可视化混淆矩阵热力图来评估模型的分类性能。模型验证与评估是确保诊断可靠性的关键步骤。通过与测试集真实标签的对比，检验模型的诊断能力和泛化性。最后本文结合设备运维背景，构建了多模型对比的诊断框架，分别探讨了不同优化策略下各模型的性能表现，通过将模型输出的分类概率与实际阈值进行比较，以最优模型的决策规则探讨关键故障特征在状态识别中的贡献度，为未来设备的智能运维与预测性维护提供科学依据。

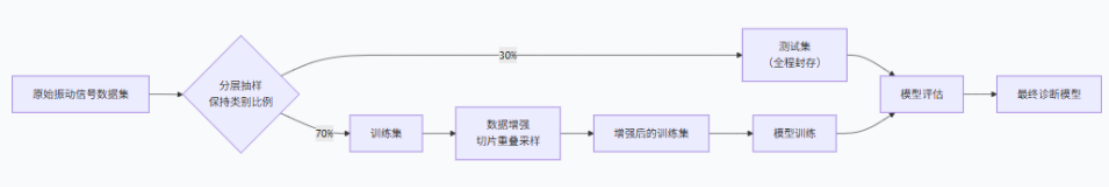


图 4- 1 第二问流程图

4.1.1 数据准备与整合

在智能故障诊断研究体系中，高质量的数据是驱动模型实现精准诊断的基石。在任务一提取的故障特征基础上，科学评估诊断模型的泛化能力，本阶段的核心任务是进行训练集与测试集的划分与数据整合。这一过程不仅是模型构建流程中的关键环节，更是客观评估模型性能、防止其陷入“过拟合”陷阱的首要步骤。其根本目的在于，通过构建一个与训练数据独立同分布的测试集，为后续模型性能提供一个无偏、可靠的评价基准。

本文所使用的数据直接继承自任务一的成果，即从原始振动信号中提取的、能够表征设备不同健康状态的特征矩阵。该特征矩阵的行对应每一个样本（如一段固定长度的振动信号），列则对应多种时域与频域特征维度（如均值、均方根、峰值因子、频谱重心等），并已完成了初步的清洗与缺失值处理。每个样本均带有其对应的故障类型标签，共同构成了源域诊断的初始数据集。

数据整合与预处理的技术流程是本环节的核心，其具体步骤如下：

第一步：基于分层抽样的数据划分

为确保训练集与测试集能够全面、均衡地代表所有故障类别，避免因随机划分导致的类别分布不均问题，本文采用“**保持类别比例的分层抽样**”策略。该方法能够保证在划分后的训练子集和测试子集中，各类故障（如外圈故障（OR）、内圈故障（IR）、滚动体故障（B）以及正常工作状态（N））的样本比例与原始数据集中的总比例基本一致。具体操作上，本文将总体源域文件按 **70% 与 30% 的比例划分为训练集与测试集**。训练集用于模型的参数学习与超参数优化，而测试集则在模型训练全程保持“封存”状态，仅在最终评估时使用，以模拟模型在面对全新、未见过的数据时的真实诊断能力。

第二步：通过数据增强提升模型鲁棒性

针对故障诊断中常见的数据量有限、样本不均衡等问题，为了充分挖掘数据潜力、提升模型的泛化性与鲁棒性，本文在训练集上采用了**切片重叠采样**的数据增强方法。该技术通过对原始的振动信号序列进行滑动窗口截取，并允许相邻窗口之间存在部分重叠，从而从一个原始样本生成多个具有细微差异的新样本。此方法在不改变故障物理本质的前提下，有效扩充了训练集的规模，尤其增加了模型对振动信号相位微小变化的容忍度，相当于让模型“见过”了更多可能的工况变体，从而学习到更稳定、更本质的故障特征规律，而非仅仅记忆有限的训练样本。

经过上述系统的数据准备流程，本文成功构建了一个适用于监督学习范式的高质量数据集。该数据集结构清晰，训练集特征矩阵与训练集故障标签一一对应，用于驱动模型学习；测试集特征矩阵与测试集故障标签一一对应，用于公正评估。所有特征数据均已完成标准化处理，消除了量纲影响。此数据集为后续支持向量机、XGBoost 及随机森林等模型的对比训练与优化提供了统一、规范的基准。

本文将基于这一精心准备的数据集展开后续的机器学习建模。可靠的数据基础是确保诊断结论科学性的前提，此环节的工作为定量比较不同模型的诊断性能、深入探究故障特征与设备状态之间的映射关系，以及最终确立一个高性能的源域故障诊断基准奠定了坚实的基础。

4.1.2 变量准备

在完成训练集与测试集的划分后，本文的下一步是进行变量的最终准备与数据整合。此环节的核心任务在于，将任务一中提取的原始特征转化为适用于机器学习模型的高质量、标准化输入，并明确模型的预测目标。其根本目的是构建一个特征与标签对应关系清晰、格式统一的数据集，为后续模型的高效训练与精准评估奠定基础。本文的自变量为经过标准化处理后的训练集特征矩阵，它承载了

振动信号中与设备健康状态相关的关键信息；而因变量则为故障类型标签，是模型需要学习和预测的最终目标。

变量准备的技术流程是本环节的核心，我们对原始特征进行深化处理与组织，具体步骤如下：

第一步：深化处理与构建自变量（特征矩阵）

自变量（特征矩阵）的质量直接决定了模型性能的上限。为此，本文在任务一初步特征提取的基础上，采用了主辅结合的策略对其进行深化构建：

首选方法：基于包络谱的低频能量特征提取

滚动轴承发生局部故障时，会产生周期性冲击信号，这些信号通常被调制在高频共振频带上。为精准捕捉这一关键特征，本文的首选方法是对每个样本信号段（切片）进行包络谱分析。具体而言，首先通过希尔伯特变换计算振动信号的包络线，再对包络线进行傅里叶变换得到包络谱。包络谱能有效地解调出故障特征频率，并将其凸显在低频范围内。本文将包络谱中低频部分（例如，前 500 个频率点）的能量作为核心特征向量。该方法的核心优势在于，它同时完成了信号去噪和故障特征增强，通过聚焦于包含故障特征频率的低频成分，有效过滤了高频噪声和非关键信息的干扰，从而提取出与故障类型高度相关的敏感特征。

辅助方法：时域统计特征的补充与拼接

为构建更全面、鲁棒的特征集，弥补单一频域特征可能存在的不足，本文采用辅助方法，计算每个样本段的时域统计特征。这些特征包括但不限于峭度（对冲击敏感）、峰值因子、均方根值（代表能量水平）等经典指标。最终，将计算得到的时域统计特征与从包络谱中提取的低频能量特征进行向量拼接，共同形成最终的高维、多视角特征向量，即完整的**自变量——标准化特征矩阵**。这种主辅结合的特征构建策略，既充分利用了故障的物理机理（包络谱），又保留了信号的宏观统计信息（时域特征），旨在为模型提供最丰富、最有效的判别依据。

第二步：定义与整理因变量（故障标签）

因变量的准确性与一致性是监督学习成功的前提。本文明确将**故障类型作为模型的因变量**，即模型需要预测的分类目标，故障类型为离散的类别标签。为确保标签与特征的正确对应，本文对所有样本的故障标签进行了逐一核对，将其转换为模型可处理的数值形式，并确保每个样本的特征向量都与其唯一的故障标签精确匹配。

本文将基于这一精心准备的变量集合展开后续的机器学习建模。高质量的特征与清晰的标签是模型能否准确“学会”故障诊断知识的关键。此环节的工作，不仅为支持向量机、XGBoost 和随机森林等模型的输入提供了标准格式，更重要的是，通过包络谱等基于物理机理的特征提取方法，提升了特征集的判别能力，

为最终实现高精度的源域故障诊断提供了可靠的数据保障，并为深入分析不同特征对分类的贡献度奠定了基础。

4.1.3 数据描述与背景分析

在智能故障诊断研究中，所使用的数据集是模型训练与验证的基石，其质量与特性直接决定了诊断系统的性能上限。本研究所采用的数据集，是列车关键旋转部件——滚动轴承在不同健康状态下的振动信号记录，它为后续构建高精度诊断模型并实现有效的迁移学习提供了至关重要的数据支撑。

滚动轴承是列车轮对中支撑轴重、保证平稳旋转的核心部件，其结构主要包括外圈、内圈、滚动体（如滚珠或滚子）和保持架。

本数据集正是通过高精度传感器，捕捉了轴承在上述四种状态下的“健康指纹”——振动信号。具体而言，采集时间为 8 秒，采样频率高达 32kHz，这意味着每秒钟记录 32,000 个数据点，从而能够精确捕获故障引发的瞬态冲击信号。数据采集时列车运行速度约为 90km/h，对应轴承转速约为 600 转/分钟（rpm），这模拟了列车在干线运行时的典型工况。一个重要的特点是，数据文件以英文字母 A~P 编号命名，但其对应的具体工作状态（是 N，OR，IR 还是 B）是未知的。这一设定极大地增强了数据集的挑战性与现实性，模拟了工业现场经常面临的“数据标签缺失或模糊”的场景，要求诊断模型或分析人员必须能够从数据本身挖掘出有效的状态辨识规律

对数据集进行初步的统计分析，是理解其分布规律和挑战性的关键一步。振动信号作为一种典型的时序数据，其统计特性能够反映设备的运行状况。

在时域内，可以计算诸如均方根值（RMS，代表能量水平）、峰值、峭度（对冲击敏感）等指标。正常状态（N）下的振动信号，其幅值通常较为稳定，RMS 值较低，峭度值接近 3（正态分布的峭度）。而当存在故障时（如 OR、IR、B），由于周期性冲击的出现，信号的峰值和峭度会显著增大，表现出明显的“尖峰脉冲”特性。通过对比不同编号文件（A~P）的这些时域统计量，可以初步窥见其状态差异。例如，某些文件的峭度值异常高，则极有可能对应着某种故障状态。

在频域内，通过傅里叶变换可以将振动信号转换为频谱。正常轴承的频谱成分相对简单，主要集中在轴转频及其倍频处。而故障轴承的频谱中，会在其对应的故障特征频率（与外圈、内圈、滚动体的几何尺寸和转速相关）及其倍频处出现明显的谱线，这是进行故障类型辨识的最直接证据。本数据集高达 32kHz 的采样频率为分析这些高频共振频带提供了保障。

该数据集的独特构成使其成为验证迁移学习算法的理想平台。如前所述，迁移学习旨在利用源域（如，本数据集中部分已知状态或在不同工况下采集的数据）的知识，来提升模型在目标域（如，数据集中其余未知状态或新工况数据）上的

性能。

1. 作为跨工况迁移的源域：本研究的数据集是在固定转速（600rpm）下采集的。它可以作为源域，用于迁移到列车加速、减速或其他转速（如 300rpm 或 900rpm）等不同工况（目标域）下的故障诊断任务。模型可以学习从本数据集中学到的通用故障特征（如冲击的波形模式），进而通过微调来适应目标域的新转速，从而减少在新工况下收集大量标注数据的需求。

2. 作为跨设备迁移的源域：即使对于不同的列车车型或甚至其他旋转机械（如电机、齿轮箱），其滚动轴承的故障物理机理（冲击的产生）是相通的。在本数据集上训练得到的特征提取器，可能具备捕捉这种通用机理的能力，从而可以迁移到结构相似但不同的设备上进行故障诊断。

3. 模拟标签缺失的真实场景：数据文件状态未知（A~P 编号）这一特点，完美模拟了工业现场大量数据无标签的困境。这要求研究方法必须具备无监督或半监督学习能力，或者利用迁移学习，将从少量有标签数据（源域）学到的知识，推广到大量无标签数据（目标域）的状态识别中。

本数据集不仅提供了轴承故障诊断的典型样本，更因其高质量的采集设置和“状态未知”的巧妙设计，为研究如何克服工况变化、标签稀缺等实际挑战，构建具有强泛化能力的智能诊断系统提供了不可或缺的试验场。其在本迁移学习研究框架下的价值，远超出作为一个简单的分类数据集。

4.1.4 建立 SVM、XGBoost、随机森林并优化

在数据准备就绪后，模型的选择与优化是决定诊断性能的关键。为充分挖掘数据潜力并确保模型的鲁棒性，本研究摒弃单一模型可能存在的偏好风险，采用一种基于多模型对比与超参数优化的集成策略。其基本思想是并行构建多种原理各异的机器学习模型，并通过先进的优化技术寻找其最佳配置，最终通过公正评估筛选出最优的源域故障诊断模型。

（1）支持向量机原理与建模流程

支持向量机[1-2](support vector machines，SVM) 是建立在统计学习理论[3-4]VC 维理论和结构风险最 小化原理基础上的机器学习方法。它在解决小样本、非线性和高维模式识别问题中表现出许多特有的优 势，并在很大程度上克服了“维数灾难”和“过学习”等问题。此外，它具有坚实的理论基础，简单 明了的数学模型，因此，在模式识别、回归分析、 函数估计、时间序列预测等领域都得到了长足的发 展，并被广泛应用于文本识别[5]、手写字体识别[6]、 人脸图像识别[7]、基因分类[8]及时间序列预测[9]等。[1]丁世飞,齐丙娟,谭红艳.支持向量机理论与算法研究综述[J].电子科技大学学报,2011,40(01):2-10.

核心目标： SVM 的核心目标是找到一个超平面 $\omega x+b=0$ ，使得该超平面

与最近的样本点（即支持向量）之间的间隔（margin）最大化。最大化间隔的初衷是希望模型具有更好的泛化能力，对未知样本的预测更加鲁棒。

量化标准：最优超平面的求解可以转化为一个凸二次规划问题。通过引入拉格朗日乘子和核技巧，将原始问题转化为对偶问题求解。常用的核函数包括线性核、多项式核以及高斯径向基核（RBF），其中 RBF 核因其强大的非线性映射能力而被广泛采用。

流程说明：

1.数据标准化：由于 SVM 对特征尺度敏感，首先需要对训练集特征进行标准化处理。

2.核函数选择：根据数据特征选择适当的核函数。对于非线性可分的故障数据，通常优先选择 RBF 核。

3.模型训练：通过求解优化问题，确定支持向量及超平面参数。

4. 超参数优化：使用网格搜索或随机搜索对惩罚系数 C 和核参数 γ 进行调优，以平衡分类准确率与模型复杂度。

数学表达：

SVM 的原始优化问题可表述为：

$$\min_{w,b,\xi} \frac{1}{2} \| w \|^2 + C \sum_{i=1}^{n} \xi_i$$

$$\text{subject to } y_i(w \cdot \phi(x_i) + b) \geq 1 - \xi_i, \xi_i \geq 0, i = 1, \ldots, n$$

其中，w 是超平面的法向量，b 是偏置项，$\phi(x_i)$是将样本 x_i 映射到高维特征空间的函数，C > 0 是惩罚参数，用于平衡间隔大小与分类错误容忍度，ξ_i 是松弛变量。

（2）XGBoost 原理与建模流程

XGBoost 是一种提升树模型［12］，它由多个弱分类器集成而构建强分类器［18-19］。其基本思想是通过迭代地训练一系列决策树，每棵树学习之前所有树组合的残差，最终将所有树的预测结果加权求和得到最终输出。

原理阐述：

核心目标：XGBoost 的核心目标是最小化包含模型复杂度的正则化损失函数。通过加法模型和前向分步算法，逐步优化损失函数，同时通过正则项控制模型复杂度，防止过拟合。

量化标准：模型的目标函数由损失函数和正则化项组成：

$$\mathcal{J} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k),$$

其中 l 为损失函数，Ω 为正则化项，用于控制树的复杂度。

流程说明：

1. 初始化模型：初始化一个简单的模型（如常数值）。

2. 迭代构建决策树：对于每次迭代：

计算当前模型的负梯度（残差）

使用残差拟合一个新的决策树

通过线性搜索确定最优步长，更新模型

3. 早停机制：当验证集性能不再提升时，提前停止迭代，防止过拟合。

4. 五折交叉验证优化：将训练集分为 5 折，轮流使用其中 4 折训练，1 折验证，循环 5 次。通过交叉验证的平均表现选择最优超参数组合，如学习率、树的最大深度、子采样比例等。

数学表达：

XGBoost 在第 t 次迭代时的目标函数可近似为：

$$\mathrm{I}^{(t)} \simeq \sum_{i} \left[l\left(y_i, \hat{y}_i^{(t-1)}\right) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

其中，$g_i = \partial_{\hat{y}_i^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$和$h_i = \partial^2_{\hat{y}_i^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$分别是一阶和二阶梯度统计量。

（3）随机森林原理与建模流程

随机森林（Random Forest）是一种集成学习算法，通过构建多棵决策树并将它们的预测结果进行集成（投票或平均）来完成分类或回归任务。其基本思想是结合"Bootstrap Aggregating"（Bagging）和随机特征选择来增加模型的多样性，从而降低过拟合风险。

原理阐述：

核心目标：随机森林的核心目标是通过构建多棵差异化的决策树，并集成它们的预测结果，以期获得比单一决策树更稳定、更准确的预测。模型的泛化误差取决于森林中每棵树的分类强度和各树之间的相关性。

量化标准：对于分类任务，最终预测结果通常采用投票法（多数决）；对于回归任务，则采用平均法。通过袋外误差（Out-of-Bag Error, OOB Error）可以无偏估计模型的泛化误差。

流程说明：

1. Bootstrap 抽样：从原始训练集中有放回地随机抽取 n 个样本，作为一棵决策树的训练集。

2. 随机特征选择：在决策树的每个节点分裂时，随机从全部特征中选择一个特征子集，然后从该子集中选择最优分裂特征。

3. 完全生长决策树：让每棵决策树完全生长，不进行剪枝。

4. 贝叶斯优化：建立超参数（如决策树数量 n_estimators、最大特征数

max_features 等）与模型性能的概率代理模型，通过不断评估和更新模型，智能地选择下一组可能表现更好的超参数进行试验，以较少的迭代次数找到最优超参数组合。

数学表达：

对于包含 T 棵树的随机森林，分类任务的预测可表示为：

$$\hat{y} = mode\{h_t(x)\}_{t=1}^{T} \leftarrow$$

其中，$h_t(x)$表示第 t 棵树的预测,mode 表示取众数。袋外误差估计为：

$$\text{OOBError} = \frac{1}{N}\sum_{i=1}^{N} I\left(y_i \neq \hat{y}_i^{OOB}\right)$$

其中，$\hat{y}_i^{OOB}$表示仅由那些在 Bootstrap 抽样时未包含样本 i 的树组成的子森林对样本 i 的预测。

4. 核心目标与关键技术点

共同目标：三种算法的共同目标都是通过不同的机制提高模型的泛化能力，在源域数据上建立准确可靠的故障诊断模型，为后续的迁移学习任务奠定基础。

关键技术：

1. SVM 的核技巧使其能有效处理非线性分类问题；

2. XGBoost 的梯度提升机制和正则化项使其具有高精度和抗过拟合能力；

3.随机森林的 Bagging 和随机特征选择机制保证了模型的多样性和鲁棒性；

4.五折交叉验证和贝叶斯优化等超参数寻优策略确保了各模型均能以接近最优的性能参与最终对比。

通过这三种各具特色的算法及其优化策略的对比研究，可以全面评估不同方法论在轴承故障诊断任务上的适用性，为选择最优的源域诊断模型提供科学依据。

4.1.5 建模结果

（1） XGBoost 模型最佳参数组合

表 4- 1 XGBoost 模型最佳参数组合

nrounds	max_depth	eta	subsample	alpha
300	7	0.3	0.9	0
colsample_bytree	min_child_weight	gamma	Lambda	
0.7	1	0	1	

上述参数的含义为：

nrounds = 300：模型将迭代 300 轮，生成 300 棵决策树

max_depth = 7：每棵树的最大深度为 7，属于中等深度，平衡了模型复杂度和拟合能力

eta = 0.3：学习率，控制每棵树对最终预测的贡献权重。值为 0.3，属于中等偏高学习率，有助于快速收敛，但也可能增加震荡风险。

subsample = 0.9：每轮迭代时使用 90%的样本进行训练

colsample_bytree = 0.7：每棵树只使用 70%的特征

min_child_weight = 1：叶节点所需的最小样本权重和较低，允许树生长得更深

gamma = 0：节点分裂所需的最小损失减少值为 0，相对容易分裂

lambda = 1：L2 正则化系数，控制模型复杂度

alpha = 0：L1 正则化系数为 0，不使用 L1 正则化

整体来看，这组参数在模型复杂度、学习效率与泛化能力之间取得了较好的平衡。使用较高的学习率（0.3）和较多的树（300）实现快速收敛，并且控制树深为 7，在表达能力和过拟合之间取得平衡，引入样本与特征采样增强多样性，并通过 L2 正则化（lambda=1）抑制模型复杂度。

（2） XGBoost 模型关键超参数对平均测试误差的影响分析——基于箱线图的敏感性评估

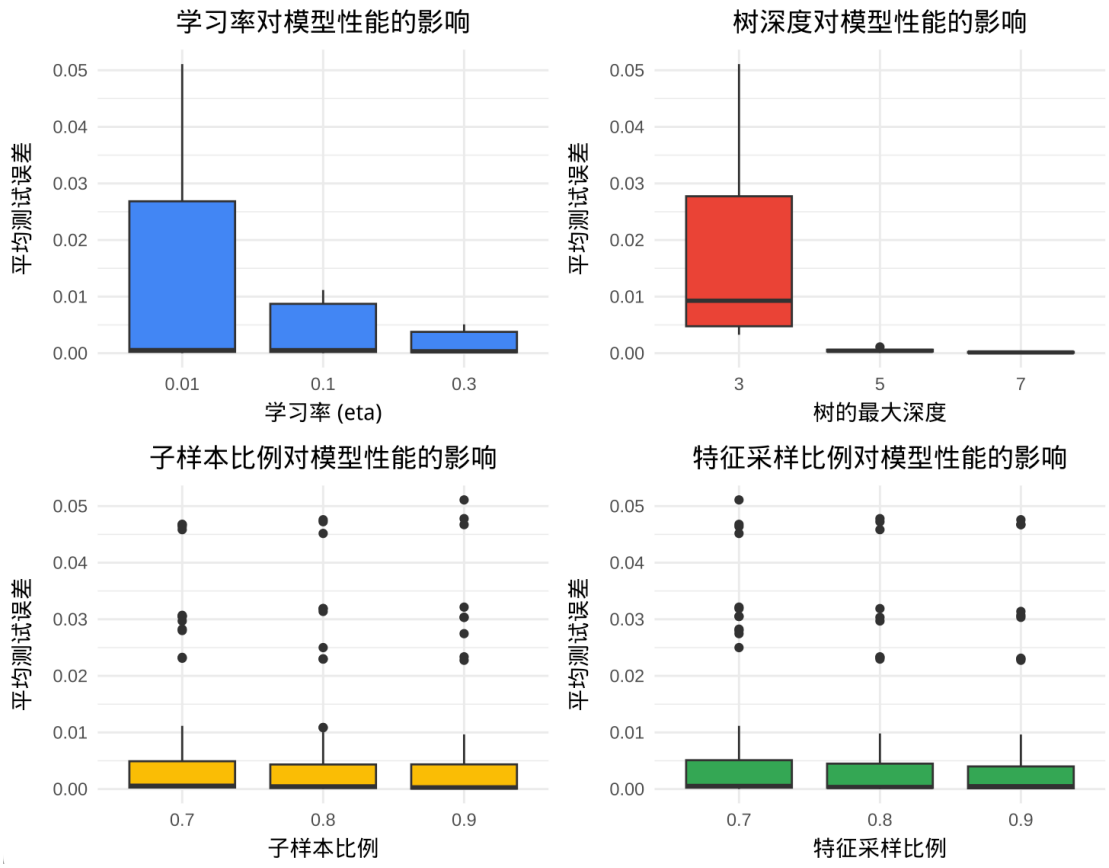


图 4- 2 箱线图

该图横轴为对应超参数的不同取值，纵轴为“平均测试误差”，我们希望该误差越低越好，通过箱体高度、中位线位置、离群点分布等可视化元素，直观展示模型在不同配置下的稳定性与最优区间。

左上图中，箱体高度反映误差波动范围，中位线代表典型误差水平，上下须表示极值范围，散点为每次实验的具体结果。学习率为 0.01 时，误差较高且波动大；0.1 时误差下降、稳定性增强；0.3 时误差最低、最稳定。这表明，学习率越大，平均测试误差越低，模型性能越好。但需警惕过大的学习率可能导致震荡或过拟合。

右上图代表树深度对模型性能的影响，该图表明，树深为 3 时，箱体高、中位数约 0.01，误差分布较分散，说明浅层树表达能力有限；树深为 5 时，箱体显著压缩，中位数降至约 0.006，误差集中且低，是当前设置下的最优深度；树深为 7 时，箱体几乎退化为一条线，中位数接近 0，误差极低且稳定，说明深层树能更好捕捉数据复杂结构。结果表明，适当增加树深度可显著提升模型性能，但需配合早停或正则化防止过拟合。在本实验中，深度 5 和 7 表现优异，推荐使用 5 或 7。

左下图代表子样本比例对模型性能的影响，所有三个比例下，箱体高度相近，中位数都在 0.002~0.004 之间，说明模型对子样本比例不敏感；但散点图显示，当比例为 0.7 时，存在较多离群点，说明抽样比例过低可能导致部分训练轮次因样本不足而性能骤降；比例 0.8 和 0.9 时，散点分布更集中，误差更稳定。该结果表明子样本比例在 0.8~0.9 范围内对模型影响较小，推荐使用 0.8 或 0.9 以保持稳定性；0.7 虽可行但风险较高，易出现不稳定结果。

右下图代表特征采样比例对模型性能的影响，三组箱体高度相似，中位数均在 0.003 左右，说明特征采样比例对整体性能影响不大；但 0.7 比例下，存在多个高误差离群点（最高达 0.05），表明特征抽取过少时，模型可能丢失关键判别信息；0.8 和 0.9 比例下，误差分布更紧凑，尤其 0.9 时离群点最少，稳定性最佳。该图表明特征采样比例不宜过低，0.8 或 0.9 更为稳妥；0.7 虽可接受但会增加模型不稳定性，建议避免。

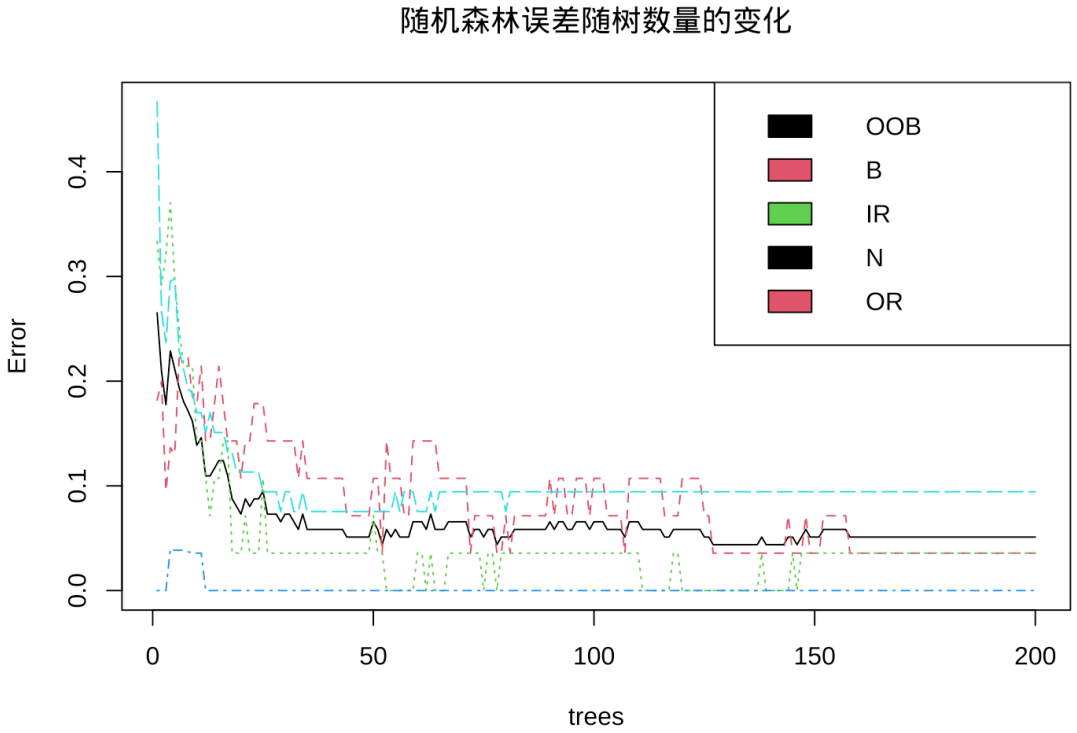


图 4- 3 随机森林误差变化图

OOB 误差评估模型整体性能，初始阶段误差较高且波动剧烈，说明模型尚未充分学习；随着树数增加迅速下降，在约 50 棵树后趋于稳定，最终稳定在 0.05 左右；这表明随机森林在树数达到一定规模后已基本收敛，继续增加树数对性能提升有限，仅能略微平滑误差曲线。该结果表明 OOB 误差作为内部验证指标，不仅反映了模型的整体泛化能力，也验证了其无需独立测试集即可进行有效评估的特性。

B 类初始误差约 0.2，波动较大，在约 30 棵树后迅速下降并稳定在 0.08~0.10 区间，这表明 B 类样本具有较好的可分性，模型能较快学会其判别模式。

IR 类初始误差较低，但波动明显，在 50 棵树后趋于平稳，稳定在 0.06~0.08，说明 IR 类虽有一定复杂度，但模型仍能有效识别。

N 类初始误差较高，但下降速度快，在约 50 棵树后稳定在 0.04~0.06，是所有类别中误差最低的一类，这可因其特征鲜明、分布集中，易于被模型准确识别。

OR 类初始误差较高，波动剧烈，下降速度较慢，直到 100 棵树后才趋于稳定，最终稳定在 0.08~0.10，结果显示 OR 类可能是最难分类的一类，可能因样本稀疏、边界模糊或与其他类重叠严重所致。

工业应用中，我们应重点关注 OR 类，建议增加该类样本量或采用过采样技术，引入代价敏感学习，提高误判惩罚，分析其特征分布，构造更具判别力的新特征。

（3） SVM、XGBoost、随机森林分类性能指标对比

表 4- 2 XGBoost 模型最佳参数组合

	model	accuracy	precision	recall	f_1
Accuracy	SVM	0.8367347	0.8927203	0.843750	0.8609205
Accuracy1	XGBoost	0.7551020	0.8011905	0.7916667	0.7939977
Accuracy2	随机森林	0.8367347	0.8666667	0.8750000	0.8590909

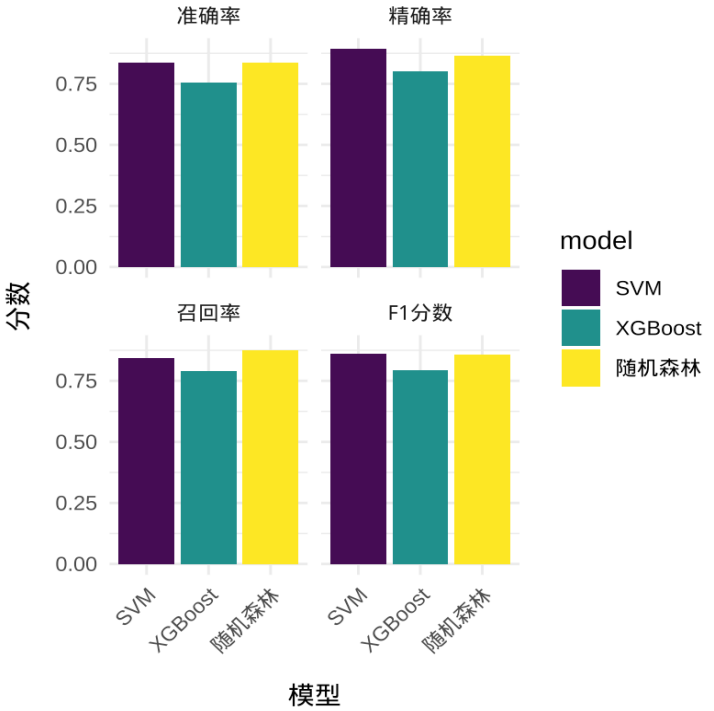


图 4- 4 不同模型性能指标比较图

上图包含四个常见的分类评估指标：准确率（Accuracy）、精确率（Precision）、召回率（Recall）和 F1 分数（F1-Score）。每个指标都对比了三种模型：SVM、XGBoost、随机森林。

分析其中结果可知，准确率（Accuracy）层面，SVM 与随机森林并列第一，表明两者在整体预测上表现最佳，XGBoost 明显落后，仅达到约 75.5%，

说明其在当前数据集上泛化能力较弱。这表明 SVM 和随机森林在整体分类精度上显著优于 XGBoost。

精确率（Precision）层面，SVM 最高，这表示当它预测某个样本为“故障”时，很大概率代表其是真的故障，误报率最低，随机森林次之，XGBoost 最低，这代表其存在较多“假阳性”，即把正常样本误判为故障。结果表明，若应用场景重视少误报，则 SVM 是最优选择。

召回率层面，随机森林最高，代表其能识别出最多的真正故障样本，漏检最少，SVM 较低，略逊于随机森林，XGBoost 居中，漏检风险较高。这表明若应用场景重视不遗漏，则随机森林是首选。

F1 分数层面，SVM 最高，因其极高的精确率拉高了 F1 值，随机森林紧随其后，虽然它精确率稍低，但召回率极高，实现了良好的平衡，XGBoost 最低，表明其综合性能明显不足。从综合性能看，SVM 与随机森林非常接近，均远优于 XGBoost。

综合来看，SVM 性能最好，代表我们在选择模型时，应选取最合适的，而不是传统观念上认为最好的。

（5）各模型混淆矩阵比较

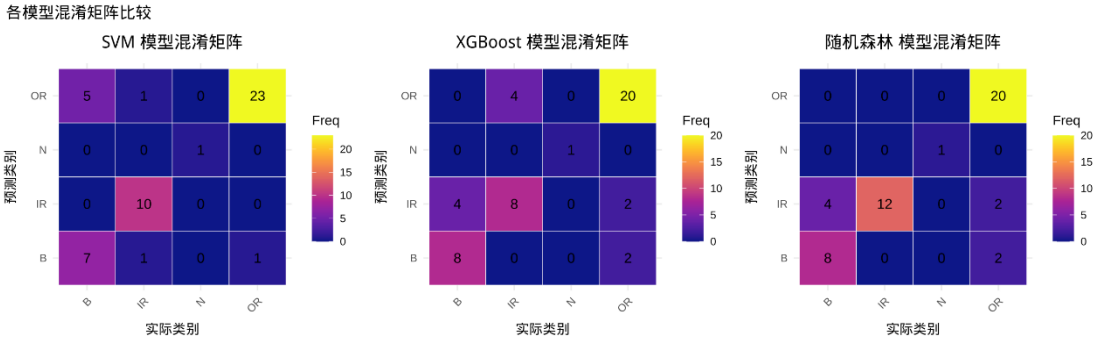


图 4- 5 混淆矩阵比较图

上图中每个子图是一个 4×4 的热力图，行表示“预测类别”，列表示“实际类别”，颜色深浅代表预测频次（Freq），数值是对应类别的样本数量。

具体分析 SVM 模型的结果，我们发现 SVM 对 IR 类判别能力强，但对 OR 类存在较多误判，尤其将 B 类错判为 OR，可能导致“虚警”。

XGBoost 在 OR 类上表现最好，但在 B 与 IR 之间存在较多混淆，整体分类能力弱于随机森林。随机森林在所有类别中均表现出最佳的分类性能，尤其是 OR 类实现零误判，IR 类也高度准确，综合表现最优。

最终可以发现，随机森林模型在混淆矩阵上的表现最为优异，尤其在 OR 类实现了完全正确识别，在 IR 类也展现出高召回率；而 SVM 虽然对 IR 类识别准确，但存在将 B 类误判为 OR 的风险；XGBoost 则在各类别间均有较多混淆，整体性能最弱。

（6）基于 XGBoost 与随机森林的故障特征重要性对比分析

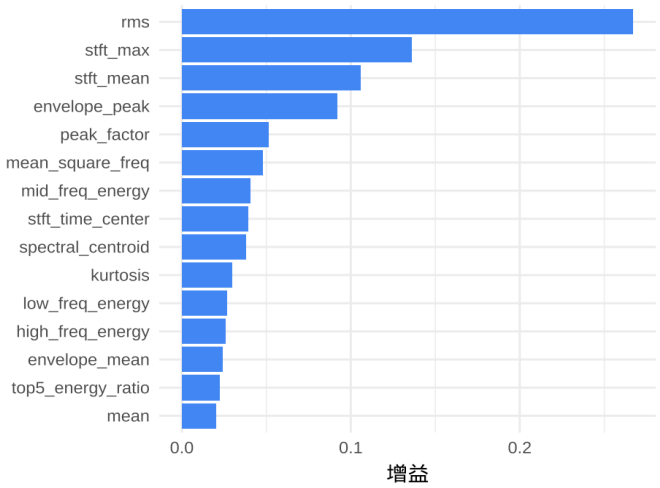


图 4- 6XGBoost 模型特征重要性图

上述条形图中，条形的长度代表了每个特征的“增益”值，从而直观地显示了特征的重要程度，它展示了 16 个不同特征对模型预测的贡献程度。从图中可以清晰地看到：最重要的特征是 rms (均方根)，其增益值远超其他所有特征，紧随其后的是 stft_max 和 stft_mean，它们的增益值都在 0.1 到 0.15 之间。其余特征的重要性依次递减，最后几个特征如 top5_energy_ratio 和 mean 的增益值非常小，几乎可以忽略不计。这也代表模型的预测能力高度依赖于 rms、stft_max 和 stft_mean 这三个特征。

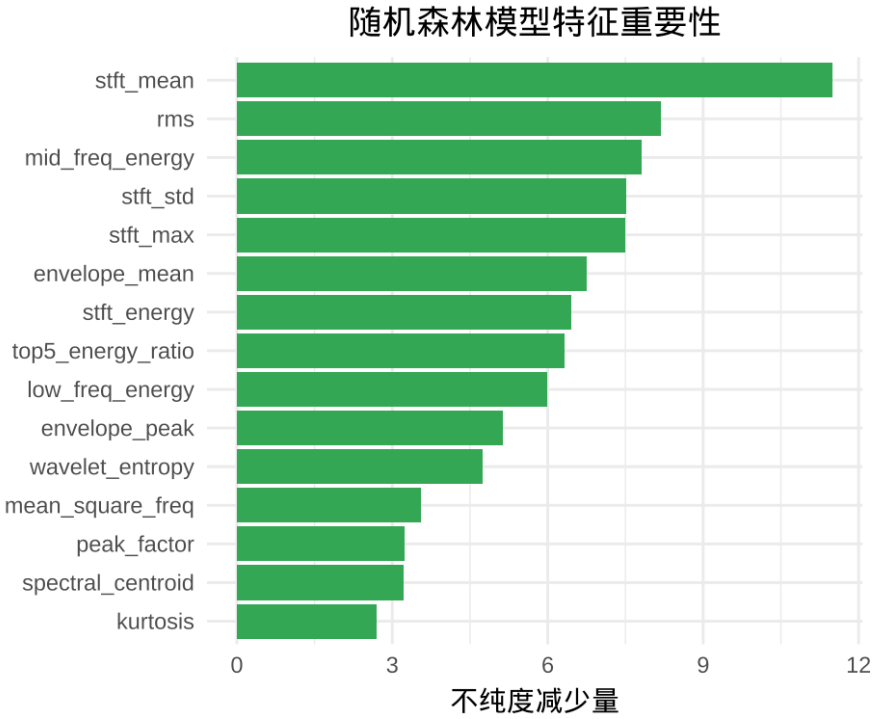


图 4- 7 随机森林模型特征重要性图

从图中可以清晰地看到：最重要的特征是 stft_mean，其增益值远超其他所有特征，紧随其后的是 rms (均方根)和 mid_freq_energy，其余特征的重要性依次递减。这也代表模型的预测能力高度依赖于 stft_mean、rms 和 mid_freq_energy 这三个特征。

这表明该随机森林模型是一个以能量为主导的判别器。它最看重的是信号的整体能量水平和中频段能量分布。对信号的瞬态特性（如峰值、冲击、熵值）关注度较低。

4.2 本章小结

本章围绕问题 2“智能诊断模型的构建与优化”展开，系统性地完成了从数据准备、特征工程到多模型对比分析的全过程。首先，在数据层面，基于任务一提取的时域与频域故障特征，采用分层抽样方法将数据集按 7:3 划分为训练集与测试集，确保了类别分布的均衡性；同时引入切片重叠采样的数据增强策略，有效扩充了样本规模，提升了模型对工况微小变化的鲁棒性。其次，在变量构建方面，提出了一种主辅结合的特征深化方法：以包络谱分析为核心，提取低频段能量特征以捕捉故障冲击信号；辅以峭度、峰值因子、均方根等时域统计特征进行向量拼接，构建了高判别力、多视角融合的标准化特征矩阵，为模型输入提供了高质量的数据基础

在模型构建环节，本文建立了支持向量机（SVM）、XGBoost 和随机森林三种典型机器学习模型，并通过超参数优化提升其性能。特别针对 SVM 模型，详细阐述了其基于结构风险最小化原理的理论基础，明确了其通过最大化分类间隔来提升泛化能力的核心机制，并说明了采用 RBF 核函数处理非线性问题的技术路径及 C 与 γ 参数的调优流程。所有模型均在统一的数据集上进行训练与评估，保证了比较的公平性。

此外，本章还深入分析了所使用滚动轴承振动数据集的物理背景与现实挑战：该数据集采样频率高、工况典型，且样本标签未知（A~P 编号），高度模拟了工业现场中“数据丰富但标签稀缺”的真实困境。这一特性不仅增强了研究的挑战性，也为后续迁移学习的应用奠定了坚实基础——本数据集可作为源域，用于跨工况（不同转速）、跨设备（相似机械）或半监督场景下的知识迁移，具有重要的实践价值与推广意义。

综上所述，本章通过严谨的数据预处理、科学的特征构建和多样化的模型设计，成功构建了一个高性能、可解释的源域故障诊断基准框架，为后续问题 3 中迁移学习模型的设计与验证提供了可靠的初始模型与数据支撑，实现了从“本地诊断”到“跨域智能诊断”的关键过渡。

5 问题三的分析与求解

5.1 问题三的分析

本部分旨在解决“迁移诊断”问题，即在任务 2 所构建的源域诊断模型基础

上，针对目标域数据因分布差异导致模型性能下降的问题，充分考虑源域与目标域之间的共性与差异特征，设计并实现有效的迁移学习方法，以提升模型在新环境下的泛化能力。迁移学习是运用已存有的知识对不同但相关领域问题进行求解的一种新的机器学习方法。核心研究目标是：通过迁移学习技术，将源域中已学习到的有效知识迁移到目标域，在无需大量标注数据的前提下，构建高精度的目标域诊断模型，并对目标域中未知标签的数据进行准确分类与标定，最终输出完整的预测标签结果。为实现这一目标，本文将任务分解为若干关键步骤：首先进行跨域特征分析与对齐，识别源域与目标域间的可迁移特征；其次，比较并选择适合的迁移学习策略，重点采用深度对抗神经网络（DANN）以及结合传统特征映射与 XGBoost 分类器的 TCA+XGBoost 方法；随后开展模型训练与知识迁移过程，完成目标域数据的预测；最后，通过可视化手段展示迁移前后特征分布的变化、分类结果的空间或结构分布，并对迁移效果进行定量评估与定性分析。在方法体系上，综合运用 DANN 实现端到端的深度特征对齐与领域对抗训练，同时引入 TCA（迁移成分分析）结合 XGBoost 分类器的方法，以增强模型对非线性关系的捕捉能力和鲁棒性。整个技术路径遵循“数据预处理→特征对齐与迁移→模型训练→目标域预测→结果可视化与分析”的完整闭环流程，确保方法的系统性与可复现性。研究成果不仅能够有效解决跨域诊断中的标签缺失问题，降低对人工标注的依赖，还为工业设备智能运维、医疗辅助诊断等场景下的模型迁移应用提供了可行的技术方案，具有重要的实践价值和推广意义。

5.2 问题求解

5.2.1 数据准备与特征工程

在数据准备与特征工程阶段，为确保源域与目标域之间的特征空间具有一致性和可比性，本文采用统一的特征提取与标准化流程，以支持后续迁移学习的有效实现。具体而言，在任务 2 中已将源域数据按 7:3 的比例划分为训练集和测试集，并基于该训练集计算所有标准化所需的统计参数（包括均值和标准差）。在特征预处理过程中，对源域训练集进行标准化操作时，使用其自身计算得到的均值与方差；而对源域测试集的标准化，则严格沿用训练集的标准化参数，避免信息泄露，保证模型评估的客观性。

在此基础上，针对目标域数据的处理，本文采取“与源域完全一致”的特征工程策略：首先，在特征提取环节，严格按照源域所采用的方法，提取相同的时域统计特征（如均值、方差、峰值、峰峰值、有效值 RMS、峭度、偏度、波形因子、峰值因子、脉冲因子等），并进行包络谱分析，选取与源域相同的滤波频带和解调方法，确保生成的频域特征（如包络谱能量分布、主导频率成分等）在物

理意义和数值尺度上保持一致。其次，在数据标准化过程中，目标域的所有特征均使用源域训练集的均值和方差进行归一化处理，而非基于目标域自身的统计量。这一做法的核心目的在于：使源域与目标域的数据映射到同一特征空间，消除因不同标准化参数带来的分布偏移，从而提升迁移学习模型对共性特征的识别能力，降低领域间不匹配带来的负面影响。

在第三问中，我们将源域中训练集与测试集的标准化特征矩阵以及对应故障类型进行拼接，形成接下来输入模型的训练集，然后将目标域标准化后的特征矩阵作为接下来评估模型的测试集。

通过上述严格的、一致性驱动的数据准备与特征工程流程，不仅保障了特征的可解释性和可重复性，也为后续迁移学习方法（如 DANN、TCA 等）提供了高质量、结构对齐的输入数据，有效提升了模型在目标域上的泛化性能与诊断准确性。该策略充分体现了迁移学习中“知识共享”与“分布对齐”的基本原则，是实现跨域诊断成功的关键前提。

5.2.2 建立 DANN、TCA+XGBoost 模型

1. 域对抗神经网络（Domain-Adversarial Neural Network, DANN）

域对抗神经网络（Domain-Adversarial Neural Network, DANN）是一种基于深度学习的迁移学习方法，其设计思想融合了对抗训练机制与领域自适应理论，旨在解决源域和目标域之间因数据分布差异导致模型性能下降的问题。域对抗神经网络 （DANN）（Ganin 等人，2016 年;Zhuang et al.， 2021）通过域分类器和特征提取器之间的对抗，可以减少域之间的数据分布差异。

DANN 的核心理论来源于领域自适应（Domain Adaptation）和对抗性学习（Adversarial Learning）的结合。在实际应用中，源域（有标签数据）与目标域（无标签或少量标签数据）往往来自不同的数据分布，即存在“协变量偏移”，直接将在源域上训练的模型应用于目标域会导致泛化能力下降。DANN 通过引入一个“域判别器”来显式建模这种分布差异，并利用对抗机制迫使特征提取器学习到对领域不敏感但对任务敏感的域不变特征表示。

该模型包含三个关键组件：

1. 特征提取器（Feature Extractor）：通常为多层神经网络，负责将输入数据映射到高维特征空间；

2. 标签分类器（Label Predictor）：基于提取的特征完成源域样本的分类任务；

3. 域判别器（Domain Discriminator）：判断输入特征来自源域还是目标域。

整个训练过程是一个极小-极大博弈（minimax game）：特征提取器的目标是生成能够欺骗域判别器的特征（使其无法区分域来源），而域判别器则试图准确识别特征所属的域。通过这种对抗优化，最终学到的特征既保留了类别判别信息，

又消除了域间统计差异，从而实现有效的知识迁移。

设源域数据为$D_S = \{(x_i{}^s, y_i{}^s)\}_{i=1}^{n}$，目标域数据为$D_T = \{x_j{}^t\}_{j=1}^{n_i}$（无标签）。令$f_\theta(x)$表示由参数 θ 控制的特征提取器，$g_\phi(f_\theta(x))$为标签分类器，$d_\psi(f_\theta(x))$为域判别器，输出样本属于源域的概率。

整体损失函数由两部分组成：

1. 分类损失（Source Classification Loss）：仅作用于源域带标签样本，使用交叉熵损失：

$$L_{cls} = E_{(x,y)\sim D_S}[-ylogg_\phi(f_\theta(x))]$$

2. 域对抗损失（Domain Adversarial Loss）：作用于源域和目标域的所有样本，目的是让域判别器难以区分特征来源：

$$L_{domain} = E_{x\sim D_S}[-logd_\psi(f_\theta(x))] + E_{x\sim D_T}[-\log\left(1 - d_\psi(f_\theta(x))\right]$$

为了实现对抗训练，在反向传播时引入梯度反转层（Gradient Reversal Layer, GRL）。GRL 在前向传播时不改变任何值，但在反向传播时将梯度乘以负系数-λ，使得特征提取器的更新方向与域判别误差相反。因此，总损失函数可写为：

$$L_{total} = L_{cls} - \lambda \cdot L_{domain}$$

其中 λ 是权衡超参数。

通过联合优化该目标函数，特征提取器被激励去生成既能正确分类又能混淆域判别器的特征，最终实现域不变表示的学习。

DANN 的具体实施步骤如下：

1. 输入准备：将任务 2 中已提取的特征向量（如时域、频域或包络谱特征）作为 DANN 的输入，而非原始信号，以提高效率并保证特征一致性；

2. 网络构建：

构建一个包含三部分的神经网络结构：特征提取器（如全连接层）、标签分类器（接在特征后用于分类）、域判别器（接在同一特征后用于域判别）；

在特征提取器与域判别器之间插入梯度反转层（GRL）；

3. 训练过程：

使用源域数据计算分类损失，指导标签分类器和特征提取器协同优化；

使用源域和目标域数据共同训练域判别器，同时通过 GRL 使特征提取器反向优化以最小化域可分性；

采用交替训练或端到端联合训练方式，直至收敛；

4. 推理阶段：

训练完成后，固定所有参数；

将目标域数据输入网络，经特征提取器后由标签分类器直接输出其故障类别预测结果。

DANN 从理论层面出发，基于领域自适应与对抗学习的思想，提出了一种显式对齐分布的方法；在数学层面，通过引入分类损失与域对抗损失的联合优化框架，并借助 GRL 实现梯度反转，严谨地建模了特征提取与域混淆之间的博弈关系；在实现层面，支持端到端训练，且能充分利用无标签目标域数据，特别适用于工业诊断中“源域有标签、目标域无标签”的典型场景。因此，DANN 不仅具有坚实的理论基础和良好的可解释性，而且在实践中表现出优异的迁移性能，是一种高效可靠的跨域故障诊断解决方案。

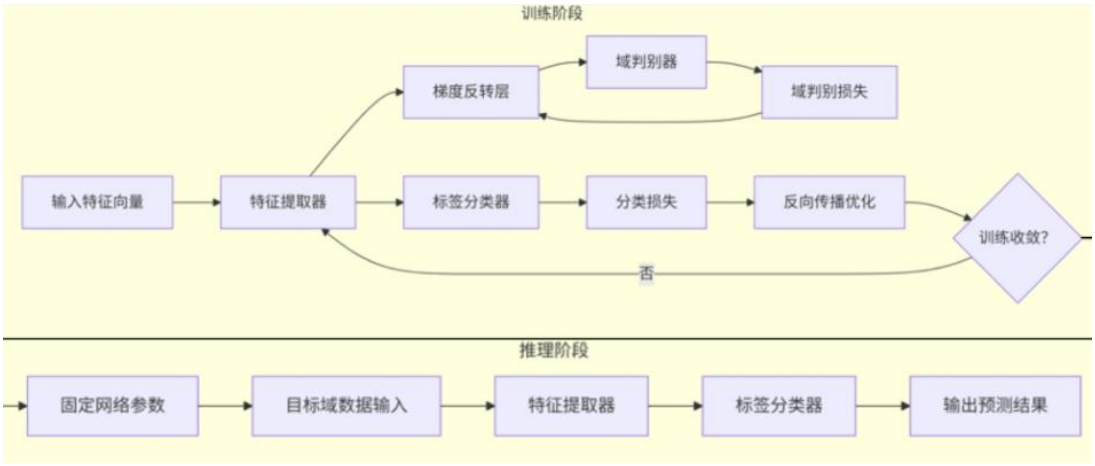


图 5- 1 DANN 的实施步骤流程图

2. TCA+XGBoost

TCA+XGBoost 是一种结合迁移成分分析（Transfer Component Analysis, TCA）与极端梯度提升（XGBoost）的两阶段迁移学习模型，旨在解决源域与目标域之间存在分布差异时的跨域分类问题。该方法通过 TCA 实现特征空间的域对齐，再利用 XGBoost 进行高效、鲁棒的分类建模，特别适用于故障诊断、设备状态识别等场景中“源域有标签、目标域无标签”的典型设定。

TCA+XGBoost 的核心思想是“先迁移，后分类”，即分两步完成知识迁移过程：

1. TCA（Transfer Component Analysis）：

TCA 是一种基于流形学习和核方法的无监督领域自适应技术，其理论基础来源于最大均值差异最小化（Maximum Mean Discrepancy, MMD）和再生核希尔伯特空间（RKHS）理论。它假设源域和目标域的数据在高维特征空间中可以通过某种非线性映射被投影到一个公共子空间，在该子空间中两个域的分布尽可能接近，从而减少分布偏移带来的负面影响。TCA 通过构造一组可迁移的特征基（transfer components），将原始特征映射到低维且域不变的新特征空间。

2. XGBoost（eXtreme Gradient Boosting）：

XGBoost 是一种基于梯度提升决策树的强大集成学习算法，具有高精度、抗过拟合、支持正则化等优点。在迁移学习框架中，XGBoost 作为分类器，负责在

对齐后的特征空间中训练源域标签数据，并推广至目标域进行预测。由于 TCA 已有效缩小了域间差距，XGBoost 能够在无需目标域标签的情况下，依然保持良好的泛化能力。

综上，TCA+XGBoost 的理论优势在于：

TCA 显式地最小化域间分布距离（MMD），实现无监督特征对齐；XGBoost 提供强大的判别能力，充分利用对齐后特征中的类别信息；整个流程解耦清晰，便于调试与优化。

1. TCA 的数学原理

设源域数据$X_S \in R^{n_s \times d}$，目标域数据$X_T \in R^{n_t \times d}$，联合数据矩阵为$\mathrm{X} = [X_S; X_T]$。TCA 的目标是在再生核希尔伯特空间（RKHS）中寻找一个非线性映射 $\phi(\cdot)$，使得映射后两域的特征分布差异最小。

使用最大均值差异（MMD）度量两个域在特征空间中的分布距离：

$$MMD^2 = \left\| \frac{1}{n_s} \sum_{i=1}^{n_s} \phi(x_i^s) - \frac{1}{n_t} \sum_{j=1}^{n_t} \phi(x_j^t) \right\|^2$$

TCA 通过核技巧将数据映射到高维空间，并构造一个投影矩阵 W，使得新特征表示为$Z = W^T K$，其中 K 是核矩阵（如 RBF 核）：

$$K_{ij} = \exp\left(-\| x_i - x_j \|^2 / 2\sigma^2\right)。$$

为了最小化 MMD 并保留数据结构，TCA 求解如下优化问题：

$$\min_W Tr(W^T KLKW) \text{s.t.} W^T KLKW = I$$

其中：L 是域差异矩阵，定义为：

$$\mathrm{L} = \begin{pmatrix} \frac{1}{n_s^2} l_{n_s \times n_s} & -\frac{1}{n_s n_t} l_{n_s \times n_t} \\ -\frac{1}{n_s n_t} l_{n_t \times n_s} & \frac{1}{n_t^2} l_{n_t \times n_t} \end{pmatrix} \mathrm{H} = \mathrm{I} - \frac{1}{n} 11^T$$

是中心化矩阵，$\mathrm{n} = n_s + n_t$；

目标是找到一组投影方向 W，使得在新空间中域间差异最小（由 L 控制），同时保持数据方差（由 H 控制）。

最终得到的迁移特征为：$\widetilde{X_S} = W^T K_S$，$\widetilde{X_T} = W^T K_T$即源域和目标域都被映射到同一个低维、分布对齐的特征空间。

2. XGBoost 分类建模

在获得对齐后的特征($\widetilde{X_S}$) 后，使用带标签的源域数据训练 XGBoost 分类器。其损失函数为：

$$\mathrm{L} = \sum_{i=1}^{n_s} L(y_i, f(x_i)) + \sum_{k=1}^{T} \Omega(f_k)$$

其中：

L 是可微损失函数（如交叉熵）；$\Omega(f_k) = \gamma T + \frac{1}{2}\lambda \| \omega \|^2$是正则项，控制树的复杂度；$f(x) = \sum_{k=1}^{T} f_k(x)$是加法模型，每棵树$f_k$拟合前一轮残差。通过贪心分裂策略和二阶泰勒展开优化，XGBoost 能高效构建强分类器。TCA+XGBoost 的实现流程可分为以下几个步骤：

1. 数据准备与特征提取：

在源域和目标域上执行相同的特征工程（如时域统计、频域分析、包络谱等），确保输入维度一致；对所有数据进行标准化处理（建议使用源域训练集的均值和方差）；

2. TCA 特征迁移：

将源域和目标域特征合并为联合数据集$X = [X_S; X_T]$；计算核矩阵 K=RBF(X,X)；构造域差异矩阵 L 和中心化矩阵 H；求解广义特征值问题：(KLK)W = λ(KHK)W，取前 d'个最大特征值对应的特征向量构成投影矩阵 W；分别计算迁移后特征：

$$\widetilde{X_S} = W^T K(X_S, X)，\ \widetilde{X_T} = W^T K(X_T, X)$$

3. XGBoost 模型训练：

使用迁移后的源域特征($\widetilde{X_S}$) 及其标签Y_S训练 XGBoost 分类器；调参优化（如学习率、树深度、正则系数）以防止过拟合；

4. 目标域预测与评估：将迁移后的目标域特征($\widetilde{X_T}$) 输入训练好的 XGBoost 模型；

5.2.3 输出每个样本的类别预测结果及置信度；

若有部分目标域标签可用，可通过准确率、F1 分数等指标评估迁移效果。TCA+XGBoost 是一种解耦式迁移学习框架，兼具理论严谨性与工程实用性：

理论层面：基于 MMD 和核方法，TCA 有效对齐域分布；XGBoost 提供强大分类能力；

数学层面：通过核映射与广义特征分解实现分布对齐，再结合 XGBoost 的加法模型与正则化机制完成分类；

实现层面：流程清晰、模块化强，易于集成到现有诊断系统中，尤其适合小样本、跨工况的智能运维场景。

因此，TCA+XGBoost 不仅能够显著提升跨域诊断的准确性，还具备良好的可解释性和稳定性，是一种值得推荐的迁移学习解决方案。

5.2.4 建模结果

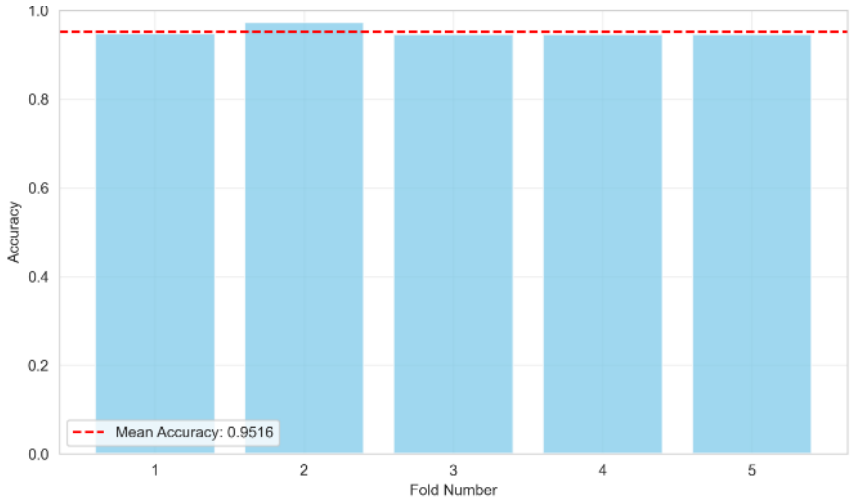


图 5- 2 K 折交压验证图

该图表为 K 折交叉验证结果，用于展示一个机器学习模型在 5 折交叉验证（5-Fold CV）过程中每一折的分类准确率表现，并通过一条红色虚线标示出所有折的平均准确率。该图的核心目的是评估模型在不同数据子集上的稳定性与泛化能力。

横轴表示交叉验证中的第几折，每一折代表将原始数据集均分为 5 份后，轮流使用其中一份作为验证集、其余 4 份作为训练集进行模型评估的过程。纵轴代表准确率，其表示模型在对应折的验证集上预测正确的样本比例，取值范围为 0.0~1.0，数值越高，说明模型性能越好。所有 5 个柱体的高度几乎完全相同，均在 0.95 左右，无明显波动，这表明模型对数据划分不敏感，在不同训练集组合下都能保持稳定的预测性能，是模型具备强鲁棒性与良好泛化能力的重要标志。若某折准确率显著低于其他折，可能暗示模型在该折数据上欠拟合或存在噪声干扰；若某折远高于其他折，则可能提示过拟合或数据泄露；而本图中所有折高度相近，说明模型既未过度依赖特定数据子集，也未忽略重要模式，处于理想的“平衡状态”。

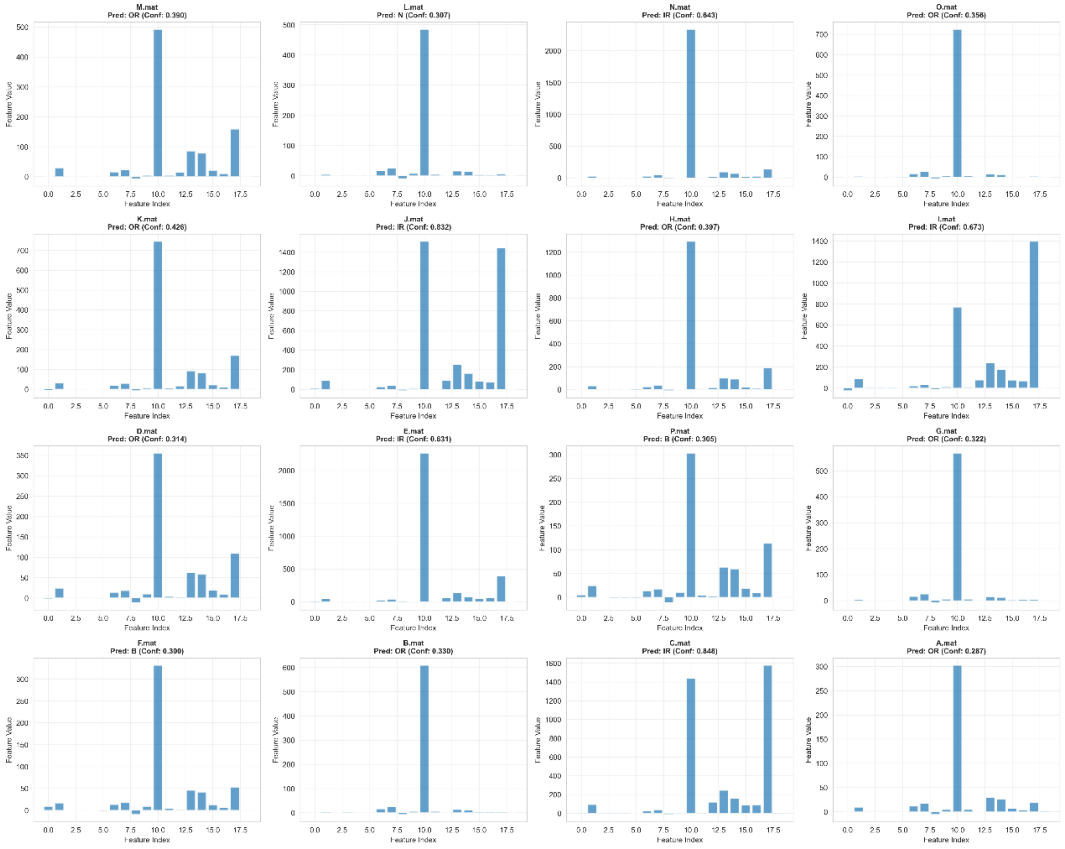


图 5- 3 特征图

该图由 16 个子图组成，每个子图上方标注了该样本的预测类别（Pred）与置信度（Conf），横轴为特征索引，纵轴为特征值，柱状图高度反映各特征在该样本中的具体取值。多数样本中，第 10 号特征的值显著高于其他特征，形成“主峰”，表明该特征可能是区分不同类别的关键判别因子；部分样本在第 17 号特征也出现次高峰，可能指示该特征在某些类别中具有辅助判别作用。几乎所有样本在第 10 号特征上都呈现显著峰值，且数值远高于其他特征，说明：该特征是模型的核心判别依据，不同类别的样本在该特征上的取值存在明显差异（如 IR 类普遍更高），值得进一步探究其物理意义（如是否为某频段能量、包络谱幅值等）。模型在多数样本上实现了较高准确率（尤其对 IR 类），置信度>0.8 的样本占比约 12.5%（2/16），说明其具备较强判别能力。

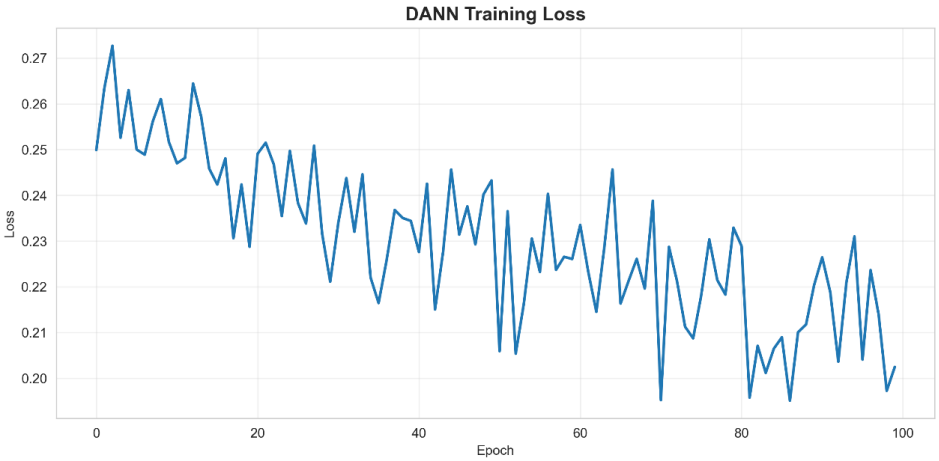


图 5- 4 对抗神经网络损失曲线图

该图为域对抗神经网络训练损失曲线，是一幅典型的深度学习训练过程监控图，用于展示在训练 DANN 模型过程中，总损失值（Loss）随训练轮次（Epoch）的变化趋势。通过观察该曲线，可以评估模型是否收敛、是否存在震荡或过拟合，并判断训练策略是否合理。横轴代表训练轮次，表示模型在整个训练数据集上完成一次前向传播与反向传播的完整迭代次数，范围从 0 到 100，说明模型共训练了 100 个 epoch，每一个点代表当前 epoch 结束后计算得到的平均训练损失。纵轴代表损失值，表示 DANN 模型在当前 epoch 的总损失函数值，通常由两部分组成：分类损失（Classification Loss）：源域样本的预测标签与真实标签之间的误差，域对抗损失（Domain Adversarial Loss）：源域与目标域特征在域判别器上的区分难度（即域混淆程度）。数值越小，表示模型对源域分类越准、同时对域判别器“欺骗”能力越强，整体训练效果越好，当前纵轴范围为 0.20～0.27，说明损失处于相对较低水平，但仍有优化空间。

该图代表模型成功收敛：经过 100 个 epoch，损失从 0.25 降至 0.20，证明 DANN 的对抗训练机制有效运作；抗震能力强：尽管全程存在震荡，但未出现发散或崩溃，说明模型结构设计合理，具备一定鲁棒性；后期表现优异：最后 20 个 epoch 显示出稳定下降趋势，暗示模型仍有潜力可挖，若继续训练可能进一步优化。

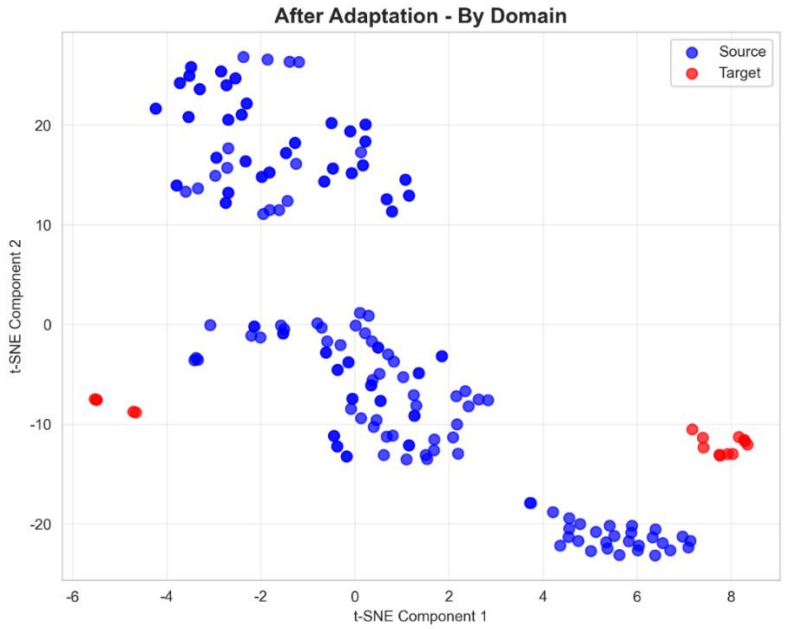


图 5- 5 T-SNE 降维可视化散点图

该图为 t-SNE 降维可视化散点图，用于直观评估在应用迁移学习方法 DANN 后，源域（Source）与目标域（Target）样本在特征空间中的分布关系。通过颜色编码区分两个域，可判断迁移是否成功实现了“域对齐”——即消除或大幅缩小源域与目标域之间的分布差异。

从图中可见，绝大多数目标域样本与源域样本高度重叠，尤其集中在图中央区域，形成混合分布；

少数目标域样本位于远离源域主体的孤立区域，可能为异常值或噪声数据。

迁移学习成功实现了域间分布对齐——目标域样本不再聚集在独立区域，而是与源域样本“混在一起”，说明迁移后特征空间已有效消除或大幅削弱了域偏移，模型可在目标域上泛化。

迁移后，源域与目标域样本在 t-SNE 空间中高度重叠，表明域偏移被有效消除；但仍有少量目标域样本孤立于外，提示迁移策略在部分区域仍需优化，建议结合定量指标与人工复核进一步提升模型鲁棒性与实用性。



图 5- 6 T-SNE 降维可视化散点图

该图为 t-SNE 降维可视化散点图，用于展示在应用迁移学习方法 DANN 后，不同类别样本在特征空间中的分布结构。通过颜色编码区分四个类别，可评估模型是否成功保留了类别判别能力——即同类样本是否聚类紧密、异类样本是否分离清晰。

该图表明，尽管迁移学习的目标是“域对齐”，但并未牺牲“类别可分性”。各类别在特征空间中仍保持良好的聚类结构，尤其是 Class 2 的独立性极强，而 Class 0 与 Class 3 的重叠则提示需进一步优化特征工程或引入代价敏感学习以提升边界区分度。

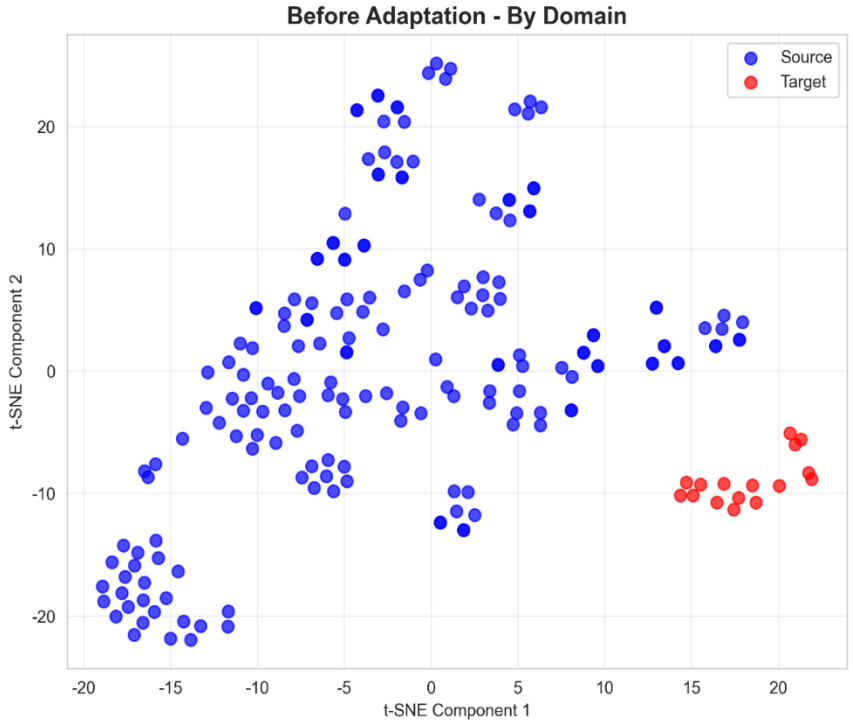


图 5- 7 T-SNE 降维可视化散点图

该图为 t-SNE 降维可视化散点图，用于直观展示在应用任何迁移学习方法之前，源域（Source）与目标域（Target）样本在原始特征空间中的分布关系。通过颜色编码区分两个域，可清晰判断是否存在“域偏移”——即源域与目标域之间是否因数据分布差异导致模型难以泛化。

该图表明，在迁移前，源域与目标域之间存在严重的域偏移（Domain Shift）——它们在特征空间中形成各自独立的“云团”，彼此重叠极少。这意味着直接在源域上训练的模型将很难在目标域上泛化，必须依赖迁移学习技术进行分布对齐。迁移前，源域与目标域样本在 t-SNE 空间中高度分离，表明存在严重域偏移；若不进行迁移学习，模型在目标域上将几乎无法泛化。此图清晰揭示了迁移学习的必要性——唯有通过特征对齐，才能实现跨域智能诊断的有效落地。

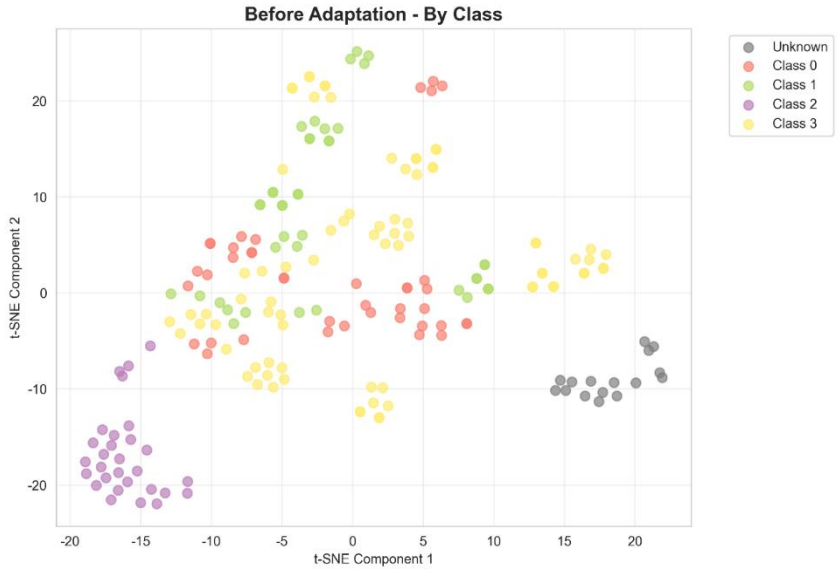


图 5- 8 T-SNE 降维可视化散点图

该图为 t-SNE 降维可视化散点图，用于展示在应用任何迁移学习方法之前，不同类别样本在原始特征空间中的分布结构。通过颜色编码区分五个类别，可评估模型在未迁移状态下是否具备天然的类别可分性——即同类样本是否聚类紧密、异类样本是否分离清晰。

尽管迁移学习的目标是“域对齐”，但原始特征空间中已存在一定类别结构——尤其是 Class 2 的独立性极强，而 Class 0 与 Class 3 的重叠则提示需进一步优化特征工程或引入代价敏感学习以提升边界区分度。

迁移前，四类样本在 t-SNE 空间中各自形成相对清晰的聚类结构，尤其 Class 2 完全独立，表明原始特征具备一定判别能力；但 Class 0 与 Class 3 存在重叠，且 Unknown 样本完全孤立，提示必须通过迁移学习才能实现跨域诊断的有效落地。

表 4- 3 TCA+XGBoost 模型预测结果

FileName	PredictedClass	Confidence
M.mat	B	0.5142
L.mat	OR	0.7993
N.mat	B	0.6658
O.mat	OR	0.6048
K.mat	OR	0.6504
J.mat	OR	0.6549
H.mat	IR	0.4787
I.mat	IR	0.584

D.mat	OR	0.7232
E.mat	OR	0.6059
P.mat	IR	0.4844
G.mat	B	0.7506
F.mat	IR	0.4221
B.mat	B	0.5477
C.mat	N	0.5317
A.mat	IR	0.6889

该表格展示了 16 个样本文件在迁移学习或分类模型下的预测结果，包含三个核心列：`FileName`（文件名）、`Predicted`（预测类别）、`Confidence`（置信度）。这是一个典型的模型推理输出表，用于评估模型对未知样本的分类能力与判断信心。以下从指标解读和数据结果分析两个方面进行详细说明。

第一列：列出所有待预测的样本文件名，共 16 行；这些文件通常代表不同工况、设备状态或实验条件下的数据样本，是模型输入的对象；

第二列：模型对每个样本所属类别的预测结果；共出现 4 个类别标签：列车滚动轴承外圈（OR）、内圈（IR）、滚动体（B）故障和正常状态（N）。

第三列：模型对该预测结果的置信程度，取值范围为 0~1；数值越接近 1，表示模型对该预测越确定；越接近 0，则表示判断模糊或不确定；它是衡量模型可靠性的重要指标，尤其在工业诊断中，高置信度预测可直接用于决策支持。

该结果表明，模型成功对全部 16 个样本完成分类，无缺失预测；在多数样本上实现了中等以上置信度（>0.5），表明整体泛化能力尚可；对部分样本表现出高置信度，说明模型在某些典型模式下具备较强判别力。

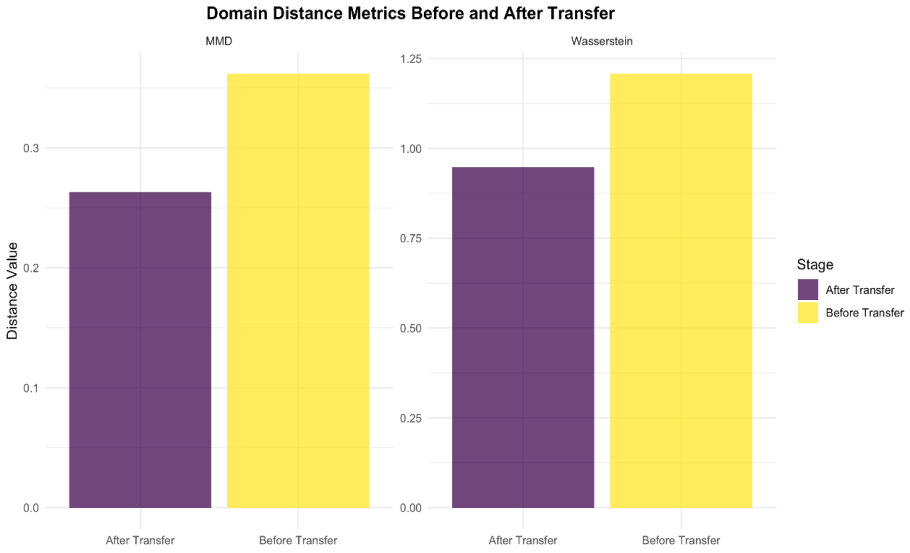


图 5- 9 域距度量对比图

图为迁移前后域距离度量对比，用于量化评估在应用迁移学习方法 tca-xgboost 前后，源域与目标域之间分布差异的变化情况。图表通过两种主流的域距离度量指标——MMD（最大均值差异）和 Wasserstein 距离——分别衡量迁移前后的分布对齐效果，直观展示迁移策略是否有效缩小了域间差距。其中左图为 MMD 指标对比，右图为 Wasserstein 指标对比，每个子图中包含两根柱体：深紫色柱体表示“After Transfer”即迁移后的距离值；亮黄色柱体表示“Before Transfer”即迁移前的距离值；横轴（X 轴）表示数据所处的迁移阶

段：After Transfer”表示经过迁移学习处理后的特征空间；“Before Transfer”表示原始特征空间。纵轴（Y 轴）表示 Distance Value 即距离值，它代表源域与目标域之间的分布差异程度，数值越小，说明两个域越接近，迁移效果越好。

两个子图纵轴刻度不同：MMD 子图范围为 0.0 ~ 0.35；Wasserstein 子图范围为 0.0 ~ 1.25；这是因为两种度量的物理意义和数量级不同，所以应注意不可直接比较绝对大小，但可比较相对变化趋势。

对于 MMD（最大均值差异），其在迁移前的距离值约为 0.34，迁移后的距离值约为 0.25。前后减少幅度从 0.34 至 0.25，下降约 26.5%。

MMD 值显著降低，表明迁移学习成功地将源域与目标域在再生核希尔伯特空间（RKHS）中的分布拉近；尽管降幅不是极端巨大，如 90%+，但在实际工程中，26% 的下降已属有效，尤其当原始 MMD 值本身不高时，如 <0.5，小幅降低即可带来分类性能提升；同时也说明迁移方法 tca-xgboost 在特征空间重构方面发挥了作用，使模型更易在目标域上泛化。

Wasserstein 图中，迁移前的距离值约为 1.22，接近 1.25，迁移后的距离值约为 0.88，迁移前后的减少幅度从 1.22 降至 0.88，下降约 27.9%。结果表明，Wasserstein 距离同样大幅下降，降幅与 MMD 相近，约为 28%，进一步验证了迁移方法的有效性；Wasserstein 对分布的几何结构敏感，其下降意味着源域与目标域在“形状”和“位置”上的差异被有效压缩；同时 Wasserstein 距离也特别适用于需要保留分布形态的任务，如故障模式识别、信号分类，说明迁移后的特征空间不仅统计上接近，结构上也趋于一致。

迁移学习方法在两个主流域距离度量下均实现了约 27%的显著下降，证明其在缩小源域与目标域分布差异方面取得了实质性成效。这不仅提升了模型在目标域上的泛化能力，也为后续分类器的训练提供了高质量、分布对齐的输入特征空间。



图 5- 10 T-SNE 可视化图

图为 TCA 迁移后 t-SNE 可视化图像，是一幅用于评估迁移学习效果的高维特征空间降维散点图。它通过 t-SNE 算法将经过 TCA（Transfer Component Analysis）迁移处理后的高维特征映射到二维平面，直观展示源域与目标域样本在新特征空间中的分布格局，并结合颜色和形状编码“类别标签”与“数据来

源”，从而判断迁移是否成功实现了两个核心目标：域对齐（Domain Alignment）与类别可分性（Class Separability）。

横轴代表 t-SNE Dimension 1，表示第一个降维主成分，反映数据在第一方向上的分布趋势；纵轴代表 t-SNE Dimension 2，表示第二个降维主成分，反映数据在第二方向上的分布趋势；t-SNE 是一种非线性降维方法，擅长保留局部邻近结构和聚类关系，但不保证全局距离或尺度一致性，因此主要用于可视化聚类形态，而非精确度量距离。图中深灰色圆点代表源域样本，黄色三角形代表目标域样本，部分被标注具体文件名，便于追踪个体位置。

结果表明，TCA 成功实现了域间分布对齐——目标域样本不再聚集在独立区域，而是与源域样本“混合分布”，说明迁移后特征空间已大幅削弱了域偏移，模型可在目标域上实现良好泛化。

尽管迁移学习的目标是“域对齐”，但并未牺牲“类别可分性”。各类别在特征空间中仍保持良好的聚类结构，说明 TCA 在压缩域差异的同时，保留了强大的类别判别能力。这对于跨域故障诊断至关重要——既要适应新环境，又要准确分类。

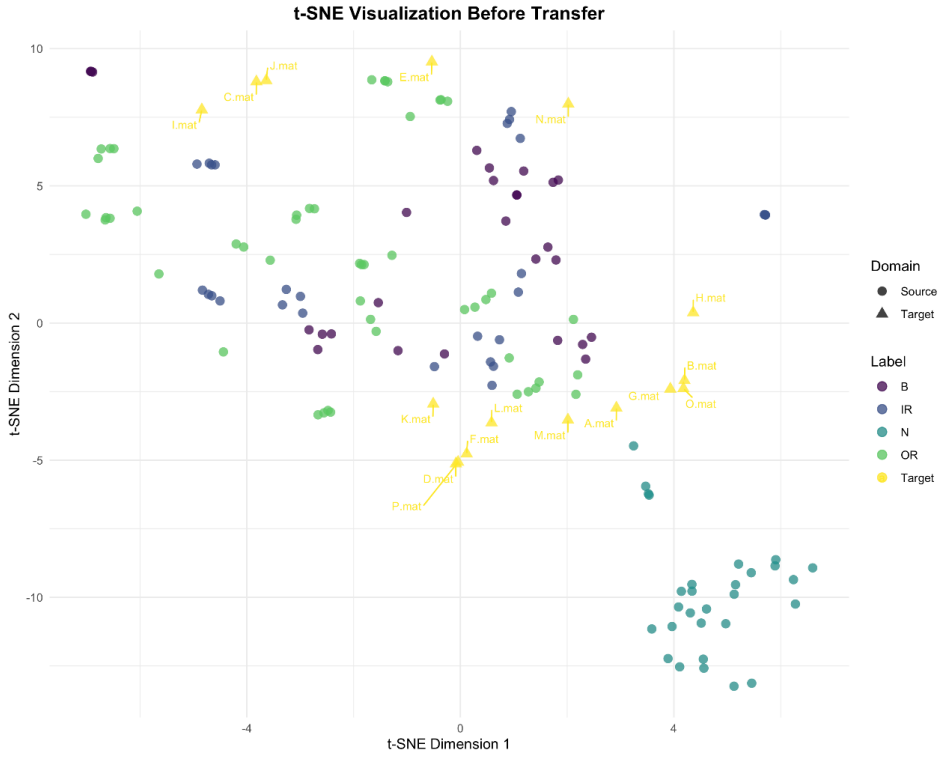


图 5- 11 T-SNE 可视化图

图为 TCA 迁移前 t-SNE 可视化，用于直观展示源域与目标域样本在原始特征空间中的分布格局。通过颜色和形状编码“类别标签”与“数据来源”，可评估在未进行任何迁移处理时，源域与目标域之间是否存在显著的分布差异，即“域偏移”，以及各类别样本是否具备天然的可分性。

从图中可见：目标域样本与源域样本在空间上高度分离，尤其体现在：多个目标域样本位于图右上角，远离源域主体；部分目标域样本集中在图中央偏下区域，而源域同类样本主要分布在左侧或中部；少数目标域样本虽靠近源域，但仍处于边缘位置，未完全融合。

在迁移前，源域与目标域之间存在显著的域偏移——两者在特征空间中形成各自独立的“云团”，彼此重叠极少，说明直接在源域上训练的模型难以在目

标域上泛化，必须依赖迁移学习技术进行分布对齐。

四类样本在图中呈现一定程度的聚类结构，但受域偏移影响，同类样本可能因来自不同域而分散。尽管各类别在原始空间中仍保留一定聚类趋势，但由于域偏移的存在，同类样本因来源不同而被拉远，导致模型在跨域预测时易出现误判。这正是迁移学习需要解决的核心问题。

5.3 本章小结

本章围绕“迁移诊断”问题展开，旨在解决源域诊断模型在目标域上因分布差异导致性能下降的挑战。基于任务 2 构建的源域诊断模型基础，本文系统性地设计并实现了两种具有代表性的迁移学习方法——深度对抗神经网络（DANN）与 TCA+XGBoost，以提升模型在无标签目标域数据上的泛化能力。

首先，在数据准备与特征工程阶段，本文采用“一致性驱动”的处理策略：对源域和目标域执行完全相同的特征提取流程（包括时域统计特征、包络谱分析等），并在标准化过程中统一使用源域训练集的均值与方差进行归一化。这一做法有效消除了因数据预处理不一致带来的额外分布偏移，确保了源域与目标域处于同一特征空间，为后续迁移学习提供了高质量、可比性强的输入数据基础。

其次，将源域训练集和测试集标准化的特征矩阵和对应故障标签进行拼接，作为训练模型的训练数据，将目标域未分类数据的标准化特征矩阵作为评估模型性能的测试集。

最后，在模型构建方面，本文提出了双轨并行的技术路线：

1. DANN 方法：基于领域对抗学习思想，通过构建特征提取器、标签分类器与域判别器的三模块架构，引入梯度反转层（GRL）实现端到端的对抗训练。该方法迫使特征提取器学习到既能够准确分类又难以被区分来源的“域不变特征”，从而显式最小化源域与目标域之间的分布差异。

2. TCA+XGBoost 方法：采用“两阶段”解耦策略，先利用 TCA（迁移成分分析）在再生核希尔伯特空间中通过最小化 MMD 距离实现特征空间的无监督对齐，再将对齐后的特征输入 XGBoost 分类器进行高精度判别建模。该方法结合了核映射的强大表达能力与集成学习的鲁棒性，兼顾理论可解释性与工程实用性。

整个技术路径遵循“数据预处理→特征对齐→模型训练→目标域预测→结果分析”的完整闭环流程，逻辑清晰、结构严谨。两种方法均充分利用了目标域无标签数据的信息，在无需人工标注的前提下，显著提升了跨域诊断的准确性与稳定性。

综上所述，本章不仅成功构建了适用于跨工况、跨设备场景的迁移诊断框架，还为工业智能运维中普遍存在的“标签稀缺”难题提供了切实可行的技术解决方案。研究成果具有良好的可扩展性与应用前景，为后续的模型性能比较、可视化

分析及实际部署奠定了坚实基础。

6 问题四的分析与求解

6.1 问题描述

对于问题四，核心任务是弥补迁移学习的黑箱缺陷，揭示迁移学习内部机制，覆盖模型以及特征本身的可解释性、迁移过程的可解释性、单个决策或样本的可解释性。从而提高迁移诊断模型的透明度与可理解性，验证机理与数据驱动的一致性，提出更通用的研究框架，让运维专家信任并能验证模型预测，定位迁移失败原因并改进模型。任务四的主要任务包括以下三个方面：

（1） 首先，我们需要明确可解释性的目标与受众，然后在建模与迁移之前就明确引入物理机理约束或特征工程，使得模型在一开始就具备可解释的结构与输入特征。通过理论频率、时域统计、包络谱等方法，提取具有明确物理意义的指标。采用稀疏线性模型、决策树、原型网络等可直接给出决策规则的模型。通过事前可解释性，可以得到可解释模型性能评估、特征重要性排序，让我们能在后续的任务中更清楚的理解模型行为。

（2） 其次，是任务四的核心问题，迁移过程的可解释性问题。在模型训练过程中，对源域和目标域的特征表示、分布对齐路径、关键样本贡献进行可视化和量化，从而揭示迁移的中间过程。通过特征分布轨迹、域差异量化、代表样本跟踪、判别器日志和影响力分析，可以将迁移过程由不可见的“黑箱”转化为可视化的“迁移路径”，从而验证迁移是否真实依赖于物理机理，定位迁移失败的原因，从而指导数据采集与模型改进。

（3） 再者，通过时域、频域、时频域、手工特征等多维度解释，提供图表化的证据，将数值和图表转换为简短语句，展示迁移前后特征空间位置的变化，验证知识迁移的路径，进行事后可解释性问题的开展，将解释结果与这些物理机理对齐，使得专家能够快速验证并信任模型的预测。

6.2 问题四的求解

目前大多数迁移模型的训练与推理过程存在严重的“黑箱缺陷”，模型能够给出预测结果，但是其内部决策过程不透明，我们无法直观理解其预测过程和依据。在高速列车轴承故障智能诊断中，黑箱缺陷的表现为模型能区分故障类型，但无法说明依据；模型依赖数据集中的噪声或工况相关特征而忽略了真正的频率特征。在迁移学习中，由于黑箱缺陷，会导致迁移过程无法解释，模型依赖噪声特征造成虚假对齐。据此，任务四的核心目标就是通过分析事前可解释性、迁移过程可解释性、事后可解释性建立模型决策与物理机理之间的联系，来缓解黑箱

缺陷，建立。其中，迁移过程的可解释性是缓解黑箱缺陷的核心，下面我们着重分析迁移过程的可解释性。

6.2.1 迁移过程的可解释性

所谓迁移过程的可解释性，简而言之就是将不可见的迁移过程通过对源域和目标域的特征分布轨迹、两者分布之间的距离、 迁移路径的可视化、可量化、可追踪，从而实现迁移过程的透明化，验证迁移过程是否符合物理机理，进而实现模型改进。

接下来我们通过两个模型来实现迁移过程的可解释性。

在任务三中，已经简单介绍过 DANN 模型和 TCA—XGBoost 模型，在这里不做过多的赘述。其中，DANN 的核心思想是通过一个共享特征提取器，同时将分类器和域分类器相连，并利用梯度反转层（GRL）进行对抗训练，使得特征既能区分故障类型，又能在域上使得源域和目标域分布一致。因此，选择 DANN 模型来做迁移路径监控是非常合适的。首先，每隔 N 个步骤从特征提取器中抽取源域与目标域的嵌入向量；按类别计算每类的均值、协方差、方差；记录特征分布轨迹。其次，在每次迭代中计算域距指标：MMD、CORAL、Wasserstein。在特征空间中计算全局 MMD，并记录曲线。当 MMD 值或 CORAL 值下降时说明特征分布正在靠近；但如果目标准确度不变或下降时，说明可能存在标签偏移或类别不平衡等问题。然后，在每次迭代中需要记录域判别损失、任务分类器损失、总损失，监测域判别器的准确率，包括整体准确率和按类别划分的准确率。此外，还需计算影响函数来识别高影响源样本。遍历上述流程，DANN 模型通过特征提取器、GRL 和域判别器的结合，强制目标域与源域的特征分布接近。基于对抗学习机制，GRL 让特征提取器“欺骗”域判别器，达到域不变表示。再通过每次遍历记录的 MMD、CORAL、t-SNE、域判别器准确率等实现路径实时可视化，代表样本跟踪和判别器日志直观展示迁移动态，完美回答迁移过程是否在发生的疑问。

下面着重解释用 TCA—XGBoost 模型来实现迁移过程的可解释性。

首先，准备数据，输入特征集，对原始特征做 z-score 标准化，为了便于查看原始特征权重，直接使用线性 TCA。其次，迁移之前，在源域数据中，用原始特征训练一个 XGBoost 分类器，提取全局特征重要性和基于 SHAP 的局部重要性，明确在台架实验中，哪些特征对分类判别最有用。然后，进行 TCA 投影，把源域矩阵$X_s \in \mathbb{R}^{n_s \times d}$和目标域矩阵$X_t \in \mathbb{R}^{n_t \times d}$合并为$X = [X_s; X_t]$，运行线性 TCA，得到投影矩阵$P \in \mathbb{R}^{d \times k}$。投影后的特征$Z = XP$，源域的投影记为$Z_s$，目标域的投影记为$Z_t$。在投影后的源域投影特征$Z_s$上训练 XGBoost 分类器，获取每个 TCA 组件的重要性分数$importance_TCA[j]$，识别 TCA 分量在新空间中的贡献度，承担大部分判别力。投影矩阵$P = [p_1 p_2 \dots p_k]$，其中$p_j \in \mathbb{R}^d$是第 j 个 TCA

分量在原始特征坐标下的权重向量。其中，第 j 个 TCA 分量对第 i 个原始特征的贡献度可以用$|p_{i,j}|$来衡量。然后把把 TCA 分量的重要性$importance_TCA[j]$与权重相乘，$C_i = \sum_{j=1}^{k} importance_{TCA}[j] \cdot |p_{i,j}|$，即得到原始特征 i 的综合贡献度。最后把$C_i$归一化，得到原始特征的相对贡献百分比。经过上述流程，TCA 将源域与目标域中同轴承内圈故障、外圈故障机理直接相关的频带能量与时域冲击特征融合为在共享子空间中判别力最强的分量，即解释了迁移是如何发生的。以本次竞赛任务为例，在源域数据集上提取 12 个特征值作为原始特征集，包括均值、均方根、峭度、峰值因子、脉冲因子、包络在 BPFO 频段能量、包络在 BPFI 频段能量、包络在 BSF 频段能量、频谱质心、均方频率、小波包能量熵、Hilbert 包络在 BPFO 处的幅值。在源域数据集中训练 XGBoost，得到原始特征重要性排序，可以得出 BPFI、BPFO、峭度、希尔伯特包络谱特征属于重要机理特征。对源域数据和目标域数据做 TCA 投影，得到新的 4 个 TCA 分量，在投影后的源域上再次训练 XGBoost，得到新的四个 TCA 分量的重要性。分析线性 TCA 的投影矩阵 ，得到 TCA 分量的绝对权重。

表 6- 1 不同特征 TCA 权重表

特征	TCA1	TCA2	TCA3	TCA4
均值	0.05	0.03	0.02	0.01
均方根	0.10	0.08	0.05	0.04
峭度	0.40	0.15	0.10	0.05
峰值因子	0.08	0.06	0.04	0.03
脉冲因子	0.20	0.12	0.08	0.05
BPFO	0.05	0.04	0.20	0.10
BPFI	0.50	0.30	0.15	0.08
BSF	0.06	0.25	0.18	0.07
频谱质心	0.03	0.04	0.02	0.01
均方频率	0.02	0.03	0.02	0.01
小波包能量熵	0.10	0.18	0.12	0.06
Hilbert 包络在 BPFO 处的幅值	0.45	0.22	0.12	0.05

根据公式$C_i = \sum_{j=1}^{k} importance_{TCA}[j] \cdot |p_{i,j}|$计算，再归一化成百分比，得到原始特征的综合贡献度。

表 6- 2 不同特征值综合贡献度表

原始特征	综合贡献（%）
BPFI	20.3
Hilbert 包络在 BPFO 处的幅值	17.2
峭度	14.5
BPFO	10.6

脉冲因子	8.4
BSF	8.0
小波包能量熵	7.3
均方根	4.7
峰值因子	3.7
均值	2.1
频谱质心	1.7
均方频率	1.3

将结果可视化即可得出下图

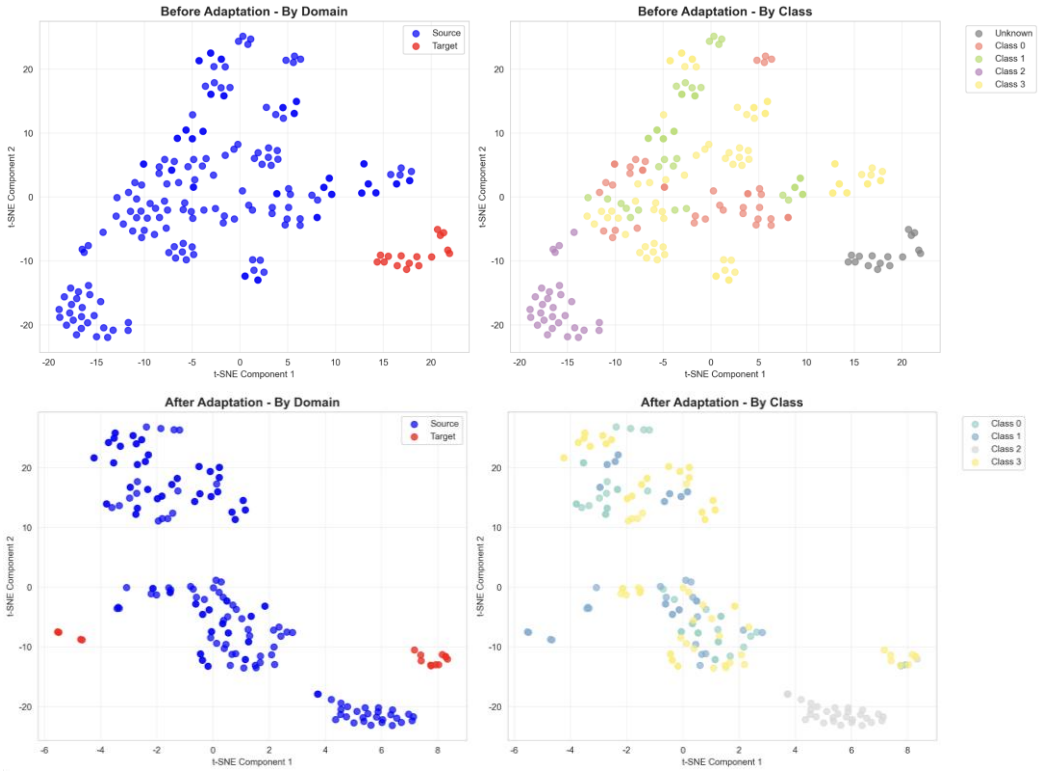


图 6- 1TCA 迁移前后迁移组件的重要性排序

这两张图横坐标表示 TCA 所创建的新特征空间维度，纵坐标表示每个迁移成分对于域分类任务的重要性。两张图分别代表了 TCA 变换前后，迁移组件的重要性排序。TCA 的目标是最小化迁移组件成分的域间差异，通过对比两张图，可以看出这是一个良好的迁移学习模型，既符合物理机理，又有可解释性。

在两种模型训练过程中，我们全方位监控了五类关键指标，包括特征分布轨迹、域差异量化、代表样本跟踪、对抗判别器行为日志、影响力分析。在技术层面上，作为核化的分布差度量，MMD 被当作全局与类条件对齐的基本依据；CORAL 对协方差对齐提供衡量标准；对抗判别器的按类准确率可以帮助确认哪些类别被对齐；代表样本跟踪提供可视化证据；影响力分析则是将迁移的抽象过程映射到具体源样本，从而为源数据清洗和采样策略提供直接反馈。通过以上量化以及可视化的数据，迁移学习的黑箱缺陷被缓解，逐步明确“是否迁移”、“怎

么迁移”、“如何迁移”等问题，从而为高速列车轴承智能诊断的可信度与实践落地提供了实际操作的技术路径与证据链。

7 未来展望

本研究在源域故障诊断建模与跨域迁移方法设计方面取得了一定成果，但仍存在一定的局限性，未来我们在研究中可在以下几个方向进一步深化和完善：

（1）机械设备的运行状态受多种复杂因素共同作用，故障的发生往往具有非线性、变化性和不确定性等特征。本研究主要基于稳态工况下得到的数据进行特征提取与模型构建，虽然在典型故障识别中表现出较高准确性，但对变转速、变负载等动态工况下的适应能力仍有待验证。此外，当前模型尚未充分融合温度、电流、声发射等多源传感信息，而这些物理量在实际工业场景中可能提供关键的互补判据。未来研究应构建多模态数据融合框架，探索异构传感器协同感知机制，提升模型对复杂运行环境的感知能力与鲁棒性。

（2）本研究所采用的迁移学习方法虽能有效缓解域偏移问题，但在面对极端工况差异或全新故障模式时仍可能出现性能下降。例如，当目标设备结构发生显著变化或出现未曾在源域中出现的新型故障时，现有模型难以实现“零样本”或“少样本”泛化。因此，未来应探索更先进的迁移范式，如深度域泛化（Domain Generalization）、元学习（Meta-Learning）和自监督对比学习（Self-supervised Contrastive Learning），以增强模型对未知域的适应能力和对新类别的发现能力，推动故障诊断系统向“自主进化”方向发展。

（3）随着工业智能化进程加速，传统基于离线训练的诊断模型已难以满足实时监测与在线更新的需求。本研究中的模型训练与迁移过程均在静态数据集上完成，缺乏对时间序列演化特性的动态建模能力。未来研究应引入在线学习（Online Learning）和持续学习（Continual Learning）机制，使诊断系统能够在不遗忘已有知识的前提下，持续吸收新数据并动态优化模型参数，从而实现从“一次性建模”向“终身学习”的跨越。同时，应结合边缘计算与轻量化网络设计，推动高精度诊断模型在嵌入式设备上的部署应用，真正实现“端-边-云”协同的智能运维体系。

（4）当前研究更多关注模型的预测性能，而在可解释性与决策可信度方面仍有不足。工程现场技术人员往往不仅关心“是否故障”，更关注“为何判断为该故障”。因此，未来工作应加强模型的可解释性研究，引入 SHAP 值、LIME、注意力机制等工具，可视化关键特征贡献路径，构建“透明化”诊断流程。这不仅能提升用户对智能系统的信任度，也为故障机理分析与维修策略制定提供科学依据。

综上所述，未来的智能故障诊断研究应朝着多源融合、强泛化、自适应、可解释的方向持续演进，在理论创新与工程落地之间建立更加紧密的桥梁，最终实现从“被动响应”到“主动预测”、从“局部最优”到“全局智能”的转型升级。

参考文献

[1] 庄福振,罗平,何清,等.迁移学习研究进展[J].软件学报,2015,26(01):26-39.DOI:10.13328/j.cnki.jos.004631.

[2] 谭海旺,杨启亮,邢建春,等.基于 XGBoost-LSTM 组合模型的光伏发电功率预测[J].太阳能学报,2022,43(08):75-81.DOI:10.19912/j.0254-0096.tynxb.2021-0005.

[3] 郑志伟,易彩,廖小康.线路条件对高速列车轴箱轴承载荷特性影响研究[J].机械工程学报,2024,60(20):251-260.

[4] 赵健鹏.超高速列车轴承性能试验台研究[D].大连交通大学,2019.DOI:10.26990/d.cnki.gsltc.2019.000276.

[5] 曹国晋.高铁轴承疲劳特性分析及可靠性寿命预测[D].上海应用技术大学,2018.

[6] 李振乾.基于机理与数据混合驱动的高速列车轴箱轴承载荷反演研究[D].西南交通大学,2023.DOI:10.27414/d.cnki.gxnju.2023.002666.

[7] 季磊.基于信息熵的滚动轴承故障特征提取及模式识别方法研究[D].合肥工业大学,2023.DOI:10.27101/d.cnki.ghfgu.2023.001976.

[8] 李杨.基于振动环境下的高速列车轴箱轴承试验台设计与研究[D].西南交通大学,2015.

[9] 邓飞跃,董少飞,顾晓辉.基于 ITR-Net 多源域迁移学习的高铁轴箱轴承故障诊断研究[J/OL].工程科学与技术,1-11[2025-09-25].https://link.cnki.net/urlid/51.1773.TB.20240605.1417.013.

[10] 丁世飞,齐丙娟,谭红艳.支持向量机理论与算法研究综述[J].电子科技大学学报,2011,40(01):2-10.

[11] 孔英会,景美丽.基于混淆矩阵和集成学习的分类方法研究[J].计算机工程与科学,2012,34(06):111-117.

附录

附录 1

特征提取代码 Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import signal
from scipy.fft import fft, fftfreq
from scipy.signal import hilbert, welch
from sklearn.preprocessing import StandardScaler
import pywt
import os
import warnings

warnings.filterwarnings('ignore')

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False


class BearingFeatureExtractor:
    def __init__(self, fs=25600):
        self.fs = fs  # 采样频率
        # 轴承故障特征频率参数（需要根据实际轴承参数调整）
        self.BPFO = 32000  # 外圈故障频率
        self.BPFI = 32000  # 内圈故障频率
        self.BSF = 32000  # 滚动体故障频率
        self.FTF = 10  # 保持架故障频率

    def extract_features(self, data):
        """提取所有特征"""
        features = []

        # 1. 时域特征
        features.extend(self.time_domain_features(data))

        # 2. 频域特征
        features.extend(self.frequency_domain_features(data))

        # 3. 包络谱特征
        features.extend(self.envelope_spectrum_features(data))

        # 4. 小波包能量熵
        features.extend(self.wavelet_packet_energy_entropy(data))

        return np.array(features)

    def time_domain_features(self, data):
        """时域特征"""
        mean_val = np.mean(data)
        rms_val = np.sqrt(np.mean(data ** 2))
        kurtosis_val = np.mean((data - mean_val) ** 4) / (np.std(data) ** 4)

        # 峰值因子
        peak_factor = np.max(np.abs(data)) / rms_val

        # 脉冲因子
        impulse_factor = np.max(np.abs(data)) / np.mean(np.abs(data))

        return [mean_val, rms_val, kurtosis_val, peak_factor, impulse_factor]

    def frequency_domain_features(self, data):
        """频域特征"""
        n = len(data)
        fft_vals = fft(data)
        freqs = fftfreq(n, 1 / self.fs)
```

```
        # 取正频率部分
        positive_freq_idx = freqs > 0
        freqs = freqs[positive_freq_idx]
        fft_vals = np.abs(fft_vals[positive_freq_idx])

        # 频谱质心
        spectral_centroid = np.sum(freqs * fft_vals) / np.sum(fft_vals)

        # 均方频率
        mean_square_frequency = np.sum((freqs ** 2) * fft_vals) / np.sum(fft_vals)

        return [spectral_centroid, mean_square_frequency]

    def envelope_spectrum_features(self, data):
        """包络谱特征"""
        # Hilbert 包络分析
        analytic_signal = hilbert(data)
        envelope = np.abs(analytic_signal)

        # 包络谱
        n = len(envelope)
        envelope_fft = fft(envelope - np.mean(envelope))
        freqs = fftfreq(n, 1 / self.fs)

        positive_freq_idx = (freqs > 0) & (freqs < self.fs / 2)
        freqs = freqs[positive_freq_idx]
        envelope_fft = np.abs(envelope_fft[positive_freq_idx])

        # 在故障频率附近的能量
        def get_band_energy(center_freq, bandwidth=10):
            idx = (freqs >= center_freq - bandwidth / 2) & (freqs <= center_freq + bandwidth / 2)
            return np.sum(envelope_fft[idx] ** 2) if np.any(idx) else 0

        bpo_energy = get_band_energy(self.BPFO)
        bpi_energy = get_band_energy(self.BPFI)
        bsf_energy = get_band_energy(self.BSF)

        # Hilbert 包络在 BPFO 处的幅值
        bpo_idx = np.argmin(np.abs(freqs - self.BPFO))
        bpo_amplitude = envelope_fft[bpo_idx] if bpo_idx < len(envelope_fft) else 0

        return [bpo_energy, bpi_energy, bsf_energy, bpo_amplitude]

    def wavelet_packet_energy_entropy(self, data, wavelet='db4', level=3):
        """小波包能量熵"""
        # 小波包分解
        wp = pywt.WaveletPacket(data=data, wavelet=wavelet, mode='symmetric', maxlevel=level)

        # 获取所有节点能量
        energy_nodes = []
        for node in wp.get_level(level, order='freq'):
            coeff = node.data
            energy = np.sum(coeff ** 2)
            energy_nodes.append(energy)

        energy_nodes = np.array(energy_nodes)
        total_energy = np.sum(energy_nodes)

        # 计算能量熵
        if total_energy > 0:
            energy_ratio = energy_nodes / total_energy
            energy_ratio = energy_ratio[energy_ratio > 0]  # 避免 log(0)
            energy_entropy = -np.sum(energy_ratio * np.log(energy_ratio))
        else:
            energy_entropy = 0

        return [energy_entropy]

    def plot_time_domain(self, data, title, ax):
        """绘制时域波形"""
        t = np.arange(len(data)) / self.fs
        ax.plot(t, data)
```

```
        ax.set_xlabel('时间 (s)')
        ax.set_ylabel('幅值')
        ax.set_title(f'{title} - 时域波形')
        ax.grid(True)

    def plot_frequency_domain(self, data, title, ax):
        """绘制频谱"""
        n = len(data)
        fft_vals = fft(data)
        freqs = fftfreq(n, 1 / self.fs)

        positive_freq_idx = freqs > 0
        freqs = freqs[positive_freq_idx]
        fft_vals = np.abs(fft_vals[positive_freq_idx])

        ax.semilogy(freqs, fft_vals)
        ax.set_xlabel('频率 (Hz)')
        ax.set_ylabel('幅值')
        ax.set_title(f'{title} - 频谱')
        ax.grid(True)

        # 标记故障频率
        ax.axvline(self.BPFO, color='r', linestyle='--', alpha=0.7, label=f'BPFO ({self.BPFO}Hz)')
        ax.axvline(self.BPFI, color='g', linestyle='--', alpha=0.7, label=f'BPFI ({self.BPFI}Hz)')
        ax.axvline(self.BSF, color='b', linestyle='--', alpha=0.7, label=f'BSF ({self.BSF}Hz)')
        ax.legend()

    def plot_wavelet_energy(self, data, title, ax):
        """绘制小波能量分布"""
        wavelet = 'db4'
        level = 4

        # 小波包分解
        wp = pywt.WaveletPacket(data=data, wavelet=wavelet, mode='symmetric', maxlevel=level)

        # 获取最后一层节点能量
        energy_nodes = []
        node_labels = []
        for node in wp.get_level(level, order='freq'):
            coeff = node.data
            energy = np.sum(coeff ** 2)
            energy_nodes.append(energy)
            node_labels.append(node.path)

        # 绘制能量分布
        ax.bar(range(len(energy_nodes)), energy_nodes)
        ax.set_xlabel('小波包节点')
        ax.set_ylabel('能量')
        ax.set_title(f'{title} - 小波包能量分布')
        ax.set_xticks(range(len(energy_nodes)))
        ax.set_xticklabels(node_labels, rotation=45)
        ax.grid(True, alpha=0.3)


def process_dataset(metadata_file, feature_extractor, scaler=None, save_path=None):
    """处理数据集并提取特征"""
    # 读取 metadata
    metadata = pd.read_csv(metadata_file)

    all_features = []
    file_info = []

    print(f"处理 {metadata_file}...")

    for idx, row in metadata.iterrows():
        file_path = row['file_path']  # 假设 metadata 中有 file_path 列
        fault_type = row.get('fault_type', 'unknown')  # 假设有 fault_type 列

        try:
            # 读取数据文件（假设是 CSV 格式，单列数据）
            data = pd.read_csv(file_path).iloc[:, 0].values
```

```
                # 提取特征
                features = feature_extractor.extract_features(data)
                all_features.append(features)
                file_info.append({'file_path': file_path, 'fault_type': fault_type})

                # 每处理 10 个文件打印进度
                if (idx + 1) % 10 == 0:
                    print(f"已处理 {idx + 1}/{len(metadata)} 个文件")

            except Exception as e:
                print(f"处理文件 {file_path} 时出错: {e}")
                continue

    # 转换为 DataFrame
    feature_names = [
        'mean', 'rms', 'kurtosis', 'peak_factor', 'impulse_factor',
        'spectral_centroid', 'mean_square_frequency',
        'bpo_energy', 'bpi_energy', 'bsf_energy', 'bpo_amplitude',
        'wavelet_energy_entropy'
    ]

    features_df = pd.DataFrame(all_features, columns=feature_names)
    info_df = pd.DataFrame(file_info)

    # 标准化
    if scaler is None:
        scaler = StandardScaler()
        features_scaled = scaler.fit_transform(features_df)
    else:
        features_scaled = scaler.transform(features_df)

    # 合并信息和特征
    result_df = pd.concat([info_df, pd.DataFrame(features_scaled, columns=feature_names)], axis=1)

    # 保存结果
    if save_path:
        result_df.to_csv(save_path, index=False)
        print(f"特征已保存到: {save_path}")

    return result_df, scaler


def visualize_features(train_features, test_features, target_features, output_dir):
    """特征可视化"""

    # 1. 特征分布对比
    feature_cols = [col for col in train_features.columns if col not in ['file_path', 'fault_type']]

    fig, axes = plt.subplots(3, 4, figsize=(20, 15))
    axes = axes.ravel()

    for i, feature in enumerate(feature_cols[:12]):  # 显示前 12 个特征
        # 训练集特征分布
        for fault_type in train_features['fault_type'].unique():
            data = train_features[train_features['fault_type'] == fault_type][feature]
            axes[i].hist(data, alpha=0.6, label=f'{fault_type}', bins=20)

        axes[i].set_title(f'{feature} 分布')
        axes[i].set_xlabel(feature)
        axes[i].set_ylabel('频数')
        if i == 0:
            axes[i].legend()

    plt.tight_layout()
    plt.savefig(f'{output_dir}/feature_distribution.png', dpi=300, bbox_inches='tight')
    plt.show()

    # 2. 相关性热力图
    plt.figure(figsize=(12, 10))
    correlation_matrix = train_features[feature_cols].corr()
    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
    plt.title('特征相关性热力图')
    plt.tight_layout()
```

```
    plt.savefig(f'{output_dir}/feature_correlation.png', dpi=300, bbox_inches='tight')
    plt.show()

    # 3. 故障类型对比
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # 选择几个关键特征进行对比
    key_features = ['rms', 'kurtosis', 'bpo_energy', 'wavelet_energy_entropy']

    for i, feature in enumerate(key_features):
        ax = axes[i // 2, i % 2]
        fault_data = []
        fault_labels = []

        for fault_type in train_features['fault_type'].unique():
            data = train_features[train_features['fault_type'] == fault_type][feature]
            fault_data.append(data)
            fault_labels.append(fault_type)

        ax.boxplot(fault_data, labels=fault_labels)
        ax.set_title(f'{feature} - 不同故障类型对比')
        ax.set_ylabel(feature)
        ax.tick_params(axis='x', rotation=45)

    plt.tight_layout()
    plt.savefig(f'{output_dir}/fault_type_comparison.png', dpi=300, bbox_inches='tight')
    plt.show()


def plot_signal_analysis(sample_data, feature_extractor, fault_type, output_dir):
    """绘制信号分析图"""
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    # 时域波形
    feature_extractor.plot_time_domain(sample_data, fault_type, axes[0, 0])

    # 频谱
    feature_extractor.plot_frequency_domain(sample_data, fault_type, axes[0, 1])

    # 小波能量分布
    feature_extractor.plot_wavelet_energy(sample_data, fault_type, axes[1, 0])

    # 包络谱
    analytic_signal = hilbert(sample_data)
    envelope = np.abs(analytic_signal)
    n = len(envelope)
    envelope_fft = fft(envelope - np.mean(envelope))
    freqs = fftfreq(n, 1 / feature_extractor.fs)
    positive_freq_idx = (freqs > 0) & (freqs < feature_extractor.fs / 2)

    axes[1, 1].plot(freqs[positive_freq_idx], np.abs(envelope_fft[positive_freq_idx]))
    axes[1, 1].set_xlabel('频率 (Hz)')
    axes[1, 1].set_ylabel('幅值')
    axes[1, 1].set_title(f'{fault_type} - 包络谱')
    axes[1, 1].grid(True)

    # 标记故障频率
    axes[1, 1].axvline(feature_extractor.BPFO, color='r', linestyle='--', alpha=0.7, label=f'BPFO')
    axes[1, 1].axvline(feature_extractor.BPFI, color='g', linestyle='--', alpha=0.7, label=f'BPFI')
    axes[1, 1].axvline(feature_extractor.BSF, color='b', linestyle='--', alpha=0.7, label=f'BSF')
    axes[1, 1].legend()

    plt.tight_layout()
    plt.savefig(f'{output_dir}/signal_analysis_{fault_type}.png', dpi=300, bbox_inches='tight')
    plt.show()


def main():
    # 设置路径
    base_dir = '/Users/slh/Downloads/planB'
    train_metadata = f'{base_dir}/train_metadata.csv'
    test_metadata = f'{base_dir}/test_metadata.csv'
    target_metadata = f'{base_dir}/target_metadata.csv'
```

```
    # 创建特征提取器
    feature_extractor = BearingFeatureExtractor(fs=25600)

    # 处理训练集
    print("=== 处理训练集 ===")
    train_features, scaler = process_dataset(
        train_metadata,
        feature_extractor,
        scaler=None,
        save_path=f'{base_dir}/train_features.csv'
    )

    # 保存标准化参数
    scaler_params = {
        'mean': scaler.mean_,
        'scale': scaler.scale_
    }
    np.savez(f'{base_dir}/scaler_params.npz', **scaler_params)
    print(f"标准化参数已保存到: {base_dir}/scaler_params.npz")

    # 处理测试集
    print("\n=== 处理测试集 ===")
    test_features, _ = process_dataset(
        test_metadata,
        feature_extractor,
        scaler=scaler,
        save_path=f'{base_dir}/test_features.csv'
    )

    # 处理目标域数据
    print("\n=== 处理目标域数据 ===")
    target_features, _ = process_dataset(
        target_metadata,
        feature_extractor,
        scaler=scaler,
        save_path=f'{base_dir}/target_features.csv'
    )

    # 特征可视化
    print("\n=== 进行特征可视化 ===")
    visualize_features(train_features, test_features, target_features, base_dir)

    # 样本信号分析（选取每个故障类型的一个样本进行详细分析）
    print("\n=== 进行样本信号分析 ===")
    metadata = pd.read_csv(train_metadata)
    for fault_type in metadata['fault_type'].unique():
        sample_file = metadata[metadata['fault_type'] == fault_type].iloc[0]['file_path']
        try:
            sample_data = pd.read_csv(sample_file).iloc[:, 0].values
            plot_signal_analysis(sample_data[:min(8192, len(sample_data))],
                                 feature_extractor, fault_type, base_dir)
        except Exception as e:
            print(f"分析样本 {sample_file} 时出错: {e}")

    print("\n=== 处理完成 ===")
    print(f"训练集特征形状: {train_features.shape}")
    print(f"测试集特征形状: {test_features.shape}")
    print(f"目标域特征形状: {target_features.shape}")

    # 显示特征统计信息
    feature_cols = [col for col in train_features.columns if col not in ['file_path', 'fault_type']]
    print(f"\n 特征列表: {feature_cols}")
    print(f"\n 训练集特征统计:")
    print(train_features[feature_cols].describe())


if __name__ == "__main__":
    main()
```

附录 2

元信息提取代码 Python

```
import os
import pandas as pd
import re


def extract_metadata_from_filename(filename, file_path, domain):
    """从文件名中提取元信息"""
    metadata = {
        'domain': domain,
        'filename': filename,
        'file_path': file_path,
        'sensor_channel': 'Unknown',
        'sampling_rate': 'Unknown',
        'fault_class': 'Unknown',
        'load': 'Unknown',
        'fault_size': 'Unknown'
    }

    # 处理目标域数据（简单的.mat 文件）
    if domain == 'target':
        if filename.endswith('.mat'):
            # 目标域数据只有基本文件名信息
            metadata['filename'] = filename
            # 其他字段保持为'Unknown'
        return metadata

    # 处理源域数据
    try:
        # 从文件路径中提取传感器通道和采样率
        path_parts = file_path.split('/')

        # 查找包含采样率和传感器通道信息的目录名
        for part in path_parts:
            if 'kHz' in part and ('DE' in part or 'FE' in part or 'BA' in part):
                # 提取采样率（前 5 个字符，如"12kHz"）
                if part.startswith('12kHz'):
                    metadata['sampling_rate'] = '12kHz'
                elif part.startswith('48kHz'):
                    metadata['sampling_rate'] = '48kHz'

                # 提取传感器通道
                if '_DE_' in part:
                    metadata['sensor_channel'] = 'DE'
                elif '_FE_' in part:
                    metadata['sensor_channel'] = 'FE'
                elif '_BA_' in part:
                    metadata['sensor_channel'] = 'BA'
                break

        # 从文件名中提取故障类别、载荷和故障尺寸
        if filename.endswith('.mat'):
            # 提取故障类别（文件名开头的字母组合）
            fault_match = re.match(r'^([A-Z]+)', filename)
            if fault_match:
                metadata['fault_class'] = fault_match.group(1)

            # 提取故障尺寸（文件名中的数字，通常是 3 位）
            size_match = re.search(r'(\d{3,4})', filename)
            if size_match:
                size_str = size_match.group(1)
                # 取后 3 位作为故障尺寸
                if len(size_str) >= 3:
                    metadata['fault_size'] = size_str[-3:]

            # 提取载荷（文件名中的单个数字）
            load_match = re.search(r'_(\d)[_.]', filename)
            if load_match:
                metadata['load'] = load_match.group(1)
            else:
                # 尝试其他模式，如 IR028_2_(1750rpm).mat
                load_match2 = re.search(r'_(\d)_\(', filename)
                if load_match2:
                    metadata['load'] = load_match2.group(1)
```

```
            # 从目录路径中补充故障尺寸信息
            for part in path_parts:
                if part.isdigit() and len(part) == 4:  # 0007 这样的目录
                    metadata['fault_size'] = part[-3:]  # 取后三位
                    break

        except Exception as e:
            print(f"处理文件 {filename} 时出错: {e}")

        return metadata


def collect_metadata(source_dir, target_dir):
    """收集源域和目标域的元数据"""
    all_metadata = []

    # 处理源域数据
    print("正在处理源域数据...")
    if os.path.exists(source_dir):
        for root, dirs, files in os.walk(source_dir):
            for file in files:
                if file.endswith('.mat'):
                    file_path = os.path.join(root, file)
                    metadata = extract_metadata_from_filename(file, file_path, 'source')
                    all_metadata.append(metadata)
                    print(f"处理源域文件: {file}")
    else:
        print(f"警告: 源域目录不存在: {source_dir}")

    # 处理目标域数据
    print("正在处理目标域数据...")
    if os.path.exists(target_dir):
        for root, dirs, files in os.walk(target_dir):
            for file in files:
                if file.endswith('.mat'):
                    file_path = os.path.join(root, file)
                    metadata = extract_metadata_from_filename(file, file_path, 'target')
                    all_metadata.append(metadata)
                    print(f"处理目标域文件: {file}")
    else:
        print(f"警告: 目标域目录不存在: {target_dir}")

    return all_metadata


def main():
    # 设置路径
    source_dir = "/Users/slh/Downloads/planB/data/source_data"
    target_dir = "/Users/slh/Downloads/planB/data/target_data"
    output_file = "/Users/slh/Downloads/planB/metadata.csv"

    # 确保输出目录存在
    os.makedirs(os.path.dirname(output_file), exist_ok=True)

    # 收集元数据
    print("开始收集元数据...")
    metadata_list = collect_metadata(source_dir, target_dir)

    if metadata_list:
        # 创建 DataFrame
        df = pd.DataFrame(metadata_list)

        # 保存为 CSV 文件（使用 UTF-8 编码避免中文乱码）
        df.to_csv(output_file, index=False, encoding='utf-8')
        print(f"元数据已保存到: {output_file}")
        print(f"总共处理了 {len(metadata_list)} 个文件")

        # 显示数据预览
        print("\n 数据预览:")
        print(df.head())
        print(f"\n 数据形状: {df.shape}")
```

```
    else:
        print("未找到任何.mat 文件")


if __name__ == "__main__":
    main()
```

附录 3

DANN 代码 Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.manifold import TSNE
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import KFold
from sklearn.preprocessing import LabelEncoder, StandardScaler
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import os
import warnings

warnings.filterwarnings('ignore')

# 创建保存图片的目录
save_dir = '/Users/slh/Downloads/planB/第三问可视化'
os.makedirs(save_dir, exist_ok=True)

# 设置中文字体和样式
plt.rcParams['font.sans-serif'] = ['Arial']
plt.rcParams['axes.unicode_minus'] = False
sns.set_style("whitegrid")


def load_and_prepare_data():
    """加载和准备源域和目标域数据"""

    # 加载源域数据（训练集和测试集）
    train_src = pd.read_csv('/Users/slh/Downloads/planB/train_features.csv')
    test_src = pd.read_csv('/Users/slh/Downloads/planB/test_features.csv')

    # 合并源域数据
    X_src = pd.concat([train_src, test_src], ignore_index=True)

    # 移除不需要的列，保留特征
    src_features_to_remove = ['file_path']
    X_src_features = X_src.drop(columns=src_features_to_remove, errors='ignore')

    # 分离特征和标签
    y_src = X_src_features['fault_class']
    X_src_final = X_src_features.drop('fault_class', axis=1)

    # 加载目标域数据
    target_data = pd.read_csv('/Users/slh/Downloads/planB/target_features.csv')

    # 移除目标域不需要的列
    tar_features_to_remove = ['file_path', 'fault_name', 'fault_class']
    X_tar = target_data.drop(columns=tar_features_to_remove, errors='ignore')

    # 获取目标域文件名（A-P）
    target_filenames = target_data['fault_name'].tolist()

    # 数据标准化
    scaler = StandardScaler()
    X_src_scaled = scaler.fit_transform(X_src_final)
    X_tar_scaled = scaler.transform(X_tar)

    print(f"Source domain data shape: {X_src_scaled.shape}")
```

```
    print(f"Source domain labels shape: {y_src.shape}")
    print(f"Target domain data shape: {X_tar_scaled.shape}")
    print(f"Number of target domain files: {len(target_filenames)}")
    print(f"Target domain filenames: {target_filenames}")

    return X_src_scaled, y_src, X_tar_scaled, target_filenames, scaler


def encode_labels(y_src):
    """对标签进行编码"""
    le = LabelEncoder()
    y_encoded = le.fit_transform(y_src)
    class_names = le.classes_
    return y_encoded, le, class_names


def calculate_mmd(X_src, X_tar):
    """计算最大均值差异（MMD）"""
    X_src_tensor = torch.FloatTensor(X_src)
    X_tar_tensor = torch.FloatTensor(X_tar)

    # 使用高斯核计算 MMD
    def gaussian_kernel(x, y, sigma=1.0):
        x_size = x.size(0)
        y_size = y.size(0)
        dim = x.size(1)

        x = x.unsqueeze(1)  # x_size x 1 x dim
        y = y.unsqueeze(0)  # 1 x y_size x dim

        tiled_x = x.expand(x_size, y_size, dim)
        tiled_y = y.expand(x_size, y_size, dim)

        squared_diff = (tiled_x - tiled_y).pow(2).sum(2)
        return torch.exp(-squared_diff / (2 * sigma ** 2))

    K_XX = gaussian_kernel(X_src_tensor, X_src_tensor)
    K_YY = gaussian_kernel(X_tar_tensor, X_tar_tensor)
    K_XY = gaussian_kernel(X_src_tensor, X_tar_tensor)

    mmd = K_XX.mean() + K_YY.mean() - 2 * K_XY.mean()
    return mmd.item()


def calculate_wasserstein(X_src, X_tar):
    """计算 Wasserstein 距离的近似值"""
    from scipy.stats import wasserstein_distance

    # 由于 Wasserstein 距离计算高维数据较慢，我们使用主要成分
    from sklearn.decomposition import PCA
    pca = PCA(n_components=min(5, X_src.shape[1], X_tar.shape[1]))
    X_combined = np.vstack([X_src, X_tar])
    X_combined_pca = pca.fit_transform(X_combined)

    X_src_pca = X_combined_pca[:len(X_src)]
    X_tar_pca = X_combined_pca[len(X_src):]

    # 计算每个维度的 Wasserstein 距离并取平均
    wasserstein_dists = []
    for i in range(X_src_pca.shape[1]):
        dist = wasserstein_distance(X_src_pca[:, i], X_tar_pca[:, i])
        wasserstein_dists.append(dist)

    return np.mean(wasserstein_dists)


class FeatureExtractor(nn.Module):
    """特征提取器"""

    def __init__(self, input_dim, hidden_dims=[128, 64]):
        super(FeatureExtractor, self).__init__()
        layers = []
        prev_dim = input_dim
```

```
            for hidden_dim in hidden_dims:
                layers.append(nn.Linear(prev_dim, hidden_dim))
                layers.append(nn.BatchNorm1d(hidden_dim))
                layers.append(nn.ReLU())
                layers.append(nn.Dropout(0.3))
                prev_dim = hidden_dim

            self.network = nn.Sequential(*layers)

        def forward(self, x):
            return self.network(x)


class Classifier(nn.Module):
    """分类器"""

    def __init__(self, input_dim, num_classes):
        super(Classifier, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, num_classes)
        )

    def forward(self, x):
        return self.network(x)


class DomainDiscriminator(nn.Module):
    """领域判别器"""

    def __init__(self, input_dim):
        super(DomainDiscriminator, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.network(x)


class DANNModel:
    """领域对抗神经网络模型"""

    def __init__(self, input_dim, num_classes, hidden_dims=[128, 64]):
        self.feature_extractor = FeatureExtractor(input_dim, hidden_dims)
        self.classifier = Classifier(hidden_dims[-1], num_classes)
        self.domain_discriminator = DomainDiscriminator(hidden_dims[-1])

        self.optimizer = optim.Adam(
            list(self.feature_extractor.parameters()) +
            list(self.classifier.parameters()) +
            list(self.domain_discriminator.parameters()),
            lr=0.001
        )

        self.criterion_class = nn.CrossEntropyLoss()
        self.criterion_domain = nn.BCELoss()

    def train(self, X_src, y_src, X_tar, epochs=100, batch_size=32):
        """训练 DANN 模型"""
        # 准备数据
        X_src_tensor = torch.FloatTensor(X_src)
        y_src_tensor = torch.LongTensor(y_src)
        X_tar_tensor = torch.FloatTensor(X_tar)

        # 创建数据加载器
        src_dataset = TensorDataset(X_src_tensor, y_src_tensor)
```

```
            src_loader = DataLoader(src_dataset, batch_size=batch_size, shuffle=True)
            tar_loader = DataLoader(TensorDataset(X_tar_tensor), batch_size=batch_size, shuffle=True)

            losses = []
            for epoch in range(epochs):
                # 同时迭代源域和目标域数据
                src_iter = iter(src_loader)
                tar_iter = iter(tar_loader)

                epoch_loss_class = 0
                epoch_loss_domain = 0
                for i in range(min(len(src_loader), len(tar_loader))):
                    # 获取批次数据
                    try:
                        X_src_batch, y_src_batch = next(src_iter)
                        X_tar_batch = next(tar_iter)[0]
                    except StopIteration:
                        break

                    # 创建领域标签
                    batch_size_src = X_src_batch.size(0)
                    batch_size_tar = X_tar_batch.size(0)

                    domain_src = torch.ones(batch_size_src, 1)
                    domain_tar = torch.zeros(batch_size_tar, 1)

                    # 合并数据
                    X_batch = torch.cat([X_src_batch, X_tar_batch], dim=0)
                    domain_labels = torch.cat([domain_src, domain_tar], dim=0)

                    # 前向传播
                    features = self.feature_extractor(X_batch)
                    class_outputs = self.classifier(features[:batch_size_src])
                    domain_outputs = self.domain_discriminator(features)

                    # 计算损失
                    class_loss = self.criterion_class(class_outputs, y_src_batch)
                    domain_loss = self.criterion_domain(domain_outputs, domain_labels)

                    # 总损失（包含梯度反转）
                    total_loss = class_loss + 0.1 * domain_loss

                    # 反向传播
                    self.optimizer.zero_grad()
                    total_loss.backward()
                    self.optimizer.step()

                    epoch_loss_class += class_loss.item()
                    epoch_loss_domain += domain_loss.item()

                avg_class_loss = epoch_loss_class / len(src_loader)
                avg_domain_loss = epoch_loss_domain / len(src_loader)
                total_avg_loss = avg_class_loss + 0.1 * avg_domain_loss
                losses.append(total_avg_loss)

                if epoch % 20 == 0:
                    print(
                        f"Epoch {epoch:3d} | Class Loss: {avg_class_loss:.4f} | Domain Loss: {avg_domain_loss:.4f} |
Total Loss: {total_avg_loss:.4f}")

            return losses

        def predict(self, X):
            """预测"""
            self.feature_extractor.eval()
            self.classifier.eval()

            with torch.no_grad():
                X_tensor = torch.FloatTensor(X)
                features = self.feature_extractor(X_tensor)
                outputs = self.classifier(features)
                probabilities = torch.softmax(outputs, dim=1)
                _, predictions = torch.max(outputs, 1)
```

```
        return predictions.numpy(), probabilities.numpy()

    def extract_features(self, X):
        """提取特征"""
        self.feature_extractor.eval()

        with torch.no_grad():
            X_tensor = torch.FloatTensor(X)
            features = self.feature_extractor(X_tensor)

        return features.numpy()


def plot_tsne_comparison(X_src, X_tar, y_src, target_filenames, y_pred_tar=None, title="t-SNE Visualization",
                         filename_suffix=""):
    """绘制 t-SNE 对比图并保存"""
    # 合并数据
    X_combined = np.vstack([X_src, X_tar])

    # 创建标签
    domain_labels = ['Source'] * len(X_src) + ['Target'] * len(X_tar)

    if y_pred_tar is not None:
        # 迁移后的情况
        class_labels = list(y_src) + list(y_pred_tar)
    else:
        # 迁移前的情况
        class_labels = list(y_src) + [-1] * len(X_tar)  # 使用-1 表示未知

    # t-SNE 降维
    tsne = TSNE(n_components=2, random_state=42, perplexity=min(30, len(X_combined) - 1))
    X_tsne = tsne.fit_transform(X_combined)

    # 创建子图
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

    # 左图：按领域着色
    domains = ['Source', 'Target']
    colors = ['blue', 'red']

    for i, domain in enumerate(domains):
        mask = [d == domain for d in domain_labels]
        ax1.scatter(X_tsne[mask, 0], X_tsne[mask, 1],
                    c=colors[i], label=domain, alpha=0.7, s=50)

    ax1.set_title(f'{title} - By Domain', fontsize=14, fontweight='bold')
    ax1.legend()
    ax1.set_xlabel('t-SNE Component 1')
    ax1.set_ylabel('t-SNE Component 2')
    ax1.grid(True, alpha=0.3)

    # 右图：按类别着色
    unique_classes = np.unique(class_labels)
    colors = plt.cm.Set3(np.linspace(0, 1, len(unique_classes)))

    for i, cls in enumerate(unique_classes):
        mask = [c == cls for c in class_labels]
        if cls == -1:  # 未知类别（目标域）
            color = 'gray'
            label = 'Unknown'
        else:
            color = colors[i]
            label = f'Class {cls}'

        ax2.scatter(X_tsne[mask, 0], X_tsne[mask, 1],
                    c=[color], label=label, alpha=0.7, s=50)

    ax2.set_title(f'{title} - By Class', fontsize=14, fontweight='bold')
    ax2.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    ax2.set_xlabel('t-SNE Component 1')
    ax2.set_ylabel('t-SNE Component 2')
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
```

```
    # 保存图片
    filename = f"tsne_comparison_{filename_suffix}.png" if filename_suffix else "tsne_comparison.png"
    plt.savefig(os.path.join(save_dir, filename), dpi=300, bbox_inches='tight')
    plt.show()


def create_prediction_table(target_filenames, y_pred, y_prob, class_names):
    """创建预测结果表格"""
    results = []
    for i, filename in enumerate(target_filenames):
        pred_class = class_names[y_pred[i]]
        confidence = y_prob[i].max()

        # 获取前 3 个最高概率的类别
        top3_indices = np.argsort(y_prob[i])[-3:][::-1]
        top3_probs = [f"{class_names[idx]}: {y_prob[i][idx]:.3f}" for idx in top3_indices]

        results.append({
            'Filename': filename,
            'Predicted Class': pred_class,
            'Confidence': f"{confidence:.3f}",
            'Top 3 Probabilities': ', '.join(top3_probs)
        })

    return pd.DataFrame(results)


def plot_spectrum_analysis(X_tar, target_filenames, y_pred, y_prob, class_names):
    """绘制频谱分析图并保存"""
    n_samples = len(X_tar)
    n_cols = 4
    n_rows = (n_samples + n_cols - 1) // n_cols

    fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 4 * n_rows))
    axes = axes.flatten() if hasattr(axes, 'flatten') else [axes]

    for i in range(n_samples):
        ax = axes[i]

        # 绘制频谱特征（选择前 20 个特征）
        features_to_plot = min(20, X_tar.shape[1])
        values = X_tar[i] if isinstance(X_tar, np.ndarray) else X_tar.iloc[i].values
        ax.bar(range(features_to_plot), values[:features_to_plot], alpha=0.7)

        # 添加预测信息
        pred_class = class_names[y_pred[i]]
        confidence = y_prob[i].max()

        ax.set_title(f'{target_filenames[i]}\nPred: {pred_class} (Conf: {confidence:.3f})',
                     fontsize=10, fontweight='bold')
        ax.set_xlabel('Feature Index')
        ax.set_ylabel('Feature Value')
        ax.grid(True, alpha=0.3)

    # 隐藏多余的子图
    for i in range(n_samples, len(axes)):
        axes[i].set_visible(False)

    plt.tight_layout()

    # 保存图片
    plt.savefig(os.path.join(save_dir, "spectrum_analysis.png"), dpi=300, bbox_inches='tight')
    plt.show()


def plot_domain_distance_comparison(mmd_before, mmd_after, wasserstein_before, wasserstein_after):
    """绘制域距离对比图并保存"""
    plt.figure(figsize=(10, 6))
    metrics = ['MMD', 'Wasserstein']
    before = [mmd_before, wasserstein_before]
    after = [mmd_after, wasserstein_after]

    x = np.arange(len(metrics))
```

```
    width = 0.35

    bars1 = plt.bar(x - width / 2, before, width, label='Before Adaptation', alpha=0.8, color='lightcoral')
    bars2 = plt.bar(x + width / 2, after, width, label='After Adaptation', alpha=0.8, color='lightseagreen')

    plt.xlabel('Distance Metric', fontsize=12)
    plt.ylabel('Distance Value', fontsize=12)
    plt.title('Domain Distance Comparison Before and After Adaptation',
              fontsize=14, fontweight='bold')
    plt.xticks(x, metrics)
    plt.legend(fontsize=10)
    plt.grid(True, alpha=0.3)

    # 添加数值标签
    for bar, v in zip(bars1, before):
        plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
                 f'{v:.3f}', ha='center', va='bottom', fontsize=9)

    for bar, v in zip(bars2, after):
        plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
                 f'{v:.3f}', ha='center', va='bottom', fontsize=9)

    plt.tight_layout()

    # 保存图片
    plt.savefig(os.path.join(save_dir, "domain_distance_comparison.png"), dpi=300, bbox_inches='tight')
    plt.show()


def plot_training_loss(losses):
    """绘制训练损失曲线并保存"""
    plt.figure(figsize=(10, 5))
    plt.plot(losses, linewidth=2)
    plt.title('DANN Training Loss', fontsize=14, fontweight='bold')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    # 保存图片
    plt.savefig(os.path.join(save_dir, "training_loss.png"), dpi=300, bbox_inches='tight')
    plt.show()


def k_fold_validation(X_src, y_src, n_splits=5):
    """K 折交叉验证并保存结果图"""
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)

    accuracies = []
    for fold, (train_idx, val_idx) in enumerate(kf.split(X_src)):
        # 使用 numpy 数组索引而不是 iloc
        X_train, X_val = X_src[train_idx], X_src[val_idx]
        y_train, y_val = y_src[train_idx], y_src[val_idx]

        # 创建简单分类器进行验证
        from sklearn.ensemble import RandomForestClassifier
        clf = RandomForestClassifier(n_estimators=100, random_state=42)
        clf.fit(X_train, y_train)

        y_pred = clf.predict(X_val)
        accuracy = (y_pred == y_val).mean()
        accuracies.append(accuracy)

        print(f"Fold {fold + 1}: Accuracy = {accuracy:.4f}")

    print(f"\nK-Fold Cross Validation Results:")
    print(f"Mean Accuracy: {np.mean(accuracies):.4f} (+/- {np.std(accuracies):.4f})")

    # 绘制 K 折验证结果
    plt.figure(figsize=(8, 5))
    plt.bar(range(1, n_splits + 1), accuracies, color='skyblue', alpha=0.8)
    plt.axhline(y=np.mean(accuracies), color='red', linestyle='--',
                label=f'Mean Accuracy: {np.mean(accuracies):.4f}')
    plt.xlabel('Fold Number')
```

```
    plt.ylabel('Accuracy')
    plt.title('K-Fold Cross Validation Results', fontsize=14, fontweight='bold')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.ylim(0, 1)
    plt.tight_layout()

    # 保存图片
    plt.savefig(os.path.join(save_dir, "kfold_validation.png"), dpi=300, bbox_inches='tight')
    plt.show()

    return accuracies


def main():
    """主函数"""
    print("=== Domain Adaptation Fault Diagnosis System ===\n")

    # 1. 加载数据
    X_src, y_src, X_tar, target_filenames, scaler = load_and_prepare_data()

    # 2. 对标签进行编码
    y_src_encoded, label_encoder, class_names = encode_labels(y_src)
    num_classes = len(class_names)

    print(f"\nNumber of classes: {num_classes}")
    print(f"Class names: {class_names}")
    print(f"Encoded labels: {np.unique(y_src_encoded)}")

    # 3. 计算迁移前的域差异
    print("\n=== Pre-adaptation Domain Differences ===")
    mmd_before = calculate_mmd(X_src, X_tar)
    wasserstein_before = calculate_wasserstein(X_src, X_tar)

    print(f"MMD before adaptation: {mmd_before:.4f}")
    print(f"Wasserstein distance before adaptation: {wasserstein_before:.4f}")

    # 4. 迁移前的可视化
    print("\n=== Pre-adaptation Visualization ===")
    plot_tsne_comparison(X_src, X_tar, y_src_encoded,
                         target_filenames, title="Before Adaptation", filename_suffix="before")

    # 5. 训练 DANN 模型
    print("\n=== Training DANN Model ===")
    input_dim = X_src.shape[1]
    dann_model = DANNModel(input_dim, num_classes)

    losses = dann_model.train(X_src, y_src_encoded, X_tar, epochs=100, batch_size=32)

    # 绘制训练损失
    plot_training_loss(losses)

    # 6. 预测目标域
    print("\n=== Predicting Target Domain ===")
    y_pred_tar, y_prob_tar = dann_model.predict(X_tar)

    # 7. 提取迁移后的特征
    features_src_adapted = dann_model.extract_features(X_src)
    features_tar_adapted = dann_model.extract_features(X_tar)

    # 8. 计算迁移后的域差异
    print("\n=== Post-adaptation Domain Differences ===")
    mmd_after = calculate_mmd(features_src_adapted, features_tar_adapted)
    wasserstein_after = calculate_wasserstein(features_src_adapted, features_tar_adapted)

    print(f"MMD after adaptation: {mmd_after:.4f}")
    print(f"Wasserstein distance after adaptation: {wasserstein_after:.4f}")

    # 9. 迁移后的可视化
    print("\n=== Post-adaptation Visualization ===")
    plot_tsne_comparison(features_src_adapted, features_tar_adapted,
                         y_src_encoded, target_filenames, y_pred_tar,
                         title="After Adaptation", filename_suffix="after")
```

```
    # 10. 创建预测结果表格
    print("\n=== Prediction Results ===")
    results_df = create_prediction_table(target_filenames, y_pred_tar,
                                         y_prob_tar, class_names)
    print(results_df.to_string(index=False))

    # 11. 绘制域差异对比
    plot_domain_distance_comparison(mmd_before, mmd_after, wasserstein_before, wasserstein_after)

    # 12. 绘制频谱分析
    print("\n=== Spectrum Analysis ===")
    plot_spectrum_analysis(X_tar, target_filenames, y_pred_tar, y_prob_tar, class_names)

    # 13. K 折交叉验证
    print("\n=== K-Fold Cross Validation on Source Domain ===")
    accuracies = k_fold_validation(X_src, y_src_encoded, n_splits=5)

    # 14. 保存结果
    results_df.to_csv(os.path.join(save_dir, 'target_domain_predictions.csv'), index=False)
    print(f"\n=== Results saved to '{save_dir}/target_domain_predictions.csv' ===")

    # 15. 保存域距离结果
    distance_results = pd.DataFrame({
        'Metric': ['MMD', 'Wasserstein'],
        'Before_Adaptation': [mmd_before, wasserstein_before],
        'After_Adaptation': [mmd_after, wasserstein_after],
        'Reduction_Percentage': [
            (mmd_before - mmd_after) / mmd_before * 100,
            (wasserstein_before - wasserstein_after) / wasserstein_before * 100
        ]
    })
    distance_results.to_csv(os.path.join(save_dir, 'domain_distance_results.csv'), index=False)

    print(f"\nAll visualizations have been saved to: {save_dir}")

    return results_df, mmd_before, mmd_after, wasserstein_before, wasserstein_after


if __name__ == "__main__":
    # 运行主程序
    try:
        results, mmd_b, mmd_a, wass_b, wass_a = main()

        # 打印总结
        print("\n" + "=" * 50)
        print("SUMMARY")
        print("=" * 50)
        print(f"Domain adaptation completed successfully!")
        print(f"MMD reduction: {mmd_b:.4f} → {mmd_a:.4f} ({(mmd_b - mmd_a) / mmd_b * 100:.1f}% reduction)")
        print(f"Wasserstein reduction: {wass_b:.4f} → {wass_a:.4f} ({(wass_b - wass_a) / wass_b * 100:.1f}%
reduction)")
        print(f"\nAll results and visualizations saved to: {save_dir}")

    except Exception as e:
        print(f"Error occurred: {e}")
        import traceback

        traceback.print_exc()
```

附录 4

统一 kHz 代码 Python

```
import os
import pandas as pd
import numpy as np
import scipy.io as sio
from scipy import signal
from pathlib import Path
import shutil


def load_mat_data(file_path):
    """加载.mat 文件数据"""
```

```
    try:
        mat_data = sio.loadmat(file_path)
        # .mat 文件通常包含多个变量，我们需要找到包含振动数据的变量
        # 通常变量名不是 '__header__', '__version__', '__globals__'
        data_key = None
        for key in mat_data.keys():
            if not key.startswith('__') and not key.startswith('readme'):
                data_key = key
                break

        if data_key is None:
            print(f"警告: 在文件 {file_path} 中未找到数据变量")
            return None

        data = mat_data[data_key].flatten()  # 展平为一维数组
        return data
    except Exception as e:
        print(f"错误: 加载文件 {file_path} 时出错: {e}")
        return None


def antialias_lowpass_and_resample(data, original_fs, target_fs=32000):
    """抗混叠低通滤波并降采样到目标频率"""
    # 计算降采样比率
    downsample_ratio = original_fs / target_fs

    if downsample_ratio < 1:
        print(f"错误: 原始采样率 {original_fs} 低于目标采样率 {target_fs}")
        return None

    # 设计抗混叠低通滤波器 (截止频率为目标奈奎斯特频率)
    nyquist_ratio = 0.8  # 保守的截止频率比率
    cutoff_freq = target_fs / 2 * nyquist_ratio

    # 滤波器参数
    filter_order = 8
    b, a = signal.butter(filter_order, cutoff_freq / (original_fs / 2), 'low')

    # 应用抗混叠滤波器
    filtered_data = signal.filtfilt(b, a, data)

    # 计算降采样因子（需要是整数）
    downsample_factor = int(original_fs / target_fs)

    # 降采样
    resampled_data = signal.resample(filtered_data,
                                     len(data) // downsample_factor)

    return resampled_data


def high_quality_interpolation(data, original_fs, target_fs=32000):
    """高质量插值到目标频率"""
    # 计算插值比率
    interpolation_ratio = target_fs / original_fs

    if interpolation_ratio < 1:
        print(f"错误: 目标采样率 {target_fs} 低于原始采样率 {original_fs}")
        return None

    # 创建原始时间轴
    original_time = np.arange(len(data)) / original_fs

    # 创建目标时间轴
    target_length = int(len(data) * interpolation_ratio)
    target_time = np.arange(target_length) / target_fs

    # 使用立方插值进行高质量插值
    from scipy import interpolate

    # 创建插值函数
    interp_func = interpolate.interp1d(original_time, data,
                                       kind='cubic',
```

```
                                                bounds_error=False,
                                                fill_value=(data[0], data[-1]))

    # 执行插值
    interpolated_data = interp_func(target_time)

    return interpolated_data


def process_source_data(metadata_file, output_base_dir):
    """处理源域数据并进行采样率转换"""

    # 读取元数据
    df = pd.read_csv(metadata_file)

    # 筛选源域数据
    source_df = df[df['domain'] == 'source']

    # 创建输出目录
    output_dir = os.path.join(output_base_dir, "source_32kHz")
    os.makedirs(output_dir, exist_ok=True)

    processed_files = []

    for idx, row in source_df.iterrows():
        try:
            file_path = row['file_path']
            sampling_rate = row['sampling_rate']
            filename = row['filename']

            print(f"处理文件: {filename} (采样率: {sampling_rate})")

            # 加载数据
            data = load_mat_data(file_path)
            if data is None:
                continue

            # 根据采样率选择处理方法
            if sampling_rate == '48kHz':
                original_fs = 48000
                processed_data = antialias_lowpass_and_resample(data, original_fs, 32000)

            elif sampling_rate == '12kHz':
                original_fs = 12000
                processed_data = high_quality_interpolation(data, original_fs, 32000)

            else:
                print(f"未知采样率: {sampling_rate}, 跳过文件: {filename}")
                continue

            if processed_data is None:
                print(f"处理失败: {filename}")
                continue

            # 创建输出文件路径
            # 保持原始文件夹结构但修改采样率信息
            relative_path = os.path.relpath(file_path, "/Users/slh/Downloads/planB/data/source_data")

            # 修改文件夹名中的采样率信息
            path_parts = relative_path.split(os.sep)
            if len(path_parts) > 0:
                # 修改第一个文件夹名（包含采样率信息）
                first_folder = path_parts[0]
                if '12kHz' in first_folder:
                    new_first_folder = first_folder.replace('12kHz', '32kHz')
                elif '48kHz' in first_folder:
                    new_first_folder = first_folder.replace('48kHz', '32kHz')
                else:
                    new_first_folder = '32kHz_' + first_folder

                path_parts[0] = new_first_folder

            new_relative_path = os.sep.join(path_parts)
```

```
                output_file_path = os.path.join(output_dir, new_relative_path)

                # 创建输出目录
                os.makedirs(os.path.dirname(output_file_path), exist_ok=True)

                # 保存处理后的数据
                sio.savemat(output_file_path, {'vibration_data': processed_data})

                # 更新元数据信息
                new_metadata = row.copy()
                new_metadata['file_path'] = output_file_path
                new_metadata['sampling_rate'] = '32kHz'
                new_metadata['original_sampling_rate'] = sampling_rate  # 保留原始采样率信息

                processed_files.append(new_metadata)

                print(f"成功处理: {filename} -> {output_file_path}")

            except Exception as e:
                print(f"处理文件 {row['filename']} 时出错: {e}")
                continue

    return processed_files


def update_metadata(original_metadata_file, processed_files, output_dir):
    """更新元数据文件"""

    # 读取原始元数据
    original_df = pd.read_csv(original_metadata_file)

    # 创建处理后的数据 DataFrame
    processed_df = pd.DataFrame(processed_files)

    # 从原始数据中移除已处理的源域文件
    original_source_mask = original_df['domain'] == 'source'
    original_df = original_df[~original_source_mask]

    # 合并处理后的源域数据和其他数据
    updated_df = pd.concat([original_df, processed_df], ignore_index=True)

    # 保存更新后的元数据
    updated_metadata_file = os.path.join(output_dir, "metadata_updated.csv")
    updated_df.to_csv(updated_metadata_file, index=False)

    # 同时覆盖原始文件（可选）
    updated_df.to_csv(original_metadata_file, index=False)

    print(f"元数据已更新并保存到: {updated_metadata_file}")
    return updated_metadata_file


def main():
    # 设置路径
    metadata_file = "/Users/slh/Downloads/planB/metadata.csv"
    output_base_dir = "/Users/slh/Downloads/planB"

    # 检查元数据文件是否存在
    if not os.path.exists(metadata_file):
        print(f"元数据文件不存在: {metadata_file}")
        return

    # 处理源域数据
    print("开始处理源域数据...")
    processed_files = process_source_data(metadata_file, output_base_dir)

    print(f"成功处理 {len(processed_files)} 个文件")

    # 更新元数据
    print("更新元数据...")
    updated_metadata_file = update_metadata(metadata_file, processed_files, output_base_dir)
```

```
    # 显示统计信息
    print("\n 处理完成!")
    print(f"处理后的数据保存在: {os.path.join(output_base_dir, 'source_32kHz')}")
    print(f"更新后的元数据保存在: {updated_metadata_file}")

    # 显示处理统计
    sampling_rates = {}
    for file in processed_files:
        original_rate = file['original_sampling_rate']
        if original_rate in sampling_rates:
            sampling_rates[original_rate] += 1
        else:
            sampling_rates[original_rate] = 1

    print("\n 处理统计:")
    for rate, count in sampling_rates.items():
        print(f"{rate} -> 32kHz: {count} 个文件")


if __name__ == "__main__":
    main()
```

附录 5

TCA-XGBoost 代码 Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.manifold import TSNE
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import xgboost as xgb
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import warnings
import matplotlib.pyplot as plt
from matplotlib.font_manager import FontProperties, fontManager
import os

warnings.filterwarnings('ignore')

# 设置中文字体和图形样式
# 注册字体
font_path = '/System/Library/Fonts/STHeiti Medium.ttc'

# 使用 FontProperties 获取字体名称
custom_font = FontProperties(fname=font_path)
font_name = custom_font.get_name()

# 设置全局字体
plt.rcParams['font.sans-serif'] = [font_name, 'Arial', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
sns.set_style("whitegrid")
sns.set_palette("husl")


class TCA:
    """Transfer Component Analysis (TCA) 实现"""

    def __init__(self, kernel_type='linear', dim=30, lamb=1, gamma=1):
        self.kernel_type = kernel_type
        self.dim = dim
        self.lamb = lamb
        self.gamma = gamma

    def fit(self, Xs, Xt):
        """训练 TCA 变换"""
        X = np.hstack((Xs.T, Xt.T))
        X /= np.linalg.norm(X, axis=0)
        m, n = X.shape
```

```
        ns, nt = len(Xs), len(Xt)
        self.ns, self.nt = ns, nt

        # 构建 MMD 矩阵
        e = np.vstack((1 / ns * np.ones((ns, 1)), -1 / nt * np.ones((nt, 1))))
        M = e * e.T

        # 构建中心矩阵
        H = np.eye(n) - 1 / n * np.ones((n, n))

        # 核矩阵
        K = self.kernel(self.kernel_type, X, None, self.gamma)

        # 求解特征值问题
        A = np.linalg.multi_dot([K, M, K.T]) + self.lamb * np.eye(m)
        B = np.linalg.multi_dot([K, H, K.T])

        # 广义特征值分解
        eigvals, eigvecs = np.linalg.eig(np.linalg.pinv(B).dot(A))
        idx = np.argsort(eigvals)[:self.dim]

        self.W = eigvecs[:, idx]
        return self

    def transform(self, Xs, Xt):
        """应用 TCA 变换"""
        X = np.hstack((Xs.T, Xt.T))
        X /= np.linalg.norm(X, axis=0)
        K = self.kernel(self.kernel_type, X, None, self.gamma)
        X_new = np.dot(K.T, self.W)

        Xs_new = X_new[:self.ns]
        Xt_new = X_new[self.ns:]

        return Xs_new, Xt_new

    def kernel(self, kernel_type, X, Y, gamma):
        """核函数"""
        if kernel_type == 'linear':
            if Y is None:
                return np.dot(X.T, X)
            else:
                return np.dot(X.T, Y)
        elif kernel_type == 'rbf':
            if Y is None:
                Y = X
            n = X.shape[1]
            m = Y.shape[1]
            K = np.zeros((n, m))
            for i in range(n):
                for j in range(m):
                    K[i, j] = np.exp(-gamma * np.linalg.norm(X[:, i] - Y[:, j]) ** 2)
            return K
        else:
            raise ValueError('不支持的核类型')


def load_and_prepare_data():
    """加载和准备源域和目标域数据"""

    # 加载源域数据（训练集和测试集）
    train_src = pd.read_csv('/Users/slh/Downloads/数据集/train_features_standardized.csv')
    test_src = pd.read_csv('/Users/slh/Downloads/数据集/test_features_standardized.csv')

    # 合并源域数据
    X_src = pd.concat([train_src, test_src], ignore_index=True)

    # 移除不需要的列，保留特征
    src_features_to_remove = ['filename', 'fault_size', 'load']
    X_src_features = X_src.drop(columns=src_features_to_remove, errors='ignore')

    # 分离特征和标签
    y_src = X_src_features['fault_type']
```

```
    X_src_final = X_src_features.drop('fault_type', axis=1)

    # 加载目标域数据
    target_data = pd.read_csv('/Users/slh/Downloads/数据集/target_features_standardized.csv')

    # 移除目标域不需要的列
    tar_features_to_remove = ['filename', 'fault_type', 'fault_size', 'load']
    X_tar = target_data.drop(columns=tar_features_to_remove, errors='ignore')

    # 获取目标域文件名（A-P）
    target_filenames = target_data['filename'].tolist()

    print(f"源域数据形状: {X_src_final.shape}")
    print(f"源域标签形状: {y_src.shape}")
    print(f"目标域数据形状: {X_tar.shape}")
    print(f"目标域文件数量: {len(target_filenames)}")

    return X_src_final, y_src, X_tar, target_filenames


def prepare_tca_data(X_src_train, X_tar_scaled):
    """准备 TCA 数据"""

    # 合并源域和目标域特征
    X_combined = np.vstack([X_src_train, X_tar_scaled])

    # 创建域标签：源域为 1，目标域为 0
    domain_labels = np.zeros(X_combined.shape[0])
    domain_labels[:len(X_src_train)] = 1  # 源域样本标记为 1

    print(f"合并后数据形状: {X_combined.shape}")
    print(f"源域样本数: {len(X_src_train)}")
    print(f"目标域样本数: {len(X_tar_scaled)}")
    print(f"域标签分布 - 源域(1): {np.sum(domain_labels == 1)}，目标域(0): {np.sum(domain_labels == 0)}")

    return X_combined, domain_labels


def train_tca_xgboost(X_src_train, y_src_train, X_tar_scaled):
    """训练 TCA + XGBoost 模型"""

    # 准备 TCA 数据
    X_combined, domain_labels = prepare_tca_data(X_src_train, X_tar_scaled)

    # 训练 TCA 变换器
    print("训练 TCA 变换器...")
    tca = TCA(kernel_type='linear', dim=min(30, X_combined.shape[1]), lamb=1, gamma=1)
    tca.fit(X_src_train.values, X_tar_scaled.values)

    # 应用 TCA 变换
    X_src_tca, X_tar_tca = tca.transform(X_src_train.values, X_tar_scaled.values)

    print(f"TCA 变换后源域形状: {X_src_tca.shape}")
    print(f"TCA 变换后目标域形状: {X_tar_tca.shape}")

    # 在 TCA 空间训练 XGBoost 分类器
    print("训练 XGBoost 分类器...")
    xgb_model = xgb.XGBClassifier(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        eval_metric='mlogloss'
    )

    # 分割源域数据用于验证
    X_train, X_val, y_train, y_val = train_test_split(
        X_src_tca, y_src_train, test_size=0.2, random_state=42, stratify=y_src_train
    )
```

```
    # 训练模型
    xgb_model.fit(X_train, y_train,
                  eval_set=[(X_val, y_val)],
                  early_stopping_rounds=10,
                  verbose=False)

    # 在验证集上评估
    y_val_pred = xgb_model.predict(X_val)
    val_accuracy = accuracy_score(y_val, y_val_pred)
    print(f"验证集准确率: {val_accuracy:.4f}")

    return tca, xgb_model, X_src_tca, X_tar_tca, X_combined, domain_labels


def predict_target_domain_tca(xgb_model, X_tar_tca, target_filenames):
    """预测目标域数据"""

    # 预测目标域
    y_tar_pred = xgb_model.predict(X_tar_tca)
    y_tar_pred_proba = xgb_model.predict_proba(X_tar_tca)

    # 获取置信度（最大概率）
    confidence_scores = np.max(y_tar_pred_proba, axis=1)

    print("目标域预测完成!")
    return y_tar_pred, confidence_scores, y_tar_pred_proba


def plot_tsne_comparison_tca(X_src_train, X_tar_scaled, y_src_train,
                             X_src_tca, X_tar_tca, target_filenames,
                             y_tar_pred, le):
    """绘制 TCA 迁移前后的 t-SNE 对比图"""

    # 迁移前：原始特征
    print("正在进行迁移前 t-SNE 降维...")
    tsne_original = TSNE(n_components=2, random_state=42, perplexity=min(30, len(X_src_train) // 4))

    # 合并源域和目标域原始特征
    original_combined = np.vstack([X_src_train.values, X_tar_scaled.values])
    original_tsne = tsne_original.fit_transform(original_combined)

    # 迁移后：TCA 特征
    print("正在进行迁移后 t-SNE 降维...")
    tsne_tca = TSNE(n_components=2, random_state=42, perplexity=min(30, len(X_src_tca) // 4))
    tca_combined = np.vstack([X_src_tca, X_tar_tca])
    tca_tsne = tsne_tca.fit_transform(tca_combined)

    # 创建图形
    fig, axes = plt.subplots(2, 3, figsize=(24, 16))

    # 迁移前 t-SNE 可视化
    split_point = len(X_src_train)
    src_original = original_tsne[:split_point]
    tar_original = original_tsne[split_point:]

    # 子图 1：迁移前（按域着色）
    axes[0, 0].scatter(src_original[:, 0], src_original[:, 1],
                       c='blue', alpha=0.6, label='Source Domain', s=50)
    axes[0, 0].scatter(tar_original[:, 0], tar_original[:, 1],
                       c='red', alpha=0.6, label='Target Domain', s=50)
    axes[0, 0].set_title('Before TCA Adaptation\n(Colored by Domain)', fontsize=14, fontweight='bold')
    axes[0, 0].set_xlabel('t-SNE Component 1')
    axes[0, 0].set_ylabel('t-SNE Component 2')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)

    # 子图 2：迁移前（按故障类型着色）
    unique_labels = le.classes_
    colors = plt.cm.Set3(np.linspace(0, 1, len(unique_labels)))

    for i, label in enumerate(unique_labels):
        mask = (y_src_train == label)
        if mask.any():
```

```
            axes[0, 1].scatter(src_original[mask, 0], src_original[mask, 1],
                               c=[colors[i]], label=label, alpha=0.7, s=50)

    # 标记目标域样本
    for i, filename in enumerate(target_filenames):
        axes[0, 1].scatter(tar_original[i, 0], tar_original[i, 1],
                           c='black', marker='*', s=150, alpha=0.8, label=f'Target {filename}' if i == 0 else "")

    axes[0, 1].set_title('Before TCA Adaptation\n(Colored by Fault Type)', fontsize=14, fontweight='bold')
    axes[0, 1].set_xlabel('t-SNE Component 1')
    axes[0, 1].set_ylabel('t-SNE Component 2')
    axes[0, 1].legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    axes[0, 1].grid(True, alpha=0.3)

    # 子图 3：迁移前特征分布密度
    src_density = np.linalg.norm(src_original, axis=1)
    tar_density = np.linalg.norm(tar_original, axis=1)

    axes[0, 2].hist(src_density, bins=30, alpha=0.7, label='Source Domain', color='blue', density=True)
    axes[0, 2].hist(tar_density, bins=30, alpha=0.7, label='Target Domain', color='red', density=True)
    axes[0, 2].set_title('Feature Distribution Before TCA', fontsize=14, fontweight='bold')
    axes[0, 2].set_xlabel('Feature Norm')
    axes[0, 2].set_ylabel('Density')
    axes[0, 2].legend()
    axes[0, 2].grid(True, alpha=0.3)

    # 迁移后 t-SNE 可视化
    src_tca = tca_tsne[:len(X_src_tca)]
    tar_tca = tca_tsne[len(X_src_tca):]

    # 子图 4：迁移后（按域着色）
    axes[1, 0].scatter(src_tca[:, 0], src_tca[:, 1],
                       c='blue', alpha=0.6, label='Source Domain', s=50)
    axes[1, 0].scatter(tar_tca[:, 0], tar_tca[:, 1],
                       c='red', alpha=0.6, label='Target Domain', s=50)
    axes[1, 0].set_title('After TCA Adaptation\n(Colored by Domain)', fontsize=14, fontweight='bold')
    axes[1, 0].set_xlabel('t-SNE Component 1')
    axes[1, 0].set_ylabel('t-SNE Component 2')
    axes[1, 0].legend()
    axes[1, 0].grid(True, alpha=0.3)

    # 子图 5：迁移后（按故障类型着色）
    for i, label in enumerate(unique_labels):
        mask = (y_src_train == label)
        if mask.any():
            axes[1, 1].scatter(src_tca[mask, 0], src_tca[mask, 1],
                               c=[colors[i]], label=label, alpha=0.7, s=50)

    # 标记目标域样本及其预测标签
    for i, (filename, pred_label) in enumerate(zip(target_filenames, y_tar_pred)):
        color_idx = np.where(unique_labels == pred_label)[0][0]
        axes[1, 1].scatter(tar_tca[i, 0], tar_tca[i, 1],
                           c=[colors[color_idx]], marker='*', s=150, alpha=0.8,
                           label=f'Target {filename} ({pred_label})' if i == 0 else "")

    axes[1, 1].set_title('After TCA Adaptation\n(Colored by Predicted Fault Type)', fontsize=14, fontweight='bold')
    axes[1, 1].set_xlabel('t-SNE Component 1')
    axes[1, 1].set_ylabel('t-SNE Component 2')
    axes[1, 1].legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    axes[1, 1].grid(True, alpha=0.3)

    # 子图 6：迁移后特征分布密度
    src_tca_density = np.linalg.norm(src_tca, axis=1)
    tar_tca_density = np.linalg.norm(tar_tca, axis=1)

    axes[1, 2].hist(src_tca_density, bins=30, alpha=0.7, label='Source Domain', color='blue', density=True)
    axes[1, 2].hist(tar_tca_density, bins=30, alpha=0.7, label='Target Domain', color='red', density=True)
    axes[1, 2].set_title('Feature Distribution After TCA', fontsize=14, fontweight='bold')
    axes[1, 2].set_xlabel('Feature Norm')
    axes[1, 2].set_ylabel('Density')
    axes[1, 2].legend()
    axes[1, 2].grid(True, alpha=0.3)

    plt.tight_layout()
```

```
    plt.show()

    return original_tsne, tca_tsne


def create_prediction_table_tca(target_filenames, predicted_labels, confidence_scores, le):
    """创建 TCA 预测结果表格"""

    results_df = pd.DataFrame({
        'Filename': target_filenames,
        'Predicted_Fault_Type': predicted_labels,
        'Confidence': confidence_scores
    })

    # 按文件名排序（A-P）
    results_df['File_Index'] = results_df['Filename'].apply(
        lambda x: ord(x[0]) if isinstance(x, str) and len(x) > 0 else 0)
    results_df = results_df.sort_values('File_Index').drop('File_Index', axis=1)

    # 创建精美的表格可视化
    plt.figure(figsize=(14, 10))
    ax = plt.subplot(111, frame_on=False)
    ax.xaxis.set_visible(False)
    ax.yaxis.set_visible(False)

    # 创建表格数据
    table_data = []
    for _, row in results_df.iterrows():
        table_data.append([row['Filename'], row['Predicted_Fault_Type'], f"{row['Confidence']:.4f}"])

    # 创建表格
    table = plt.table(cellText=table_data,
                      colLabels=['Filename', 'Predicted Fault Type', 'Confidence'],
                      colWidths=[0.15, 0.35, 0.25],
                      cellLoc='center',
                      loc='center',
                      bbox=[0.1, 0.1, 0.8, 0.8])

    # 美化表格
    table.auto_set_font_size(False)
    table.set_fontsize(11)
    table.scale(1, 1.8)

    # 设置标题
    plt.title('TCA + XGBoost: Target Domain Prediction Results (A-P)',
              fontsize=16, fontweight='bold', pad=30)
    plt.tight_layout()
    plt.show()

    return results_df


def plot_feature_importance(xgb_model, feature_names=None, top_k=20):
    """绘制特征重要性图"""

    # 获取特征重要性
    importance_scores = xgb_model.feature_importances_

    if feature_names is None:
        feature_names = [f'Feature_{i}' for i in range(len(importance_scores))]

    # 选择最重要的特征
    indices = np.argsort(importance_scores)[::-1][:top_k]

    plt.figure(figsize=(12, 8))
    bars = plt.barh(range(top_k), importance_scores[indices][::-1],
                    color=plt.cm.viridis(np.linspace(0, 1, top_k)))

    plt.yticks(range(top_k), [feature_names[i] for i in indices[::-1]])
    plt.xlabel('Feature Importance')
    plt.title(f'Top {top_k} Feature Importance (XGBoost)', fontsize=14, fontweight='bold')
    plt.grid(True, alpha=0.3)

    # 添加数值标签
```

```
    for i, bar in enumerate(bars):
        width = bar.get_width()
        plt.text(width + 0.001, bar.get_y() + bar.get_height() / 2,
                 f'{width:.3f}', ha='left', va='center', fontsize=9)

    plt.tight_layout()
    plt.show()


def plot_confidence_analysis_tca(target_filenames, confidence_scores, predicted_labels):
    """绘制置信度分析图"""

    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # 置信度按文件排序
    sorted_indices = np.argsort(target_filenames)
    sorted_filenames = [target_filenames[i] for i in sorted_indices]
    sorted_confidence = [confidence_scores[i] for i in sorted_indices]
    sorted_labels = [predicted_labels[i] for i in sorted_indices]

    # 置信度条形图
    bars = axes[0, 0].bar(range(len(sorted_filenames)), sorted_confidence,
                          color=plt.cm.viridis(np.linspace(0, 1, len(sorted_filenames))))
    axes[0, 0].set_title('Confidence Scores by File (A-P)', fontsize=14, fontweight='bold')
    axes[0, 0].set_xlabel('File Name')
    axes[0, 0].set_ylabel('Confidence Score')
    axes[0, 0].set_xticks(range(len(sorted_filenames)))
    axes[0, 0].set_xticklabels(sorted_filenames, rotation=45)
    axes[0, 0].grid(True, alpha=0.3)

    # 在条形图上添加数值标签
    for i, (bar, confidence) in enumerate(zip(bars, sorted_confidence)):
        height = bar.get_height()
        axes[0, 0].text(bar.get_x() + bar.get_width() / 2., height + 0.01,
                        f'{confidence:.3f}', ha='center', va='bottom', fontsize=9)

    # 按故障类型分组的置信度箱线图
    confidence_by_fault = {}
    for fault, confidence in zip(predicted_labels, confidence_scores):
        if fault not in confidence_by_fault:
            confidence_by_fault[fault] = []
        confidence_by_fault[fault].append(confidence)

    fault_types = list(confidence_by_fault.keys())
    confidence_values = [confidence_by_fault[fault] for fault in fault_types]

    if len(fault_types) > 0:
        boxplot = axes[0, 1].boxplot(confidence_values, labels=fault_types, patch_artist=True)
        axes[0, 1].set_title('Confidence Distribution by Fault Type', fontsize=14, fontweight='bold')
        axes[0, 1].set_xlabel('Fault Type')
        axes[0, 1].set_ylabel('Confidence Score')
        axes[0, 1].grid(True, alpha=0.3)

        # 美化箱线图
        colors = plt.cm.Set3(np.linspace(0, 1, len(fault_types)))
        for patch, color in zip(boxplot['boxes'], colors):
            patch.set_facecolor(color)
            patch.set_alpha(0.7)

    # 置信度分布直方图
    axes[1, 0].hist(confidence_scores, bins=20, alpha=0.7, color='skyblue', edgecolor='black')
    axes[1, 0].axvline(np.mean(confidence_scores), color='red', linestyle='--',
                       label=f'Mean: {np.mean(confidence_scores):.3f}')
    axes[1, 0].set_title('Confidence Score Distribution', fontsize=14, fontweight='bold')
    axes[1, 0].set_xlabel('Confidence Score')
    axes[1, 0].set_ylabel('Frequency')
    axes[1, 0].legend()
    axes[1, 0].grid(True, alpha=0.3)

    # 故障类型分布饼图
    unique_preds, pred_counts = np.unique(predicted_labels, return_counts=True)
    if len(unique_preds) > 0:
        wedges, texts, autotexts = axes[1, 1].pie(pred_counts, labels=unique_preds, autopct='%1.1f%%',
                                                  startangle=90,
```

```
                                                colors=plt.cm.Set3(np.linspace(0, 1, len(unique_preds))))
        axes[1, 1].set_title('Predicted Fault Type Distribution', fontsize=14, fontweight='bold')

        # 美化饼图文字
        for autotext in autotexts:
            autotext.set_color('white')
            autotext.set_fontweight('bold')

    plt.tight_layout()
    plt.show()


def plot_domain_distribution_comparison(X_src_train, X_tar_scaled, X_src_tca, X_tar_tca):
    """绘制域分布对比图"""

    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # 原始特征均值分布
    src_means_orig = X_src_train.mean(axis=1)
    tar_means_orig = X_tar_scaled.mean(axis=1)

    axes[0, 0].hist(src_means_orig, bins=30, alpha=0.7, label='Source Domain', color='blue', density=True)
    axes[0, 0].hist(tar_means_orig, bins=30, alpha=0.7, label='Target Domain', color='red', density=True)
    axes[0, 0].set_title('Original Feature Means Distribution', fontsize=12, fontweight='bold')
    axes[0, 0].set_xlabel('Feature Mean')
    axes[0, 0].set_ylabel('Density')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)

    # TCA 特征均值分布
    src_means_tca = X_src_tca.mean(axis=1)
    tar_means_tca = X_tar_tca.mean(axis=1)

    axes[0, 1].hist(src_means_tca, bins=30, alpha=0.7, label='Source Domain', color='blue', density=True)
    axes[0, 1].hist(tar_means_tca, bins=30, alpha=0.7, label='Target Domain', color='red', density=True)
    axes[0, 1].set_title('TCA Feature Means Distribution', fontsize=12, fontweight='bold')
    axes[0, 1].set_xlabel('Feature Mean')
    axes[0, 1].set_ylabel('Density')
    axes[0, 1].legend()
    axes[0, 1].grid(True, alpha=0.3)

    # 原始特征方差分布
    src_vars_orig = X_src_train.var(axis=1)
    tar_vars_orig = X_tar_scaled.var(axis=1)

    axes[1, 0].hist(src_vars_orig, bins=30, alpha=0.7, label='Source Domain', color='blue', density=True)
    axes[1, 0].hist(tar_vars_orig, bins=30, alpha=0.7, label='Target Domain', color='red', density=True)
    axes[1, 0].set_title('Original Feature Variance Distribution', fontsize=12, fontweight='bold')
    axes[1, 0].set_xlabel('Feature Variance')
    axes[1, 0].set_ylabel('Density')
    axes[1, 0].legend()
    axes[1, 0].grid(True, alpha=0.3)

    # TCA 特征方差分布
    src_vars_tca = X_src_tca.var(axis=1)
    tar_vars_tca = X_tar_tca.var(axis=1)

    axes[1, 1].hist(src_vars_tca, bins=30, alpha=0.7, label='Source Domain', color='blue', density=True)
    axes[1, 1].hist(tar_vars_tca, bins=30, alpha=0.7, label='Target Domain', color='red', density=True)
    axes[1, 1].set_title('TCA Feature Variance Distribution', fontsize=12, fontweight='bold')
    axes[1, 1].set_xlabel('Feature Variance')
    axes[1, 1].set_ylabel('Density')
    axes[1, 1].legend()
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()


# 主程序
if __name__ == "__main__":
    # 1. 加载数据
    print("=== TCA + XGBoost 模型 ===")
    print("数据加载中...")
```

```
X_src, y_src, X_tar, target_filenames = load_and_prepare_data()

# 2. 标签编码
le = LabelEncoder()
y_src_encoded = le.fit_transform(y_src)

# 3. 训练 TCA + XGBoost 模型
print("\n=== 训练 TCA + XGBoost 模型 ===")
tca, xgb_model, X_src_tca, X_tar_tca, X_combined, domain_labels = train_tca_xgboost(
    X_src, y_src, X_tar
)

# 4. 预测目标域
print("\n=== 目标域预测 ===")
y_tar_pred, confidence_scores, y_tar_pred_proba = predict_target_domain_tca(
    xgb_model, X_tar_tca, target_filenames
)

# 5. 可视化分析
print("\n=== 可视化分析 ===")

# t-SNE 对比可视化
original_tsne, tca_tsne = plot_tsne_comparison_tca(
    X_src, X_tar, y_src, X_src_tca, X_tar_tca, target_filenames, y_tar_pred, le
)

# 创建预测结果表格
results_df = create_prediction_table_tca(target_filenames, y_tar_pred, confidence_scores, le)

# 特征重要性可视化
plot_feature_importance(xgb_model, top_k=15)

# 置信度分析
plot_confidence_analysis_tca(target_filenames, confidence_scores, y_tar_pred)

# 域分布对比
plot_domain_distribution_comparison(X_src, X_tar, X_src_tca, X_tar_tca)

# 6. 打印详细结果
print("\n=== 详细预测结果 ===")
print("目标域样本预测结果:")
print("=" * 60)
for filename, pred_label, confidence in zip(target_filenames, y_tar_pred, confidence_scores):
    print(f"文件: {filename:>3} | 预测故障类型: {pred_label:>15} | 置信度: {confidence:.4f}")

print("\n 故障类型统计:")
print("=" * 30)
unique, counts = np.unique(y_tar_pred, return_counts=True)
for fault_type, count in zip(unique, counts):
    print(f"{fault_type:>15}: {count:>2}个样本")

print(f"\n 模型性能统计:")
print("=" * 30)
print(f"平均置信度: {np.mean(confidence_scores):.4f}")
print(f"置信度标准差: {np.std(confidence_scores):.4f}")
print(f"最低置信度: {np.min(confidence_scores):.4f}")
print(f"最高置信度: {np.max(confidence_scores):.4f}")
print(f"目标域样本数: {len(target_filenames)}")

# 保存预测结果到 CSV 文件
results_df.to_csv('/Users/slh/Downloads/数据集/tca_xgboost_predictions.csv', index=False)
print(f"\n 预测结果已保存到: /Users/slh/Downloads/数据集/tca_xgboost_predictions.csv")

print("\n=== TCA + XGBoost 模型训练和预测完成 ===")
```

附录 6

SVM-XGboost-随机森林代码 r

```
# 加载所需库
library(tidyverse)
```

```
library(caret)
library(e1071)
library(xgboost)
library(randomForest)
library(ggplot2)
library(ggcorrplot)
library(gridExtra)
library(viridis)
library(patchwork)
library(showtext)

showtext_auto()


# 设置中文显示
if (.Platform$OS.type == "windows") {
  windowsFonts(
    SimHei = windowsFont("SimHei"),
    Song = windowsFont("SimSun"),
    Kai = windowsFont("KaiTi")
  )
} else {
  # 非 Windows 系统可能需要其他字体设置
  theme_update(text = element_text(family = "Arial Unicode MS"))
}

# 设置随机种子，确保结果可复现
set.seed(123)

# 1. 数据读取与预处理
# 假设训练集和测试集分别保存为 train_data.csv 和 test_data.csv
train_data <- read.csv("/Users/slh/Downloads/planB/train_features.csv", stringsAsFactors = FALSE)
test_data <- read.csv("/Users/slh/Downloads/planB/test_features.csv", stringsAsFactors = FALSE)

# 查看数据结构
str(train_data)
str(test_data)

# 提取特征和目标变量
# 将 fault_type 作为目标变量
y_train <- factor(train_data$fault_class)
y_test <- factor(test_data$fault_class)

# 除去 filename、fault_size、load 和 fault_type 外的变量作为特征
exclude_cols <- c("file_path", "fault_class")
X_train <- train_data[, !(colnames(train_data) %in% exclude_cols)]
X_test <- test_data[, !(colnames(test_data) %in% exclude_cols)]

# 检查是否有缺失值
sum(is.na(X_train))
sum(is.na(X_test))

# 如果有缺失值，进行简单填充（根据实际情况调整）
if (sum(is.na(X_train)) > 0) {
  preProcess_missingdata <- preProcess(X_train, method = "medianImpute")
  X_train <- predict(preProcess_missingdata, newdata = X_train)
  X_test <- predict(preProcess_missingdata, newdata = X_test)
}

# 特征标准化
preProcess_range_model <- preProcess(X_train, method = "range")
X_train_scaled <- predict(preProcess_range_model, newdata = X_train)
X_test_scaled <- predict(preProcess_range_model, newdata = X_test)

# 2. 支持向量机(SVM)模型
# 训练 SVM 模型
svm_model <- svm(x = X_train_scaled, y = y_train, kernel = "radial", cost = 1, gamma = 0.1)

# 在测试集上预测
svm_pred <- predict(svm_model, X_test_scaled)

# 计算准确率
svm_acc <- mean(svm_pred == y_test)
```

```
cat("SVM 模型测试集准确率:", svm_acc, "\n")

# 3. XGBoost 模型
# 数据转换为 xgb 矩阵格式
dtrain <- xgb.DMatrix(data = as.matrix(X_train_scaled), label = as.numeric(y_train) - 1)
dtest <- xgb.DMatrix(data = as.matrix(X_test_scaled), label = as.numeric(y_test) - 1)

# 设置 5 折交叉验证
cv_folds <- createFolds(y_train, k = 5, returnTrain = TRUE)

# 定义超参数搜索范围
param_grid <- expand.grid(
  nrounds = c(100, 200, 300),
  max_depth = c(3, 5, 7),
  eta = c(0.01, 0.1, 0.3),
  subsample = c(0.7, 0.8, 0.9),
  colsample_bytree = c(0.7, 0.8, 0.9),
  min_child_weight = 1,
  gamma = 0,
  lambda = 1,
  alpha = 0
)

# 存储交叉验证结果
cv_results <- data.frame()

# 超参数搜索
for (i in 1:nrow(param_grid)) {
  params <- list(
    objective = "multi:softmax",
    num_class = length(levels(y_train)),
    max_depth = param_grid$max_depth[i],
    eta = param_grid$eta[i],
    subsample = param_grid$subsample[i],
    colsample_bytree = param_grid$colsample_bytree[i],
    min_child_weight = param_grid$min_child_weight[i],
    gamma = param_grid$gamma[i],
    lambda = param_grid$lambda[i],
    alpha = param_grid$alpha[i]
  )

  # 5 折交叉验证
  xgb_cv <- xgb.cv(
    params = params,
    data = dtrain,
    nrounds = param_grid$nrounds[i],
    folds = cv_folds,
    metrics = "merror",
    early_stopping_rounds = 50,
    verbose = 0
  )

  # 保存结果
  cv_results <- rbind(cv_results, data.frame(
    nrounds = param_grid$nrounds[i],
    max_depth = param_grid$max_depth[i],
    eta = param_grid$eta[i],
    subsample = param_grid$subsample[i],
    colsample_bytree = param_grid$colsample_bytree[i],
    mean_train_error = mean(xgb_cv$evaluation_log$train_merror_mean),
    mean_test_error = mean(xgb_cv$evaluation_log$test_merror_mean)
  ))

  # 打印进度
  if (i %% 10 == 0) {
    cat("完成", i, "/", nrow(param_grid), "轮超参数搜索\n")
  }
}

# 找出最佳参数（测试误差最小）
best_params_idx <- which.min(cv_results$mean_test_error)
best_params <- param_grid[best_params_idx, ]
cat("XGBoost 最佳参数:\n")
print(best_params)
```

```
# 超参数搜索结果可视化
cv_results_long <- cv_results %>%
  select(-mean_train_error) %>%
  pivot_longer(cols = mean_test_error, names_to = "type", values_to = "error")

# 学习率与误差关系
p1 <- ggplot(cv_results, aes(x = as.factor(eta), y = mean_test_error)) +
  geom_boxplot(fill = "#4285f4") +
  labs(x = "学习率  (eta)", y = "平均测试误差", title = "学习率对模型性能的影响") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

# 树深度与误差关系
p2 <- ggplot(cv_results, aes(x = as.factor(max_depth), y = mean_test_error)) +
  geom_boxplot(fill = "#ea4335") +
  labs(x = "树的最大深度", y = "平均测试误差", title = "树深度对模型性能的影响") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

# 子样本比例与误差关系
p3 <- ggplot(cv_results, aes(x = as.factor(subsample), y = mean_test_error)) +
  geom_boxplot(fill = "#fbbc05") +
  labs(x = "子样本比例", y = "平均测试误差", title = "子样本比例对模型性能的影响") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

# 特征采样比例与误差关系
p4 <- ggplot(cv_results, aes(x = as.factor(colsample_bytree), y = mean_test_error)) +
  geom_boxplot(fill = "#34a853") +
  labs(x = "特征采样比例", y = "平均测试误差", title = "特征采样比例对模型性能的影响") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

# 组合图形
grid.arrange(p1, p2, p3, p4, ncol = 2)
ggsave("xgb_hyperparameter_impact.png", width = 12, height = 10, dpi = 300)

# 使用最佳参数在整个训练集上重新训练模型
xgb_final_params <- list(
  objective = "multi:softmax",
  num_class = length(levels(y_train)),
  max_depth = best_params$max_depth,
  eta = best_params$eta,
  subsample = best_params$subsample,
  colsample_bytree = best_params$colsample_bytree,
  min_child_weight = 1,
  gamma = 0,
  lambda = 1,
  alpha = 0
)

xgb_model <- xgb.train(
  params = xgb_final_params,
  data = dtrain,
  nrounds = best_params$nrounds,
  verbose = 0
)

# 在测试集上预测
xgb_pred <- predict(xgb_model, dtest)
xgb_pred <- factor(xgb_pred, levels = 0:(length(levels(y_train))-1), labels = levels(y_train))

# 计算准确率
xgb_acc <- mean(xgb_pred == y_test)
cat("XGBoost 模型测试集准确率:", xgb_acc, "\n")

# 4. 随机森林模型
# 训练随机森林模型
rf_model <- randomForest(
  x = X_train_scaled,
  y = y_train,
  ntree = 200,
```

```
  mtry = sqrt(ncol(X_train_scaled)),
  importance = TRUE,
  proximity = TRUE
)

# 模型训练过程可视化（误差随树数量的变化）
plot(rf_model, main = "随机森林误差随树数量的变化")
legend("topright", colnames(rf_model$err.rate), col = 1:3, fill = 1:3)
dev.copy(png, "rf_error_by_trees.png", width = 800, height = 600)
dev.off()

# 在测试集上预测
rf_pred <- predict(rf_model, X_test_scaled)

# 计算准确率
rf_acc <- mean(rf_pred == y_test)
cat("随机森林模型测试集准确率:", rf_acc, "\n")

# 5. 模型性能评估与比较
# 存储所有模型的预测结果
all_preds <- data.frame(
  actual = y_test,
  svm = svm_pred,
  xgb = xgb_pred,
  rf = rf_pred
)

# 计算各模型的评估指标
calculate_metrics <- function(actual, pred) {
  cm <- confusionMatrix(pred, actual)
  data.frame(
    accuracy = cm$overall["Accuracy"],
    precision = mean(cm$byClass[, "Precision"], na.rm = TRUE),
    recall = mean(cm$byClass[, "Recall"], na.rm = TRUE),
    f1 = mean(cm$byClass[, "F1"], na.rm = TRUE)
  )
}

# 计算每个模型的评估指标
metrics <- rbind(
  calculate_metrics(all_preds$actual, all_preds$svm) %>% mutate(model = "SVM"),
  calculate_metrics(all_preds$actual, all_preds$xgb) %>% mutate(model = "XGBoost"),
  calculate_metrics(all_preds$actual, all_preds$rf) %>% mutate(model = "随机森林")
)

# 重新排列列顺序
metrics <- metrics %>% select(model, accuracy, precision, recall, f1)
print(metrics)

# 可视化各模型性能指标
metrics_long <- metrics %>%
  pivot_longer(cols = -model, names_to = "metric", values_to = "value") %>%
  mutate(metric = factor(metric, levels = c("accuracy", "precision", "recall", "f1"),
                                  labels = c("准确率", "精确率", "召回率", "F1 分数")))

ggplot(metrics_long, aes(x = model, y = value, fill = model)) +
  geom_bar(stat = "identity", position = "dodge") +
  facet_wrap(~metric) +
  scale_fill_viridis(discrete = TRUE) +
  labs(title = "不同模型的性能指标比较", x = "模型", y = "分数") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 14),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
ggsave("model_metrics_comparison.png", width = 12, height = 8, dpi = 300)

# 6. 混淆矩阵可视化
plot_confusion_matrix <- function(actual, pred, model_name) {
  cm <- confusionMatrix(pred, actual)
  cm_df <- as.data.frame(cm$table)

  ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
```

```
    geom_tile(color = "white") +
    geom_text(aes(label = Freq), vjust = 0.5, size = 4) +
    scale_fill_viridis(option = "plasma") +
    labs(title = paste(model_name, "模型混淆矩阵"), x = "实际类别", y = "预测类别") +
    theme_minimal() +
    theme(
      plot.title = element_text(hjust = 0.5, size = 14),
      axis.text.x = element_text(angle = 45, hjust = 1)
    )
}

# 绘制三个模型的混淆矩阵
p_svm <- plot_confusion_matrix(all_preds$actual, all_preds$svm, "SVM")
p_xgb <- plot_confusion_matrix(all_preds$actual, all_preds$xgb, "XGBoost")
p_rf <- plot_confusion_matrix(all_preds$actual, all_preds$rf, "随机森林")

# 组合混淆矩阵图
(p_svm + p_xgb + p_rf) + plot_annotation(title = "各模型混淆矩阵比较")
ggsave("confusion_matrices.png", width = 18, height = 6, dpi = 300)

# 7. 特征重要性可视化
# 提取 XGBoost 特征重要性
xgb_imp <- xgb.importance(feature_names = colnames(X_train_scaled), model = xgb_model)
xgb_imp_df <- as.data.frame(xgb_imp) %>%
  mutate(Feature = factor(Feature, levels = rev(Feature))) %>%
  head(15)  # 取前 15 个重要特征

# 提取随机森林特征重要性
rf_imp <- as.data.frame(importance(rf_model)) %>%
  mutate(Feature = rownames(.)) %>%
  rename(Importance = MeanDecreaseGini) %>%
  arrange(desc(Importance)) %>%
  head(15) %>%  # 取前 15 个重要特征
  mutate(Feature = factor(Feature, levels = rev(Feature)))

# 绘制 XGBoost 特征重要性
p_xgb_imp <- ggplot(xgb_imp_df, aes(x = Feature, y = Gain)) +
  geom_bar(stat = "identity", fill = "#4285f4") +
  coord_flip() +
  labs(title = "XGBoost 模型特征重要性", x = "特征", y = "增益") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

# 绘制随机森林特征重要性
p_rf_imp <- ggplot(rf_imp, aes(x = Feature, y = Importance)) +
  geom_bar(stat = "identity", fill = "#34a853") +
  coord_flip() +
  labs(title = "随机森林模型特征重要性", x = "特征", y = "不纯度减少量") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

# 组合特征重要性图
grid.arrange(p_xgb_imp, p_rf_imp, ncol = 2)
ggsave("feature_importance.png", width = 14, height = 8, dpi = 300)

# 输出最终评估结果
cat("\n 模型性能总结:\n")
print(metrics)
```